

PHÂN PHỐI HỆ THỐNG



MAARTEN VAN STEEN
ANDREW S. TANENBAUM

ẤN BẢN THỨ 4

01
PHIÊN BẢN

Bìa nghệ thuật của Max van Steen

HỆ THỐNG PHÂN TÁN

Phiên bản thứ tư

Phiên bản 4.01

(Tháng 1 năm 2023)

Maarten van Steen
Andrew S. Tanenbaum

Bản quyền © 2023 Maarten van Steen và Andrew S. Tanenbaum Xuất bản bởi Maarten van Steen Phiên bản đầu tiên và thứ hai của cuốn sách này trừ ớc đây đã đợc Pearson Education, Inc. xuất bản.

ISBN: 978-90-815406-3-6 (bản in)

ISBN: 978-90-815406-4-3 (phiên bản kỹ thuật số)

Phiên bản: 4. Phiên bản: 01 (tháng 1 năm 2023)

Maarten van Steen và Andrew S. Tanenbaum giữ mọi quyền đối với văn bản và hình minh họa. Không đợc sao chép, tái bản hoặc dịch toàn bộ hoặc một phần tác phẩm này mà không có sự cho phép bằng văn bản của nhà xuất bản, ngoại trừ các trích đoạn ngắn trong bài đánh giá hoặc phân tích học thuật. Nghiêm cấm sử dụng với bất kỳ hình thức lưu trữ và truy xuất thông tin nào, chuyển thể điện tử hoặc bất kỳ hình thức nào, phần mềm máy tính hoặc bằng các phương pháp tự động hoặc khác nhau hiện đợc biết đến hoặc phát triển trong tương lai mà không có sự cho phép bằng văn bản của nhà xuất bản.

Gửi Mariëlle, Max và Elke

- MvS

Gửi Suzanne, Barbara, Marvin, Aron, Nathan, Olivia và Mirte

- AST

MỤC LỤC

Lời nói đầu	xi
1 Giới thiệu	1
1.1 Từ hệ thống mạng đến hệ thống phân tán	3
1.1.1 Hệ thống phân tán so với hệ thống phi tập trung	3
1.1.2 Tại sao việc phân biệt lại có liên quan	7
1.1.3 Nghiên cứu hệ thống phân tán.	8
1.2 Mục tiêu thiết kế	10
1.2.1 Chia sẻ tài nguyên	10
1.2.2 Tính minh bạch trong phân phối.	11
1.2.3 Sự cởi mở	15
1.2.4 Độ tin cậy	18
1.2.5 Bảo mật	21
1.2.6 Khả năng mở rộng	24
1.3 Phân loại đơn giản các hệ thống phân tán.	32
1.3.1 Điện toán phân tán hiệu suất cao	32
1.3.2 Hệ thống thông tin phân tán.	37
1.3.3 Hệ thống lan tỏa	43
1.4 Những cạm bẫy	52
1.5 Tóm tắt	53
2 Kiến trúc	55
2.1 Phong cách kiến trúc	56
2.1.1 Kiến trúc phân lớp	57
2.1.2 Kiến trúc hướng dịch vụ.	62
2.1.3 Kiến trúc xuất bản-đăng ký.	68
2.2 Phần mềm trung gian và hệ thống phân tán.	73
2.2.1 Tổ chức phần mềm trung gian.	74
2.2.2 Phần mềm trung gian có thể sửa đổi.	78
2.3 Kiến trúc hệ thống phân lớp.	78
2.3.1 Kiến trúc máy khách-máy chủ đơn giản.	79
2.3.2 Kiến trúc đa tầng.	80
2.3.3 Ví dụ: Hệ thống tập tin mạng.	83
2.3.4 Ví dụ: Web	85
2.4 Kiến trúc hệ thống phân tán đối xứng	88
2.4.1 Hệ thống ngang hàng có cấu trúc.	90
2.4.2 Hệ thống ngang hàng không có cấu trúc.	92

2.4.3 Mạng ngang hàng được tổ chức theo hệ thống phân cấp.	95
2.4.4 Ví dụ: BitTorrent	96
2.5 Kiến trúc hệ thống lai.	98
2.5.1 Điện toán đám mây	98
2.5.2 Kiến trúc đám mây biên	100
2.5.3 Kiến trúc chuỗi khối	104
2.6 Tóm tắt	108
3 Tiến trình 3.1	111
Luồng	112
3.1.1 Giới thiệu về luồng	113
3.1.2 Luồng trong hệ thống phân tán	122
3.2 Ảo hóa	127
3.2.1 Nguyên lý ảo hóa	127
3.2.2 Các thùng chứa	133
3.2.3 So sánh máy ảo và container	138
3.2.4 Ứng dụng máy ảo vào hệ thống phân tán	139
3.3 Khách hàng	141
3.3.1 Giao diện người dùng được kết nối mạng	141
3.3.2 Môi trường máy tính để bàn ảo	144
3.3.3 Phần mềm phía máy khách để phân phối minh bạch.	148
3.4 Máy chủ	149
3.4.1 Các vấn đề thiết kế chung.	149
3.4.2 Máy chủ đối tượng.	154
3.4.3 Ví dụ: Máy chủ web Apache.	159
3.4.4 Cụm máy chủ.	161
3.5 Di chuyển mã.	167
3.5.1 Lý do di chuyển mã	167
3.5.2 Các mô hình di chuyển mã	171
3.5.3 Di cư trong các hệ thống không đồng nhất.	174
3.6 Tóm tắt	177
4 Giao tiếp	181
4.1 Nền tảng	183
4.1.1 Giao thức phân lớp	183
4.1.2 Các loại hình giao tiếp	190
4.2 Gọi thủ tục từ xa.	192
4.2.1 Hoạt động RPC cơ bản.	192
4.2.2 Truyền tham số.	197
4.2.3 Hỗ trợ ứng dụng dựa trên RPC	201
4.2.4 Các biến thể của RPC.	205
4.3 Truyền thông theo hướng thông điệp	208
4.3.1 Nhắn tin tạm thời đơn giản với socket.	208
4.3.2 Nhắn tin tạm thời nâng cao	213

MỤC LỤC	vii
4.3.3 Truyền thông liên tục theo hướng tin nhắn	220
4.3.4 Ví dụ: Giao thức xếp hàng tin nhắn nâng cao (AMQP)	227
4.4 Truyền thông đa hướng	232
4.4.1 Đa hướng dựa trên cây cấp ứng dụng	232
4.4.2 Phát đa hướng dựa trên ngập lụt.	236
4.4.3 Phát tán dữ liệu dựa trên tin đồn.	240
4.5 Tóm tắt	245
5 Phối hợp	247
5.1 Đồng bộ hóa đồng hồ	249
5.1.1 Đồng hồ vật lý	250
5.1.2 Thuật toán đồng bộ hóa đồng hồ.	253
5.2 Đồng hồ logic.	260
5.2.1 Đồng hồ logic của Lamport.	260
5.2.2 Đồng hồ vectơ	266
5.3 Loại trừ lẫn nhau	272
5.3.1 Tổng quan	272
5.3.2 Thuật toán tập trung.	273
5.3.3 Một thuật toán phân tán.	274
5.3.4 Thuật toán vòng mã thông báo.	276
5.3.5 Một thuật toán phi tập trung.	277
5.3.6 Ví dụ: Khóa đơn giản với ZooKeeper	280
5.4 Thuật toán bầu cử	283
5.4.1 Thuật toán bất nạt.	283
5.4.2 Thuật toán vòng	285
5.4.3 Ví dụ: Bầu chọn người lãnh đạo trong ZooKeeper	286
5.4.4 Ví dụ: Bầu cử thủ lĩnh trong Raft.	289
5.4.5 Bầu cử trong các hệ thống quy mô lớn.	290
5.4.6 Bầu cử trong môi trường không dây.	294
5.5 Phối hợp dựa trên tin đồn	297
5.5.1 Tổng hợp	297
5.5.2 Dịch vụ lấy mẫu ngang hàng.	298
5.5.3 Xây dựng lớp phủ dựa trên tin đồn.	299
5.5.4 Buồn chuyện an toàn	303
5.6 So khớp sự kiện phân tán.	306
5.6.1 Triển khai tập trung.	307
5.6.2 Giải pháp đăng ký-xuất bản an toàn.	313
5.7 Hệ thống định vị	315
5.7.1 GPS: Hệ thống định vị toàn cầu.	315
5.7.2 Khi GPS không phải là một lựa chọn.	317
5.7.3 Vị trí logic của các nút.	318
5.8 Tóm tắt	322
6 Đặt tên	325

6.1 Tên, mã định danh và địa chỉ	326
6.2 Đặt tên phẳng.	329
6.2.1 Giải pháp đơn giản	329
6.2.2 Các phương pháp tiếp cận tại nhà	331
6.2.3 Bảng băm phân tán	333
6.2.4 Phương pháp tiếp cận theo thứ bậc	338
6.2.5 Đặt tên phẳng an toàn.	343
6.3 Đặt tên có cấu trúc.	344
6.3.1 Không gian tên	344
6.3.2 Giải quyết tên.	347
6.3.3 Việc triển khai không gian tên.	352
6.3.4 Ví dụ: Hệ thống tên miền.	359
6.3.5 Ví dụ: Hệ thống tập tin mạng.	369
6.4 Đặt tên dựa trên thuộc tính.	375
6.4.1 Dịch vụ thư mục	375
6.4.2 Triển khai theo phân cấp: LDAP 6.4.3	376
Triển khai phi tập trung	380
6.5 Mạng dữ liệu được đặt tên.	385
6.5.1 Cơ bản	385
6.5.2 Định tuyến	387
6.5.3 Bảo mật trong mạng dữ liệu được đặt tên.	388
6.6 Tóm tắt	389
7 Sự nhất quán và sao chép	391
thiếu	392
7.1.1 Lý do sao chép	393
7.1.2 Sao chép như một kỹ thuật mở rộng quy mô.	394
7.2 Mô hình nhất quán lấy dữ liệu làm trung tâm	395
7.2.1 Thứ tự các hoạt động nhất quán.	396
7.2.2 Sự nhất quán cuối cùng	406
7.2.3 Sự nhất quán liên tục	410
7.3 Mô hình nhất quán lấy khách hàng làm trung tâm	415
7.3.1 Đọc đơn điệu	417
7.3.2 Viết đơn điệu	418
7.3.3 Đọc bài viết của bạn.	420
7.3.4 Ghi theo sau đọc.	421
7.3.5 Ví dụ: tính nhất quán lấy khách hàng làm trung tâm trong ZooKeeper.	422
7.4 Quản lý bản sao	423
7.4.1 Tìm vị trí máy chủ tốt nhất	424
7.4.2 Sao chép và sắp xếp nội dung.	426
7.4.3 Phân phối nội dung	430
7.4.4 Quản lý các đối tượng được sao chép	434
7.5 Giao thức nhất quán.	437
7.5.1 Tính nhất quán tuần tự: Giao thức dựa trên chính.	438

Phiên bản 4.01

9.3	Xác thực	571
9.3.1	Giới thiệu về xác thực	571
9.3.2	Giao thức xác thực	572
9.4	Tin tư ởng vào hệ thống phân tán.	585
9.4.1	Tin tư ởng trư ớc những thất bại đau đớn.	586
9.4.2	Tin tư ởng vào danh tính	586
9.4.3	Tin tư ởng vào hệ thống	591
9.5	Ủy quyền	593
9.5.1	Các vấn đề chung về kiểm soát truy cập	593
9.5.2	Kiểm soát truy cập dựa trên thuộc tính	598
9.5.3	Ủy quyền.	601
9.5.4	Ủy quyền phi tập trung: một ví dụ	605
9.6	Giám sát	609
9.6.1	Tư ởng lửa	609
9.6.2	Phát hiện xâm nhập: những điều cơ bản	611
9.6.3	Phát hiện xâm nhập cộng tác	612
9.7	Tóm tắt	613
Mục lục		615
Tài liệu tham khảo		631
Thuật ngữ		665

LỜI NÓI ĐẦU

Đây là phiên bản thứ tư của “Hệ thống phân tán”. Chúng tôi đã bám sát thiết lập của phiên bản thứ ba, bao gồm các ví dụ về (một phần) các hệ thống phân tán hiện có gần nơi thảo luận về các nguyên tắc chung. Ví dụ, chúng tôi đã đưa tài liệu về hệ thống blockchain và thảo luận về các thành phần khác nhau của chúng trong suốt cuốn sách. Một lần nữa, chúng tôi đã sử dụng các phần đóng hộp đặc biệt cho tài liệu có thể bỏ qua khi đọc lần đầu.

Văn bản đã được xem xét, sửa đổi và cập nhật kỹ lưỡng. Đặc biệt, tất cả mã Python đã được cập nhật lên Python3, trong khi gói kênh đã được sửa đổi và đơn giản hóa gần như hoàn toàn. Các ví dụ mã hóa trong sách bỏ qua nhiều chi tiết để dễ đọc, nhưng các ví dụ đầy đủ có sẵn thông qua Trang web của sách, được lưu trữ tại www.distributed-systems.net. Chúng tôi đã đảm bảo rằng hầu như tất cả các ví dụ đều có thể được thực hiện ngay lập tức thông qua một tập lệnh đơn giản. Tuy nhiên, sẽ cần phải tải xuống và cài đặt các gói đặc biệt, chẳng hạn như Redis.

Như trước đây, Trang web cũng chứa các slide ở định dạng PDF và PPT, cũng như các nguồn để tạo slide bằng LATEX với lớp Beamer. Tất cả các hình ảnh, hiện bao gồm cả các hình ảnh cho bảng và ví dụ mã hóa, đều có sẵn ở định dạng PDF và Định dạng PNG.

Giống như phiên bản trước, cuốn sách có thể được tải xuống (miễn phí), giúp bạn dễ dàng sử dụng siêu liên kết khi cần thiết. Đồng thời, chúng tôi cung cấp phiên bản in thông qua Amazon.com, có sẵn với chi phí tối thiểu.

Vì cuốn sách được số hóa hoàn toàn nên chúng tôi có thể cập nhật khi cần. Chúng tôi dự định chạy các bản cập nhật hàng năm, trong khi vẫn giữ các phiên bản trước đó ở dạng kỹ thuật số cũng như phiên bản in gốc. Việc chạy các bản cập nhật thường xuyên không phải lúc nào cũng là điều đúng đắn khi xét về góc độ giảng dạy, nhưng việc cập nhật hàng năm và duy trì các phiên bản trước đó có vẻ là một sự thỏa hiệp tốt. Bản cập nhật thường bao gồm các bản sửa lỗi nhỏ, loại thường xuất hiện trong danh sách lỗi chính tả. Bên cạnh đó, cuốn sách hiện cũng có phần mục lục, cũng như phần chú giải thuật ngữ. Đây thường là những phần thường mất rất nhiều thời gian để biên soạn, nhưng cũng là những phần thường phát triển khi sử dụng sách. Tư ơn tự như vậy, chúng tôi có thể tìm cách cải thiện cách kết hợp siêu liên kết. Những vấn đề như vậy không ảnh hưởng đến văn bản chính, trong khi có khả năng cải thiện khả năng sử dụng của phiên bản kỹ thuật số.

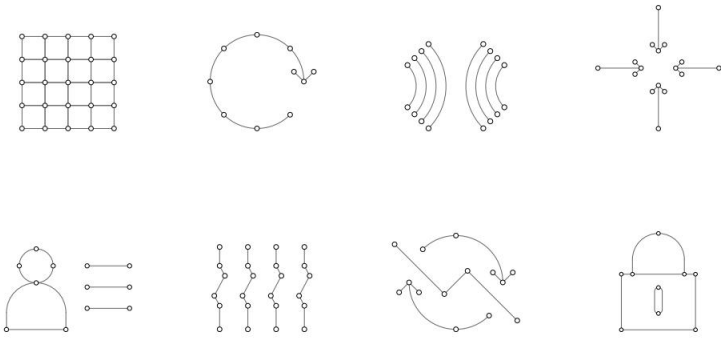
Lời cảm Ơn

Nhiều giáo viên và học sinh đã giúp phát hiện ra những lỗi không thể tránh khỏi trong ấn bản thứ ba, và chúng tôi nợ họ rất nhiều lời cảm Ơn, như ng trong mọi trường hợp, xin cảm Ơn Michael May, Linh Phan, Juan Abadie và Christian Zircpins. Đối với ấn bản thứ tư, một số đồng nghiệp đã rất tử tế khi xem lại một số phần của tài liệu. Đặc biệt, chúng tôi muốn cảm Ơn Armin Stocker, Hermann de Meer, Pim Otte, Johan Pouwelse, Michael P. Anderson, Ivo Varenhorst, Aditya Pappu và Alexander Iosup. Xin gửi lời cảm Ơn đặc biệt đến Hein Meling của Đại học Stavanger. Ông ấy không chỉ giúp chúng tôi rất nhiều trong việc hiểu cách Paxos hoạt động (đã có trong phiên bản thứ ba), chúng tôi còn biết Ơn khi áp dụng và điều chỉnh tệp định dạng LATEX của ông ấy để tạo biểu đồ trình tự tin nhắn. Những con số này hiện đã nhất quán hơn nhiều. Chúng tôi cảm Ơn Max van Steen đã thiết kế bìa sách.

Maarten van Steen

Andrew S. Tanenbaum

GIỚI THIỆU



Tốc độ thay đổi của hệ thống máy tính đã, đang và vẫn đang rất nhanh. Từ năm 1945, khi kỷ nguyên máy tính hiện đại bắt đầu, cho đến khoảng năm 1985, máy tính rất lớn và đắt tiền. Hơn nữa, do không có cách nào để kết nối chúng, những máy tính này hoạt động độc lập với nhau.

Tuy nhiên, bắt đầu từ giữa những năm 1980, hai tiến bộ trong công nghệ đã bắt đầu thay đổi tình hình đó. Đầu tiên là sự phát triển của các bộ vi xử lý mạnh mẽ. Ban đầu, đây là những máy 8 bit, nhưng chẳng mấy chốc, CPU 16, 32 và 64 bit đã trở nên phổ biến. Với các CPU đa lõi mạnh mẽ, giờ đây chúng ta lại phải đối mặt với thách thức là phải thích ứng và phát triển các chương trình để khai thác tính song song. Trong mọi trường hợp, thế hệ máy hiện tại có sức mạnh tính toán của các máy chủ lớn được triển khai cách đây 30 hoặc 40 năm, nhưng với giá chỉ bằng 1/1000 hoặc thấp hơn.

Sự phát triển thứ hai là phát minh ra mạng máy tính tốc độ cao. Mạng cục bộ hoặc LAN cho phép hàng ngàn máy trong một tòa nhà được kết nối theo cách mà lưu lượng thông tin nhỏ có thể được truyền trong vài micro giây hoặc lâu hơn. Lưu lượng dữ liệu lớn hơn có thể được di chuyển giữa các máy với tốc độ hàng tỷ bit mỗi giây (bps). Mạng diện rộng hoặc WAN cho phép hàng trăm triệu máy trên khắp thế giới được kết nối với tốc độ thay đổi từ hàng chục nghìn đến hàng trăm triệu bps trở lên.

Song song với sự phát triển của các máy móc ngày càng mạnh mẽ và được kết nối mạng, chúng ta cũng có thể chứng kiến sự thu nhỏ của các hệ thống máy tính, có lẽ điện thoại thông minh là kết quả ấn tượng nhất. Được trang bị cảm biến, nhiều bộ nhớ và CPU đa lõi mạnh mẽ, những thiết bị này không gì khác hơn là máy tính hoàn chỉnh. Tất nhiên, chúng cũng có khả năng kết nối mạng. Tương tự như vậy, cái gọi là máy tính nano đã trở nên dễ dàng có sẵn. Những máy tính bằng nhỏ, đơn giản, thường có kích thước bằng một thẻ tín dụng, có thể dễ dàng cung cấp hiệu suất gần như máy tính để bàn. Các ví dụ nổi tiếng bao gồm hệ thống Raspberry Pi và Arduino.

Và câu chuyện vẫn tiếp tục. Khi quá trình số hóa xã hội của chúng ta tiếp tục, chúng ta ngày càng nhận thức được có bao nhiêu máy tính thực sự đang được sử dụng, thường xuyên được nhúng vào các hệ thống khác như ô tô, máy bay, tòa nhà, cầu, lưới điện, v.v. Thật không may, nhận thức này tăng lên khi các hệ thống như vậy đột nhiên trở nên có thể bị hack. Ví dụ, vào năm 2021, một đường ống dẫn nhiên liệu ở Hoa Kỳ đã bị đóng cửa hiệu quả do một cuộc tấn công bằng phần mềm tống tiền. Trong trường hợp này, hệ thống máy tính bao gồm sự kết hợp của các cảm biến, bộ truyền động, bộ điều khiển, máy tính nhúng, máy chủ, v.v., tất cả được kết hợp lại thành một hệ thống duy nhất. Điều mà nhiều người trong chúng ta không nhận ra là các cơ sở hạ tầng quan trọng, chẳng hạn như đường ống dẫn nhiên liệu, được giám sát và điều khiển bởi các hệ thống máy tính được kết nối mạng. Tương tự như vậy, có lẽ đã đến lúc bắt đầu nhận ra rằng một chiếc ô tô hiện đại thực sự là một máy tính di động được kết nối mạng, hoạt động tự động. Trong trường hợp này, thay vì máy tính di động được một người mang theo, chúng ta cần phải xử lý máy tính di động mang theo người đi.

Kích thước của một hệ thống máy tính được kết nối mạng có thể thay đổi từ một số ít thiết bị đến hàng triệu máy tính. Mạng kết nối có thể là có dây, không dây hoặc kết hợp cả hai. Hơn nữa, các hệ thống này thường rất năng động, theo nghĩa là máy tính có thể tham gia và rời đi, với cấu trúc mạng và hiệu suất của mạng cơ bản gần như liên tục thay đổi.

Thật khó để nghĩ đến các hệ thống máy tính không được kết nối mạng. Và thực tế là, hầu hết các hệ thống máy tính được kết nối mạng có thể được truy cập từ bất kỳ nơi nào trên thế giới vì chúng được kết nối với Internet. Việc nghiên cứu để hiểu các hệ thống này có thể dễ dàng trở nên cực kỳ phức tạp. Trong chương này, chúng ta bắt đầu bằng cách làm sáng tỏ những gì cần hiểu để xây dựng bức tranh toàn cảnh mà không bị lạc lối.

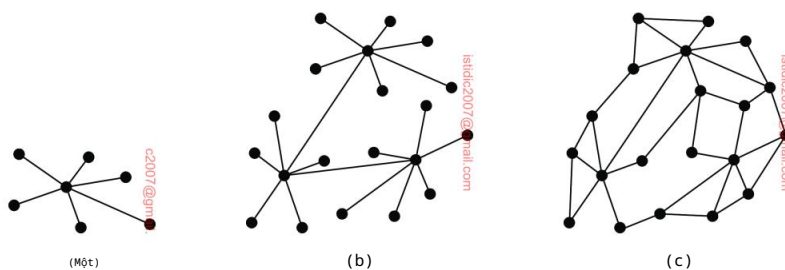
1.1 Từ hệ thống mạng đến hệ thống phân tán

Trước khi đi sâu vào các khía cạnh khác nhau của hệ thống phân tán, trước tiên chúng ta hãy xem xét phân phối hoặc phi tập trung thực sự bao gồm những gì.

1.1.1 Hệ thống phân tán so với hệ thống phi tập trung

Khi xem xét các nguồn khác nhau, có khá nhiều ý kiến về hệ thống phân tán so với hệ thống phi tập trung. Thông thường, sự khác biệt được minh họa bằng ba tổ chức khác nhau của hệ thống máy tính được kết nối mạng, như thể hiện trong Hình 1.1, trong đó mỗi nút đại diện cho một hệ thống máy tính và một cạnh đại diện cho một liên kết truyền thông giữa hai nút.

Mức độ hữu ích của những sự phân biệt như vậy vẫn còn phải xem xét, đặc biệt là khi thảo luận về ưu và nhược điểm của từng tổ chức. Ví dụ, người ta thường nói rằng các tổ chức tập trung không mở rộng quy mô tốt. Tuy nhiên như vậy, các tổ chức phân tán được cho là có khả năng chống lại thất bại tốt hơn. Như chúng ta sẽ thấy, không có tuyên bố nào trong số này là đúng cả.



Hình 1.1: Tổ chức của một hệ thống (a) tập trung, (b) phi tập trung và (c) phân tán, theo nhiều nguồn phổ biến. Chúng tôi áp dụng một cách tiếp cận khác, vì những con số như thế này thực sự không có ý nghĩa gì.

Chúng tôi có cách tiếp cận khác. Nếu chúng ta nghĩ về một hệ thống máy tính được kết nối mạng như một tập hợp các máy tính được kết nối trong một mạng, chúng ta có thể tự hỏi làm thế nào những máy tính này thậm chí được kết nối với nhau ngay từ đầu. Có khoảng hai quan điểm mà người ta có thể đưa ra.

Quan điểm đầu tiên, quan điểm tích hợp, là cần phải kết nối các hệ thống máy tính hiện có (mạng) với nhau. Thông thường, điều này xảy ra khi các dịch vụ chạy trên một hệ thống cần được cung cấp cho người dùng và các ứng dụng mà trước đây chưa từng nghĩ đến. Điều này có thể xảy ra, ví dụ, khi tích hợp các dịch vụ tài chính với các dịch vụ quản lý dự án, như thường xảy ra trong một tổ chức duy nhất. Trong lĩnh vực nghiên cứu khoa học, chúng ta đã thấy những nỗ lực kết nối vô số tài nguyên thường tốn kém (máy tính chuyên dụng, siêu máy tính, hệ thống cơ sở dữ liệu rất lớn, v.v.) thành thứ được gọi là máy tính lưu trữ.

Quan điểm thứ hai, mở rộng là một hệ thống hiện có cần được mở rộng thông qua các máy tính bổ sung. Quan điểm này thường liên quan nhất đến lĩnh vực hệ thống phân tán. Nó đòi hỏi phải mở rộng một hệ thống bằng máy tính để giữ các tài nguyên gần nơi cần các tài nguyên đó. Việc mở rộng cũng có thể được thúc đẩy bởi nhu cầu cải thiện độ tin cậy: nếu một máy tính bị hỏng, thì sẽ có những máy tính khác có thể tiếp quản. Một loại mở rộng quan trọng là khi một dịch vụ cần được cung cấp cho người dùng và ứng dụng từ xa, ví dụ, bằng cách cung cấp giao diện Web hoặc ứng dụng điện thoại thông minh. Ví dụ cuối cùng này cũng cho thấy sự khác biệt giữa quan điểm tích hợp và quan điểm mở rộng không rõ ràng.

Trong cả hai trường hợp, chúng ta thấy rằng hệ thống mạng chạy các dịch vụ, trong đó mỗi dịch vụ được triển khai như một tập hợp các quy trình và tài nguyên trải rộng trên nhiều máy tính. Hai quan điểm dẫn đến sự phân biệt tự nhiên giữa hai loại hệ thống máy tính mạng:

- Hệ thống phi tập trung là hệ thống máy tính được kết nối mạng trong đó các quy trình và tài nguyên cần phải được phân bổ trên nhiều máy tính.
- Hệ thống phân tán là hệ thống máy tính được kết nối mạng trong đó các quy trình và tài nguyên được phân bổ đủ trên nhiều máy tính.

Trước khi thảo luận về lý do tại sao sự phân biệt này lại quan trọng, chúng ta hãy xem xét một số ví dụ về từng loại hệ thống.

Các hệ thống phi tập trung chủ yếu liên quan đến quan điểm tích hợp của các hệ thống máy tính được kết nối mạng. Chúng ra đời vì chúng ta muốn kết nối các hệ thống, nhưng có thể bị cản trở bởi các ranh giới hành chính. Ví dụ, nhiều ứng dụng trong lĩnh vực trí tuệ nhân tạo đòi hỏi lưu trữ dữ liệu khổng lồ để xây dựng các mô hình dự đoán đáng tin cậy. Thông thường, dữ liệu được đưa đến các máy tính hiệu suất cao, theo nghĩa đen là đào tạo các mô hình trước khi chúng có thể được sử dụng. Nhưng khi dữ liệu cần nằm trong phạm vi của một tổ chức (và có thể có nhiều lý do tại sao điều này là cần thiết), chúng ta cần đưa quá trình đào tạo vào dữ liệu. Kết quả được gọi là học liên bang,

Hai quan điểm Integrative view và Expansive view khác nhau về mặt cách tiếp cận vì chúng đề cập đến các hình thức máy tính như sau:

1. Integrative View (Quan điểm tích hợp):

Mục tiêu chính: Kết nối các hệ thống máy tính có (networked systems) chúng có thể chia sẻ tài nguyên hoặc cung cấp dịch vụ mới. Tình huống điển hình: Khi các hệ thống có các dịch vụ khác nhau (ví dụ: tích hợp dịch vụ tài chính và quản lý dự án trong một tổ chức). Trong lĩnh vực nghiên cứu khoa học, kết nối các tài nguyên từ nhiều siêu máy tính hoặc các siêu dữ liệu thành hình thức tính toán lưới (grid computing).

Cụm từ: Tập trung vào việc liên kết các hệ thống hiện có để tạo ra hình thức máy tính mới.

2. Expansive View (Quan điểm mở rộng):

Mục tiêu chính: Mở rộng hệ thống bằng cách thêm các tài nguyên mới. Tình huống điển hình: Khi cần thêm tài nguyên để xử lý công việc tăng cường hoặc để mở rộng quy mô của hệ thống. Làm dịch vụ khách hàng cho người dùng từ xa hoặc giao diện web hoặc ứng dụng di động.

Cụm từ: Tập trung vào việc mở rộng quy mô, tăng cường tính năng và tính khả dụng.

Khác biệt chính: Integrative view: Kết nối các hệ thống có sẵn.

Expansive view: Mở rộng hệ thống hiện có.

Tuy nhiên, trong thực tế, hai quan điểm này không hoàn toàn tách biệt vì nhiều trường hợp có thể đòi hỏi cả hai (ví dụ: làm dịch vụ khách hàng cho người dùng từ xa và tích hợp với các hệ thống khác).

Hệ thống phân tán (distributed systems) chủ yếu liên quan đến quan điểm mở rộng (expansive view) và minh bạch (transparency) của dịch vụ email như Google Mail. Đây là các ý chính:

Cách hoạt động góc nhìn người dùng

Người dùng truy cập email thông qua giao diện web hoặc ứng dụng email trên máy tính cá nhân (ví dụ: laptop).

Sử dụng người dùng trên các hình thức thông tin máy tính, như:

Máy chủ nhận thư: imap.gmail.com

Máy chủ gửi thư: smtp.gmail.com

Đúng như có hai máy chủ xử lý toàn bộ email của người dùng.

Thật ra, ngược lại:

Vấn đề người dùng (tính mở rộng năm 2022) và hơn 300 tỷ email mỗi năm (~10.000 email/giây), không thể có hai máy chủ xử lý tất cả.

Thay vào đó, Google Mail triển khai trên nhiều máy tính, tạo thành một hệ thống phân tán.

Hệ thống này:

Mở rộng khả năng mở rộng (scalability): Hệ thống phân tán người dùng và email.

Mở rộng khả năng chịu lỗi (fault tolerance): Giảm thiểu sự gián đoạn dịch vụ email do lỗi hệ thống. Tính minh bạch:

Người dùng không nhận ra sự phức tạp ngược lại, vì hệ thống phân tán "thực tế" vì tính phân phối tài nguyên (tính minh bạch).

Khả năng mở rộng linh hoạt:

Hệ thống có thể mở rộng hoặc thu nhỏ dựa trên nhu cầu vận hành (dependability) và số lượng người dùng.

Tóm lại, Google Mail là một ví dụ điển hình về cách hệ thống phân tán hoạt động trong môi trường duy trì tính minh bạch, trong khi vận hành hệ thống người dùng và minh bạch.

và được thực hiện bởi một hệ thống phi tập trung, trong đó nhu cầu phân bổ quy trình và tài nguyên được quyết định bởi các chính sách hành chính.

Một ví dụ khác về hệ thống phi tập trung là sổ cái phân tán, còn được gọi là blockchain. Trong trường hợp này, chúng ta cần giải quyết tình huống các bên tham gia không tin tưởng lẫn nhau đủ để thiết lập các chương trình đơn giản để hợp tác. Thay vào đó, về cơ bản, những gì họ làm là làm cho các giao dịch giữa nhau hoàn toàn công khai (và có thể xác minh được) bằng một sổ cái chỉ mở rộng lưu giữ hồ sơ về các giao dịch đó. Bản thân sổ cái được phân bổ hoàn toàn cho những người tham gia và những người tham gia là những người xác thực các giao dịch (của những người khác) trước khi chấp nhận chúng vào sổ cái. Kết quả là một hệ thống phi tập trung trong đó các quy trình và tài nguyên thực sự cần phải được phân bổ trên nhiều máy tính, trong trường hợp này là do thiếu sự tin tưởng.

Ví dụ cuối cùng về hệ thống phi tập trung, hãy xem xét các hệ thống phân tán theo địa lý tự nhiên. Điều này thường xảy ra với các hệ thống cần giám sát một vị trí thực tế, ví dụ như trong trường hợp nhà máy điện, tòa nhà, môi trường tự nhiên cụ thể, v.v. Hệ thống, kiểm soát các màn hình và nơi đưa ra quyết định, có thể dễ dàng được đặt ở nơi khác ngoài vị trí đang được giám sát. Một ví dụ rõ ràng là giám sát và kiểm soát vệ tinh, nhưng cũng có những tình huống trần tục hơn như giám sát và kiểm soát giao thông, tàu hỏa, v.v. Trong những ví dụ này, nhu cầu phân tán các quy trình và tài nguyên xuất phát từ một lập luận về không gian.

Như chúng tôi đã đề cập, các hệ thống phân tán chủ yếu liên quan đến góc nhìn mở rộng của các hệ thống máy tính được kết nối mạng. Một ví dụ nổi tiếng là sử dụng các dịch vụ email, chẳng hạn như Google Mail. Điều thường xảy ra là người dùng đăng nhập vào hệ thống thông qua giao diện Web để đọc và gửi email. Tuy nhiên, thường xuyên hơn là người dùng định cấu hình máy tính cá nhân của họ (chẳng hạn như máy tính xách tay) để sử dụng một ứng dụng email cụ thể. Để đạt được mục đích đó, họ cần định cấu hình một số cài đặt, chẳng hạn như máy chủ đến và máy chủ đi. Trong trường hợp của Google Mail, chúng lần lượt là imap.gmail.com và smtp.gmail.com. Về mặt logic, có vẻ như hai máy chủ này sẽ xử lý tất cả email của bạn. Tuy nhiên, với ước tính gần 2 tỷ người dùng tính đến năm 2022, không có khả năng chỉ có hai máy tính có thể xử lý tất cả email của họ (ước tính là hơn 300 tỷ mỗi năm, tức là khoảng 10.000 email mỗi giây). Tất nhiên, đằng sau hậu trường, toàn bộ dịch vụ Google Mail đã được triển khai và phân tán trên nhiều máy tính, cùng nhau tạo thành một hệ thống phân tán. Hệ thống đó đã được thiết lập để đảm bảo rằng rất nhiều người dùng có thể xử lý thư của họ (tức là đảm bảo khả năng mở rộng), nhưng cũng đảm bảo rằng rủi ro mất thư do lỗi là tối thiểu (tức là hệ thống đảm bảo khả năng chịu lỗi). Tuy nhiên, đối với người dùng, hình ảnh chỉ có hai máy chủ được duy trì (tức là bản thân quá trình phân phối rất minh bạch đối với người dùng). Hệ thống phân tán triển khai dịch vụ email, chẳng hạn như Google Mail, thường mở rộng (hoặc thu hẹp) tùy theo yêu cầu về độ tin cậy, đến lượt nó, phụ thuộc vào số lượng người dùng của nó.

Một loại hệ thống phân tán hoàn toàn khác được hình thành bằng cách tập hợp các cái gọi là Mạng phân phối nội dung, hay gọi tắt là CDN. Một ví dụ nổi tiếng là Akamai, vào năm 2022, có hơn 400.000 máy chủ trên toàn thế giới. Chúng ta sẽ thảo luận về nguyên tắc hoạt động của CDN sau trong Chương 3. Về cơ bản, nội dung của một Trang web thực tế được sao chép và phân tán trên nhiều máy chủ khác nhau của CDN. Khi truy cập một Trang web, người dùng được chuyển hướng một cách minh bạch đến một máy chủ gần đó lưu trữ toàn bộ hoặc một phần nội dung của Trang web đó. Việc lựa chọn máy chủ để chuyển hướng người dùng có thể phụ thuộc vào nhiều yếu tố, nhưng chắc chắn khi xử lý nội dung phát trực tuyến, người ta sẽ chọn máy chủ có thể đảm bảo hiệu suất tốt về độ trễ và băng thông. CDN đảm bảo rằng máy chủ được chọn sẽ có sẵn nội dung cần thiết, cũng như cập nhật nội dung đó khi cần hoặc xóa nội dung đó khỏi máy chủ khi không có hoặc có rất ít người dùng phục vụ tại đó. Trong khi đó, người dùng không biết gì về những gì đang diễn ra đằng sau hậu trường (một lần nữa, đây là một hình thức minh bạch phân phối). Chúng ta cũng thấy trong ví dụ này, nội dung không được sao chép vào tất cả các máy chủ, mà chỉ đến nơi có ý nghĩa, tức là đủ và vì lý do hiệu suất. CDN cũng sao chép nội dung vào nhiều máy chủ để cung cấp mức độ tin cậy cao.

Là một hệ thống phân tán cuối cùng, nhỏ hơn nhiều, hãy xem xét thiết lập dựa trên thiết bị lưu trữ gắn mạng, còn gọi là NAS. Đối với mục đích sử dụng trong nước, một NAS thông thường bao gồm 2-4 khe cắm cho ổ cứng bên trong. NAS hoạt động như một máy chủ tệp: có thể truy cập thông qua mạng (thường là không dây) cho bất kỳ thiết bị được ủy quyền nào và do đó có thể cung cấp các dịch vụ như lưu trữ dùng chung, sao lưu tự động, phông tiện truyền phát, v.v. Bản thân NAS có thể được coi là một máy tính duy nhất được tối ưu hóa để lưu trữ tệp và cung cấp khả năng dễ dàng chia sẻ các tệp đó. Điều sau rất quan trọng và cùng với nhiều người dùng, về cơ bản chúng ta có một thiết lập của một hệ thống phân tán. Người dùng sẽ làm việc với một tập hợp các tệp có thể truy cập cục bộ (tức là từ máy tính xách tay của họ) dễ dàng (trên thực tế, dường như được tích hợp vào hệ thống tệp cục bộ), đồng thời cũng có thể được người dùng khác truy cập trực tiếp. Một lần nữa, nơi và cách lưu trữ các tệp được chia sẻ được ẩn (tức là, việc phân phối là minh bạch). Giả sử rằng chia sẻ tệp là mục tiêu, thì chúng ta thấy rằng NAS thực sự có thể cung cấp đủ khả năng phân tán các quy trình và tài nguyên.

Lưu ý 1.1 (Thông tin thêm: Giải pháp tập trung có tệp không?)

Có vẻ như có một quan niệm sai lầm cố hữu rằng các giải pháp tập trung không thể mở rộng quy mô. Hơn nữa, chúng hầu như luôn đi kèm với việc đưa ra một điểm lỗi duy nhất. Cả hai lý do này thường được coi là đủ để bác bỏ các giải pháp tập trung là một lựa chọn tốt khi thiết kế các hệ thống phân tán.

Điều mà nhiều người quên là cần phải phân biệt giữa thiết kế logic và thiết kế vật lý. Một giải pháp tập trung logic có thể được triển khai theo cách phân tán có khả năng mở rộng cao. Một ví dụ tuyệt vời là Hệ thống tên miền (DNS), mà chúng ta sẽ thảo luận rộng rãi trong Chương 6. Về mặt logic, DNS là

được tổ chức như một cây lớn, trong đó mỗi đường dẫn từ gốc đến một nút lá biểu diễn một tên đủ điều kiện, chẳng hạn như `www.distributed-systems.net`. Sẽ là một sai lầm khi nghĩ rằng nút gốc được triển khai bởi chỉ một máy chủ duy nhất. Trên thực tế, nút gốc được triển khai bởi 13 máy chủ gốc khác nhau, mỗi máy chủ, đến lượt mình, được triển khai như một máy tính cụm lớn. Tổ chức vật lý của DNS cũng cho thấy rằng gốc không phải là điểm lỗi duy nhất. Được sao chép ở mức độ cao, sẽ cần rất nhiều nỗ lực để hạ gục gốc đó và cho đến nay, mọi nỗ lực để làm như vậy đều thất bại.

Các giải pháp tập trung không tệ chỉ vì chúng có vẻ tập trung.

Trên thực tế, như chúng ta sẽ gặp nhiều lần trong suốt cuốn sách này, (về mặt logic và thậm chí về mặt vật lý) các giải pháp tập trung thường tốt hơn nhiều so với các giải pháp phân tán vì lý do đơn giản là chỉ có một điểm lỗi duy nhất. Ví dụ, điều này giúp chúng dễ quản lý hơn nhiều và chắc chắn là khi so sánh với những nơi có thể có nhiều điểm lỗi. Hơn nữa, điểm lỗi duy nhất đó có thể được củng cố để chống lại nhiều loại lỗi cũng như nhiều loại tấn công bảo mật. Khi nói đến việc trở thành nút thắt cổ chai về hiệu suất, chúng ta cũng sẽ thấy rằng có nhiều cách có thể thực hiện để đảm bảo rằng ngay cả điều đó cũng không thể chống lại sự tập trung hóa.

Theo nghĩa này, chúng ta đừng quên rằng các giải pháp tập trung thậm chí đã chứng minh được khả năng mở rộng và mạnh mẽ cực kỳ. Chúng được gọi là các giải pháp dựa trên đám mây. Một lần nữa, việc triển khai chúng có thể sử dụng các giải pháp phân tán rất tinh vi, nhưng ngay cả khi đó, chúng ta sẽ thấy rằng ngay cả những giải pháp đó đôi khi cũng cần dựa vào một nhóm nhỏ các máy vật lý, nếu chỉ để đảm bảo hiệu suất.

1.1.2 Tại sao việc phân biệt lại có liên quan

Tại sao chúng ta lại phân biệt giữa hệ thống phi tập trung và hệ thống phân tán? Điều quan trọng là phải nhận ra rằng các giải pháp tập trung thường đơn giản hơn nhiều, và cũng đơn giản hơn theo các tiêu chí khác nhau. Phi tập trung, tức là hành động phân tán việc triển khai một dịch vụ trên nhiều máy tính vì chúng ta tin rằng điều đó là cần thiết, là một quyết định cần được cân nhắc cẩn thận. Thật vậy, các giải pháp phân tán và phi tập trung vốn khó khăn:

- Có nhiều sự phụ thuộc, thường là không mong muốn, cản trở việc hiểu hành vi của các hệ thống này.
- Các hệ thống phân tán và phi tập trung gần như liên tục gặp phải lỗi cục bộ: một số quy trình hoặc tài nguyên, ở đâu đó trên một trong các máy tính tham gia, không hoạt động theo mong đợi. Việc phát hiện ra lỗi thực sự có thể mất một thời gian, trong khi những lỗi như vậy tốt nhất nên được che giấu (tức là chúng không gây chú ý cho người dùng và ứng dụng), bao gồm cả quá trình phục hồi sau lỗi.

- Liên quan nhiều đến các lỗi cục bộ là thực tế rằng trong nhiều hệ thống máy tính được kết nối mạng, các nút tham gia, quy trình, tài nguyên, v.v., xuất hiện và biến mất. Điều này làm cho các hệ thống này có tính năng động cao, đến lượt nó đòi hỏi các hình thức quản lý và bảo trì tự động, đến lượt nó làm tăng tính phức tạp.
 - Thực tế là các hệ thống phân tán và phi tập trung được kết nối mạng, được nhiều người sử dụng và ứng dụng sử dụng và thường vượt qua nhiều ranh giới quản trị, khiến chúng đặc biệt dễ bị tấn công bảo mật.
- Do đó, để hiểu được các hệ thống này và hành vi của chúng, chúng ta cần phải hiểu cách chúng có thể và được bảo mật. Thật không may, việc hiểu được bảo mật không dễ dàng như vậy.

Sự khác biệt của chúng tôi là giữa tính đủ và tính cần thiết để phân tán các quy trình và tài nguyên trên nhiều máy tính. Trong suốt cuốn sách này, chúng tôi có quan điểm rằng phi tập trung không bao giờ có thể là mục tiêu tự thân, và rằng nó nên tập trung vào tính đủ để phân tán các quy trình và tài nguyên trên nhiều máy tính. Về nguyên tắc, càng ít phân tán thì càng tốt. Tuy nhiên, đồng thời, chúng ta cần nhận ra rằng đôi khi việc phân tán thực sự cần thiết, như được minh họa bằng các ví dụ về hệ thống phi tập trung. Từ quan điểm đủ này, cuốn sách thực sự nói về các hệ thống phân tán và khi thích hợp, chúng ta sẽ nói về các hệ thống phi tập trung.

Tương tự như vậy, xét đến việc các hệ thống phân tán và phi tập trung vốn phức tạp, thì việc xem xét các giải pháp càng đơn giản càng tốt cũng quan trọng không kém. Do đó, chúng ta sẽ không thảo luận về việc tối ưu hóa các giải pháp, tin chắc rằng tác động của việc đóng góp tiêu cực của chúng vào việc tăng độ phức tạp lớn hơn tầm quan trọng của việc đóng góp tích cực của chúng vào việc tăng bất kỳ loại hiệu suất nào.

1.1.3 Nghiên cứu hệ thống phân tán

Xem xét rằng các hệ thống phân tán vốn đã khó, điều quan trọng là phải có cách tiếp cận có hệ thống để nghiên cứu chúng. Một trong những mối quan tâm chính của chúng tôi là có quá nhiều sự phụ thuộc rõ ràng và ngầm định trong các hệ thống phân tán. Ví dụ, không có thứ gọi là mô-đun giao tiếp riêng biệt hoặc mô-đun bảo mật riêng biệt. Cách tiếp cận của chúng tôi là xem xét các hệ thống phân tán từ một số lượng hạn chế như theo các góc nhìn khác nhau. Mỗi góc nhìn được xem xét trong một chương riêng biệt.

- Có nhiều cách tổ chức hệ thống phân tán. Chúng ta bắt đầu thảo luận bằng cách lấy góc nhìn kiến trúc: các tổ chức chung là gì, các phong cách chung là gì? Góc nhìn kiến trúc sẽ giúp nắm bắt đầu tiên về cách các thành phần khác nhau của các hệ thống hiện có tương tác và phụ thuộc vào nhau.

- Hệ thống phân tán là tất cả về các quy trình. Quan điểm quy trình là tất cả về việc hiểu các dạng quy trình khác nhau xảy ra trong hệ thống phân tán, có thể là luồng, quá trình ảo hóa các quy trình phân cứng, máy khách, máy chủ, v.v. Các quy trình tạo thành xương sống phần mềm của hệ thống phân tán và việc hiểu chúng là điều cần thiết để hiểu hệ thống phân tán.
- Rõ ràng, với nhiều máy tính đang bị đe dọa, giao tiếp giữa các quy trình là điều cần thiết. Quan điểm giao tiếp liên quan đến các tiện ích mà các hệ thống phân tán cung cấp để trao đổi dữ liệu giữa các quy trình. Về cơ bản, nó bao gồm việc mô phỏng các cuộc gọi thủ tục trên nhiều máy tính, truyền thông điệp cấp cao với nhiều tùy chọn ngữ nghĩa và nhiều loại giao tiếp khác nhau giữa các tập hợp quy trình.
- Để hệ thống phân tán hoạt động, những gì diễn ra bên trong các ứng dụng được thực thi là các quy trình phối hợp mọi thứ. Ví dụ, họ phối hợp với nhau để bù đắp cho việc thiếu đồng hồ toàn cầu, để thực hiện quyền truy cập độc quyền lẫn nhau vào các tài nguyên được chia sẻ, v.v. Quan điểm phối hợp mô tả một số nhiệm vụ phối hợp cơ bản cần được thực hiện như một phần của hầu hết các hệ thống phân tán.
- Để truy cập các quy trình và tài nguyên, chúng ta cần đặt tên. Cụ thể, chúng ta cần các lược đồ đặt tên, khi sử dụng, sẽ dẫn đến quy trình, tài nguyên hoặc bất kỳ loại thực thể nào khác đang được đặt tên. Mặc dù điều này có vẻ đơn giản, nhưng việc đặt tên không chỉ trở nên quan trọng trong các hệ thống phân tán mà còn có nhiều cách hỗ trợ việc đặt tên. Quan điểm đặt tên tập trung hoàn toàn vào việc giải quyết tên thành điểm truy cập của thực thể được đặt tên.
- Một khía cạnh quan trọng của các hệ thống phân tán là chúng hoạt động tốt về mặt hiệu quả và độ tin cậy. Công cụ chính cho cả hai khía cạnh này là sao chép tài nguyên. Vấn đề duy nhất với sao chép là các bản cập nhật có thể xảy ra, ngụ ý rằng tất cả các bản sao của một tài nguyên cũng cần được cập nhật. Ở đây, việc duy trì vẻ ngoài của một hệ thống không phân tán trở nên khó khăn. Quan điểm về tính nhất quán và sao chép về cơ bản tập trung vào sự đánh đổi giữa tính nhất quán, sao chép và hiệu suất.
- Chúng tôi đã đề cập rằng các hệ thống phân tán có thể bị lỗi một phần. Quan điểm về khả năng chịu lỗi đi sâu vào các phương tiện che giấu lỗi và phục hồi lỗi. Nó đã được chứng minh là một trong những quan điểm khó nhất để hiểu các hệ thống phân tán, chủ yếu là vì có quá nhiều sự đánh đổi cần thực hiện và cũng vì việc che giấu hoàn toàn lỗi và phục hồi lỗi là điều không thể chứng minh được.

- Như đã đề cập, không có hệ thống phân tán nào không được bảo mật. Quan điểm bảo mật tập trung vào cách đảm bảo quyền truy cập được ủy quyền vào tài nguyên. Để đạt được mục đích đó, chúng ta cần thảo luận về sự tin cậy trong các hệ thống phân tán, cùng với xác thực, cụ thể là xác minh danh tính đã khai báo. Quan điểm về an ninh được đưa ra cuối cùng, tuy nhiên sau trong chương này chúng ta sẽ thảo luận về một số công cụ cơ bản cần thiết để hiểu vai trò của an ninh trong các quan điểm trước đây.

1.2 Mục tiêu thiết kế

Chỉ vì có thể xây dựng các hệ thống phân tán không nhất thiết có nghĩa là đó là một ý tưởng hay. Trong phần này, chúng ta sẽ thảo luận về bốn mục tiêu quan trọng cần đạt được để việc xây dựng một hệ thống phân tán trở nên xứng đáng. Một hệ thống phân tán phải giúp các tài nguyên dễ dàng truy cập; phải ẩn đi sự thật rằng các tài nguyên được phân phối trên một mạng; phải mở; và phải có khả năng mở rộng.

1.2.1 Chia sẻ tài nguyên

Một mục tiêu quan trọng của hệ thống phân tán là giúp người dùng (và ứng dụng) dễ dàng truy cập và chia sẻ tài nguyên từ xa. Tài nguyên có thể là bất kỳ thứ gì, nhưng các ví dụ điển hình bao gồm thiết bị ngoại vi, cơ sở lưu trữ, dữ liệu, tệp, dịch vụ và mạng, chỉ kể đến một vài ví dụ. Có nhiều lý do để muốn chia sẻ tài nguyên. Một lý do rõ ràng là kinh tế. Ví dụ, việc chia sẻ một cơ sở lưu trữ đáng tin cậy cao cấp sẽ rẻ hơn so với việc phải mua và duy trì lưu trữ cho từng người dùng riêng biệt.

Việc kết nối người dùng và tài nguyên cũng giúp việc cộng tác và trao đổi thông tin dễ dàng hơn, điều này được minh họa bằng sự thành công của Internet với các giao thức đơn giản để trao đổi tệp, thư, tài liệu, âm thanh và video.

Khả năng kết nối Internet đã cho phép những nhóm người ở xa nhau về mặt địa lý có thể làm việc cùng nhau bằng đủ loại phần mềm nhóm, tức là phần mềm biên tập cộng tác, hội nghị truyền hình, v.v., như được minh họa bằng các công ty phát triển phần mềm đa quốc gia đã thuê ngoài phần lớn hoạt động sản xuất mã của họ cho Châu Á, cũng như vô số các công cụ cộng tác trở nên (dễ dàng hơn) có sẵn nhờ đại dịch COVID-19.

Chia sẻ tài nguyên trong các hệ thống phân tán cũng được minh họa bằng sự thành công của các mạng ngang hàng chia sẻ tệp như BitTorrent. Các hệ thống phân tán này giúp người dùng dễ dàng chia sẻ tệp qua Internet. Các mạng ngang hàng thường liên quan đến việc phân phối các tệp phụ trợ tiện như âm thanh và video. Trong các trường hợp khác, công nghệ này được sử dụng để phân phối lưu trữ dữ liệu lớn, như trong trường hợp cập nhật phần mềm, dịch vụ sao lưu và đồng bộ hóa dữ liệu trên nhiều máy chủ.

Việc tích hợp liền mạch các tiện ích chia sẻ tài nguyên trong môi trường mạng hiện cũng trở nên phổ biến. Một nhóm người dùng có thể chỉ cần đặt các tệp vào một

thư mục chia sẻ đặc biệt được duy trì bởi bên thứ ba ở đâu đó trên Internet. Sử dụng phần mềm đặc biệt, thư mục chia sẻ hầu như không thể phân biệt được với các thư mục khác trên máy tính của người dùng. Trên thực tế, các dịch vụ này thay thế việc sử dụng thư mục chia sẻ trên hệ thống tệp phân tán cục bộ, giúp dữ liệu có sẵn cho người dùng bất kể tổ chức họ thuộc về và bất kể họ ở đâu. Dịch vụ được cung cấp cho các hệ điều hành khác nhau. Nơi lưu trữ dữ liệu chính xác hoàn toàn ẩn khỏi người dùng cuối.

1.2.2 Minh bạch phân phối

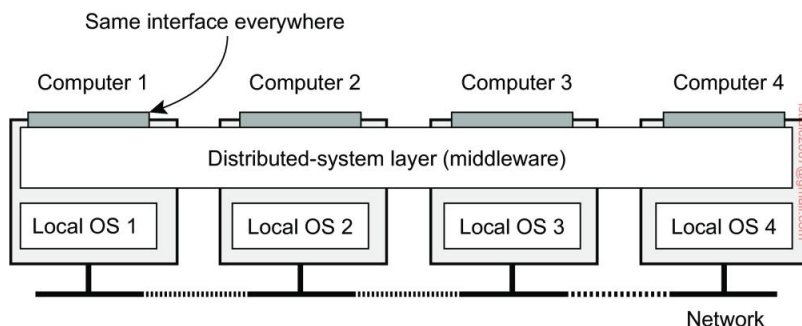
Một mục tiêu quan trọng của hệ thống phân tán là che giấu thực tế rằng các quy trình và tài nguyên của nó được phân phối vật lý trên nhiều máy tính, có thể cách nhau bởi khoảng cách lớn. Nói cách khác, nó cố gắng làm cho việc phân phối các quy trình và tài nguyên trở nên minh bạch, tức là vô hình, đối với người dùng cuối và các ứng dụng. Như chúng ta sẽ thảo luận sâu hơn trong Chương 2, việc đạt được tính minh bạch phân phối được thực hiện thông qua cái được gọi là phần mềm trung gian, được phác họa trong Hình 1.2 (xem Gazis và Katsiri [2022] để biết phần giới thiệu đầu tiên).

Về bản chất, những gì các ứng dụng nhìn thấy là cùng một giao diện ở mọi nơi, trong khi đằng sau giao diện đó, nơi và cách thức các quy trình và tài nguyên và cách chúng được truy cập được giữ trong suốt. Có nhiều loại minh bạch khác nhau, chúng ta sẽ thảo luận tiếp theo.

Các loại minh bạch phân phối

Khái niệm về tính minh bạch có thể được áp dụng cho một số khía cạnh của hệ thống phân tán, trong đó những khía cạnh quan trọng nhất được liệt kê trong Hình 1.3. Chúng tôi sử dụng thuật ngữ đối tượng để chỉ một quy trình hoặc một tài nguyên.

Tính minh bạch của Access liên quan đến việc ẩn các khác biệt trong biểu diễn dữ liệu và cách các đối tượng có thể được truy cập. Ở cấp độ cơ bản, chúng tôi muốn ẩn các khác biệt trong kiến trúc máy, nhưng quan trọng hơn là chúng tôi đạt được



Hình 1.2: Thực hiện tính minh bạch phân phối thông qua lớp phần mềm trung gian.

Mô tả về tính minh bạch	
Truy cập	Ẩn sự khác biệt trong cách biểu diễn dữ liệu và cách một đối tượng được truy cập
Vị trí	Ẩn vị trí của một đối tượng
Di dời	Ẩn rằng một đối tượng có thể được di chuyển đến một vị trí khác trong khi đang sử dụng
Di cư	Ẩn rằng một đối tượng có thể di chuyển đến một vị trí khác
Sao chép	Ẩn rằng một đối tượng được sao chép
Đồng thời	Ẩn rằng một đối tượng có thể được chia sẻ bởi nhiều đối tượng độc lập ngư ời sử dụng
Sự thất bại	Ẩn lỗi và phục hồi của một đối tượng

Hình 1.3: Các dạng minh bạch khác nhau trong hệ thống phân tán (xem ISO [1995]). Một đối tượng có thể là một tài nguyên hoặc một quy trình.

thỏa thuận về cách dữ liệu được biểu diễn bởi các máy móc và hệ điều hành khác nhau. Ví dụ, một hệ thống phân tán có thể có các hệ thống máy tính chạy các hệ điều hành khác nhau, mỗi hệ điều hành có quy ước đặt tên tệp riêng. Sự khác biệt trong quy ước đặt tên, sự khác biệt trong thao tác tệp hoặc sự khác biệt trong cách thức giao tiếp cấp thấp với các quy trình khác diễn ra nơi, là những ví dụ về các vấn đề truy cập mà tốt nhất nên được ẩn khỏi người dùng và ứng dụng.

Một nhóm quan trọng của các loại minh bạch liên quan đến vị trí của quá trình hoặc tài nguyên. Tính minh bạch về vị trí đề cập đến thực tế là người dùng không thể cho biết vị trí vật lý của một đối tượng trong hệ thống. Đặt tên đóng vai trò quan trọng trong việc đạt được sự minh bạch về vị trí. Đặc biệt, tính minh bạch về vị trí thường có thể đạt được bằng cách chỉ gán tên logic đối với các nguồn tài nguyên, nghĩa là các tên mà vị trí của một nguồn tài nguyên không được bí mật được mã hóa. Một ví dụ về tên như vậy là bộ định vị tài nguyên thống nhất (URL) <https://www.distributed-systems.net/>, mà không đưa ra manh mối nào về vị trí thực tế của máy chủ Web nơi cuốn sách này được cung cấp. URL cũng

không cung cấp manh mối liệu các tệp tin tại trang web đó có luôn ở vị trí hiện tại của chúng hay không hoặc mới được chuyển đến đó. Ví dụ, toàn bộ trang web có thể đã được dịch và di chuyển từ trung tâm dữ liệu này sang trung tâm dữ liệu khác, nhưng người dùng không nên để ý. Cái sau là một ví dụ về tính minh bạch trong việc di dời, đang ngày càng trở nên quan trọng trong bối cảnh điện toán đám mây: hiện tượng mà dịch vụ được cung cấp bởi các bộ sưu tập lớn các máy chủ từ xa. Chúng tôi quay trở lại điện toán đám mây trong các chương tiếp theo và đặc biệt là trong Chương 2.

Trong đó tính minh bạch của việc di dời đề cập đến việc được di chuyển bởi các phân phối hệ thống, tính minh bạch di chuyển được cung cấp bởi một hệ thống phân tán khi nó hỗ trợ tính di động của các quy trình và tài nguyên do người dùng khởi tạo, mà không ảnh hưởng đến hoạt động và giao tiếp đang diễn ra. Một ví dụ điển hình là giao tiếp giữa các điện thoại di động: bất kể hai người

thực sự đang di chuyển, điện thoại di động sẽ cho phép họ tiếp tục cuộc trò chuyện. Các ví dụ khác có thể kể đến bao gồm theo dõi và truy tìm hàng hóa trực tuyến khi chúng đang được vận chuyển từ nơi này đến nơi khác và hội nghị truyền hình (một phần) bằng các thiết bị được trang bị Internet di động.

Như chúng ta sẽ thấy, sao chép đóng vai trò quan trọng trong các hệ thống phân tán. Ví dụ, tài nguyên có thể được sao chép để tăng tính khả dụng hoặc cải thiện hiệu suất bằng cách đặt một bản sao gần nơi truy cập.

Tính minh bạch của bản sao liên quan đến việc ẩn thực tế là có nhiều bản sao của một tài nguyên tồn tại hoặc nhiều quy trình đang hoạt động ở một dạng chế độ đồng bộ để một quy trình có thể tiếp quản khi quy trình khác bị lỗi. Để ẩn bản sao khỏi người dùng, tất cả các bản sao phải có cùng tên. Do đó, một hệ thống hỗ trợ tính minh bạch của bản sao thứ ờng cũng phải hỗ trợ tính minh bạch về vị trí, vì nếu không thì không thể tham chiếu đến các bản sao ở các vị trí khác nhau.

Chúng tôi đã đề cập rằng một mục tiêu quan trọng của các hệ thống phân tán là cho phép chia sẻ tài nguyên. Trong nhiều trường hợp, việc chia sẻ tài nguyên được thực hiện theo hình thức hợp tác, như trong trường hợp các kênh truyền thông. Tuy nhiên, cũng có nhiều ví dụ về việc chia sẻ tài nguyên mang tính cạnh tranh. Ví dụ, hai người dùng độc lập có thể lưu trữ các tệp của họ trên cùng một máy chủ tệp hoặc có thể truy cập vào cùng một bảng trong cơ sở dữ liệu dùng chung. Trong những trường hợp như vậy, điều quan trọng là mỗi người dùng không nhận thấy rằng người kia đang sử dụng cùng một tài nguyên. Hiện tượng này được gọi là tính minh bạch đồng thời. Một vấn đề quan trọng là việc truy cập đồng thời vào một tài nguyên được chia sẻ khiến tài nguyên đó ở trạng thái nhất quán. Tính nhất quán có thể đạt được thông qua các cơ chế khóa, theo đó người dùng được cấp quyền truy cập độc quyền vào tài nguyên mong muốn. Một cơ chế tính vi hơn là sử dụng các giao dịch, nhưng chúng có thể khó triển khai trong một hệ thống phân tán, đặc biệt là khi khả năng mở rộng là một vấn đề.

Cuối cùng nhưng chắc chắn không kém phần quan trọng, điều quan trọng là hệ thống phân tán phải cung cấp tính minh bạch về lỗi. Điều này có nghĩa là người dùng hoặc ứng dụng không nhận thấy rằng một số phần của hệ thống không hoạt động bình thường và sau đó hệ thống (và tự động) phục hồi sau lỗi đó. Việc che giấu lỗi là một trong những vấn đề khó khăn nhất trong hệ thống phân tán và thậm chí là không thể khi đưa ra một số giả định thực tế, như chúng ta sẽ thảo luận trong Chương 8. Khó khăn chính trong việc che giấu và phục hồi minh bạch sau lỗi nằm ở chỗ không thể phân biệt giữa một quy trình đã chết và một quy trình phản hồi chậm đến mức đau đớn. Ví dụ, khi liên hệ với một máy chủ Web bận, trình duyệt cuối cùng sẽ hết thời gian chờ và báo cáo rằng trang Web không khả dụng. Tại thời điểm đó, người dùng không thể biết liệu máy chủ thực sự ngừng hoạt động hay mạng bị tắc nghẽn nghiêm trọng.

Mức độ minh bạch phân phối

Mặc dù tính minh bạch trong phân phối thường được coi là tốt hơn đối với bất kỳ hệ thống phân phối nào, nhưng có những tình huống mà việc cố gắng che giấu mọi khía cạnh phân phối khỏi người dùng một cách mù quáng không phải là một ý tưởng hay. Một ví dụ đơn giản là yêu cầu báo điện tử của bạn xuất hiện trong hộp thư của bạn trước 7 giờ sáng theo giờ địa phương, như thư ông lệ, trong khi hiện tại bạn đang ở đầu bên kia của thế giới sống trong một múi giờ khác. Tờ báo buổi sáng của bạn sẽ không phải là tờ báo buổi sáng mà bạn thường đọc.

Tương tự như vậy, một hệ thống phân tán diện rộng kết nối một quy trình ở San Francisco với một quy trình ở Amsterdam không thể che giấu được sự thật rằng Mẹ Thiên nhiên sẽ không cho phép nó gửi một thông điệp từ quy trình này sang quy trình khác trong thời gian ít hơn khoảng 35 mili giây. Thực tế cho thấy rằng thực tế phải mất đến vài trăm mili giây khi sử dụng mạng máy tính. Việc truyền tín hiệu không chỉ bị giới hạn bởi tốc độ ánh sáng mà còn bởi khả năng xử lý hạn chế và độ trễ trong các công tắc trung gian.

Cũng có sự đánh đổi giữa mức độ minh bạch cao và hiệu suất của hệ thống. Ví dụ, nhiều ứng dụng Internet liên tục cố gắng liên lạc với máy chủ trước khi cuối cùng từ bỏ. Do đó, việc cố gắng che giấu lỗi máy chủ tạm thời trước khi thử một lỗi khác có thể làm chậm toàn bộ hệ thống. Trong trường hợp như vậy, có thể tốt hơn là từ bỏ sớm hơn hoặc ít nhất là để người dùng hủy các nỗ lực liên lạc.

Một ví dụ khác là khi chúng ta cần đảm bảo rằng một số bản sao, nằm trên các lục địa khác nhau, phải luôn nhất quán. Nói cách khác, nếu một bản sao bị thay đổi, thay đổi đó phải được truyền đến tất cả các bản sao trước khi cho phép bất kỳ hoạt động nào khác. Một hoạt động cập nhật duy nhất hiện có thể mất vài giây để hoàn thành, điều này không thể ẩn khỏi người dùng.

Cuối cùng, có những tình huống mà việc ẩn phân phối không phải là một ý tưởng hay. Khi các hệ thống phân phối đang mở rộng sang các thiết bị mà mọi người mang theo bên mình và khi mà khái niệm về vị trí và nhận thức ngữ cảnh ngày càng trở nên quan trọng, thì tốt nhất là thực sự phơi bày phân phối thay vì cố gắng ẩn nó. Một ví dụ rõ ràng là sử dụng các dịch vụ dựa trên vị trí, thường có thể tìm thấy trên điện thoại di động, chẳng hạn như tìm cửa hàng gần nhất hoặc bất kỳ người nào ở gần.

Có những lập luận khác chống lại tính minh bạch phân phối. Nhận ra rằng tính minh bạch phân phối hoàn toàn là điều không thể, chúng ta nên tự hỏi liệu có khôn ngoan không khi giả vờ rằng chúng ta có thể đạt được điều đó. Có thể tốt hơn nhiều khi làm cho phân phối trở nên rõ ràng để người dùng và nhà phát triển ứng dụng không bao giờ bị lừa tin rằng có thứ gọi là tính minh bạch. Kết quả là người dùng sẽ hiểu rõ hơn nhiều về hành vi (đôi khi không mong đợi) của một hệ thống phân tán và do đó chuẩn bị tốt hơn nhiều để đối phó với hành vi này.

Kết luận là việc hướng tới tính minh bạch trong phân phối có thể là một mục tiêu tốt khi thiết kế và triển khai các hệ thống phân phối, nhưng

nó nên được xem xét cùng với các vấn đề khác như hiệu suất và khả năng hiểu được. Cái giá để đạt được sự minh bạch hoàn toàn có thể cao đến mức đáng ngạc nhiên.

Ghi chú 1.2 (Thảo luận: Chống lại sự minh bạch trong phân phối)

Một số nhà nghiên cứu đã lập luận rằng việc ẩn phân phối sẽ chỉ làm phức tạp thêm quá trình phát triển các hệ thống phân tán, chính xác là vì lý do không bao giờ có thể đạt được tính minh bạch phân phối hoàn toàn. Một kỹ thuật phổ biến để đạt được tính minh bạch truy cập là mở rộng các lệnh gọi thủ tục đến các máy chủ từ xa. Tuy nhiên, Waldo et al. [1997] đã chỉ ra rằng việc cố gắng ẩn phân phối bằng các lệnh gọi thủ tục từ xa như vậy có thể dẫn đến ngữ nghĩa không được hiểu rõ, vì lý do đơn giản là một lệnh gọi thủ tục sẽ thay đổi khi được thực hiện qua một liên kết truyền thông bị lỗi.

Một giải pháp thay thế, nhiều nhà nghiên cứu và chuyên gia hiện đang tranh luận về việc giảm tính minh bạch, ví dụ, bằng cách sử dụng giao tiếp theo kiểu tin nhắn rõ ràng hơn hoặc đăng yêu cầu rõ ràng hơn đến và nhận kết quả từ các máy từ xa, như được thực hiện trên Web khi truy xuất các trang. Các giải pháp như vậy sẽ được thảo luận chi tiết trong chương tiếp theo.

Một quan điểm có phần cấp tiến đã được Wams [2012] đưa ra khi tuyên bố rằng các lỗi cục bộ ngăn cản việc dựa vào việc thực hiện thành công một dịch vụ từ xa. Nếu không thể đảm bảo độ tin cậy như vậy, thì tốt nhất là luôn chỉ thực hiện các lần thực hiện cục bộ, dẫn đến nguyên tắc sao chép trừu tượng khi sử dụng. Theo nguyên tắc này, dữ liệu chỉ có thể được truy cập sau khi chúng đã được chuyển đến máy của quy trình muốn có dữ liệu đó. Hơn nữa, không nên sửa đổi một mục dữ liệu. Thay vào đó, nó chỉ có thể được cập nhật lên phiên bản mới. Không khó để tư tưởng tư tưởng rằng nhiều vấn đề khác sẽ xuất hiện. Tuy nhiên, Wams cho thấy nhiều ứng dụng hiện có có thể được trang bị thêm cho phương pháp thay thế này mà không làm mất đi chức năng.

1.2.3 Tính cởi mở

Một mục tiêu quan trọng khác của hệ thống phân tán là tính mở. Một hệ thống phân tán mở về cơ bản là một hệ thống cung cấp các thành phần có thể dễ dàng được sử dụng bởi hoặc tích hợp vào các hệ thống khác. Đồng thời, bản thân một hệ thống phân tán mở thường bao gồm các thành phần có nguồn gốc từ nơi khác.

Khả năng tương tác, khả năng kết hợp và khả năng mở rộng

Mở có nghĩa là các thành phần phải tuân thủ các quy tắc chuẩn mô tả cú pháp và ngữ nghĩa của những gì các thành phần đó cung cấp (tức là dịch vụ mà chúng cung cấp). Một cách tiếp cận chung là định nghĩa các dịch vụ thông qua giao diện bằng Ngôn ngữ định nghĩa giao diện (IDL). Định nghĩa giao diện được viết bằng IDL gần như luôn chỉ nắm bắt cú pháp của các dịch vụ. Nói cách khác, chúng chỉ định chính xác tên của các hàm có sẵn

cùng với các loại tham số, giá trị trả về, các ngoại lệ có thể được đưa ra, v.v. Phần khó là chỉ định chính xác những dịch vụ đó làm gì, tức là ngữ nghĩa của giao diện. Trong thực tế, các thông số kỹ thuật như vậy được đưa ra theo cách không chính thức bằng ngôn ngữ tự nhiên.

Nếu được chỉ định đúng, định nghĩa giao diện cho phép một quy trình tùy ý cần một giao diện nhất định, giao tiếp với một quy trình khác cung cấp giao diện đó. Nó cũng cho phép hai bên độc lập xây dựng các triển khai hoàn toàn khác nhau của các giao diện đó, dẫn đến hai thành phần riêng biệt hoạt động theo cùng một cách chính xác.

Các thông số kỹ thuật phù hợp là đầy đủ và trung lập. Đầy đủ có nghĩa là mọi thứ cần thiết để thực hiện triển khai thực sự đã được chỉ định. Tuy nhiên, nhiều định nghĩa giao diện không hoàn chỉnh, do đó nhà phát triển cần thêm các chi tiết cụ thể cho triển khai.

Điều quan trọng không kém là các thông số kỹ thuật không quy định việc triển khai sẽ như thế nào; chúng phải mang tính trung lập.

Như đã chỉ ra trong Blair và Stefani [1998], tính hoàn chỉnh và tính trung lập rất quan trọng đối với khả năng tương tác và tính di động. Khả năng tương tác đặc trưng cho mức độ mà hai triển khai hệ thống hoặc thành phần từ các nhà sản xuất khác nhau có thể cùng tồn tại và hoạt động cùng nhau chỉ bằng cách dựa vào các dịch vụ của nhau theo quy định của một tiêu chuẩn chung. Tính di động đặc trưng cho mức độ mà một ứng dụng được phát triển cho một hệ thống phân tán A có thể được thực thi, mà không cần sửa đổi, trên một hệ thống phân tán B khác triển khai cùng giao diện với A.

Một mục tiêu quan trọng khác đối với hệ thống phân tán mở là phải dễ dàng cấu hình hệ thống từ các thành phần khác nhau (có thể từ các nhà phát triển khác nhau). Hơn nữa, phải dễ dàng thêm các thành phần mới hoặc thay thế các thành phần hiện có mà không ảnh hưởng đến các thành phần vẫn giữ nguyên. Nói cách khác, một hệ thống phân tán mở cũng phải có khả năng mở rộng. Ví dụ, trong một hệ thống có khả năng mở rộng, việc thêm các phần chạy trên một hệ điều hành khác hoặc thậm chí thay thế toàn bộ hệ thống tệp phải tương đối dễ dàng.

Ví dụ tương đối đơn giản về khả năng mở rộng là các plug-in cho trình duyệt Web, nhưng cũng có các plug-in cho trang web, chẳng hạn như plug-in được sử dụng cho WordPress.

Ghi chú 1.3 (Thảo luận: Hệ thống mở trong thực tế)

Tất nhiên, những gì chúng tôi vừa mô tả là một tình huống lý tưởng. Thực tế cho thấy nhiều hệ thống phân tán không mở như chúng ta mong muốn và vẫn cần rất nhiều nỗ lực để ghép nhiều phần lại với nhau để tạo thành một hệ thống phân tán. Một cách thoát khỏi tình trạng thiếu cởi mở là chỉ cần tiết lộ tất cả các chi tiết đẫm máu của một thành phần và cung cấp cho các nhà phát triển mã nguồn thực tế. Cách tiếp cận này đang ngày càng trở nên phổ biến, dẫn đến cái gọi là các dự án nguồn mở, nơi nhiều nhóm người đóng góp vào việc cải thiện và gỡ lỗi hệ thống. Phải thừa nhận rằng, đây là cách mở nhất mà một hệ thống có thể đạt được, nhưng liệu đây có phải là cách tốt nhất hay không vẫn còn là một câu hỏi.

Tách chính sách khỏi cơ chế

Để đạt được tính linh hoạt trong các hệ thống phân tán mở, điều quan trọng là hệ thống phải được tổ chức như một tập hợp các thành phần tương đối nhỏ và dễ thay thế hoặc thích ứng. Điều này ngụ ý rằng chúng ta nên cung cấp các định nghĩa không chỉ về các giao diện cấp cao nhất, tức là các giao diện mà người dùng và ứng dụng nhìn thấy, mà còn về các định nghĩa cho các giao diện với các phần bên trong của hệ thống và mô tả cách các phần đó tương tác. Cách tiếp cận này tương đối mới. Nhiều hệ thống cũ hơn và thậm chí là hiện đại được xây dựng bằng cách sử dụng cách tiếp cận đơn khối trong đó các thành phần chỉ được tách biệt về mặt logic nhưng được triển khai như một chương trình lớn. Cách tiếp cận này khiến việc thay thế hoặc thích ứng một thành phần trở nên khó khăn mà không ảnh hưởng đến toàn bộ hệ thống. Do đó, các hệ thống đơn khối có xu hướng đóng thay vì mở.

Nhu cầu thay đổi hệ thống phân tán thường xuất phát từ một thành phần không cung cấp chính sách tối ưu cho người dùng hoặc ứng dụng cụ thể.

Ví dụ, hãy xem xét bộ nhớ đệm trong trình duyệt Web. Có nhiều thông số khác nhau cần được xem xét:

Lưu trữ: Dữ liệu sẽ được lưu trữ ở đâu? Thông thường, sẽ có bộ nhớ đệm trong bộ nhớ bên cạnh bộ nhớ lưu trữ trên đĩa. Trong trường hợp sau, cần phải xem xét vị trí chính xác trong hệ thống tệp cục bộ.

Miễn trừ: Khi bộ nhớ đệm đầy, dữ liệu nào sẽ bị xóa để các trang mới tải về có thể được lưu trữ không?

Chia sẻ: Mỗi trình duyệt có sử dụng bộ nhớ đệm riêng hay bộ nhớ đệm có thể được chia sẻ giữa các trình duyệt của nhiều người dùng khác nhau?

Làm mới: Khi nào trình duyệt kiểm tra xem dữ liệu được lưu trong bộ nhớ đệm có còn được cập nhật không? Bộ nhớ đệm hiệu quả nhất khi trình duyệt có thể trả về các trang mà không cần phải liên hệ với Trang web gốc. Tuy nhiên, điều này có nguy cơ trả về dữ liệu cũ. Cũng lưu ý rằng tốc độ làm mới phụ thuộc rất nhiều vào dữ liệu thực sự được lưu trong bộ nhớ đệm: trong khi lịch trình tàu hỏa hầu như không thay đổi, thì điều này không đúng đối với các trang web hiển thị tình trạng giao thông đường bộ hiện tại hoặc tệ hơn là giá cổ phiếu.

Điều chúng ta cần là sự tách biệt giữa chính sách và cơ chế. Ví dụ, trong trường hợp lưu trữ đệm Web, trình duyệt lý tưởng nhất là cung cấp các tiện ích chỉ để lưu trữ tài liệu (tức là một cơ chế) và đồng thời cho phép người dùng quyết định tài liệu nào được lưu trữ và trong bao lâu (tức là một chính sách). Trong thực tế, điều này có thể được thực hiện bằng cách cung cấp một tập hợp các tham số phong phú mà người dùng có thể thiết lập (động). Khi thực hiện bước này xa hơn, trình duyệt thậm chí có thể cung cấp các tiện ích để cấm các chính sách mà người dùng đã triển khai như một thành phần riêng biệt.

Lưu ý 1.4 (Thảo luận: Liệu sự tách biệt nghiêm ngặt có thực sự là điều chúng ta cần không?)

Về mặt lý thuyết, việc tách biệt chặt chẽ chính sách khỏi cơ chế có vẻ là giải pháp cần thực hiện.

Tuy nhiên, có một sự đánh đổi quan trọng cần cân nhắc: sự phân tách càng chặt chẽ thì chúng ta càng cần phải đảm bảo cung cấp bộ cơ chế phù hợp.

Trên thực tế, điều này có nghĩa là một tập hợp các tính năng phong phú được cung cấp, dẫn đến nhiều tham số cấu hình. Ví dụ, trình duyệt Firefox phổ biến đi kèm với một vài trăm tham số cấu hình. Hãy tưởng tượng không gian cấu hình bùng nổ như thế nào khi xem xét các hệ thống phân tán lớn bao gồm nhiều thành phần. Nói cách khác, việc tách biệt chặt chẽ các chính sách và cơ chế có thể dẫn đến các vấn đề cấu hình cực kỳ phức tạp.

Một lựa chọn để giảm bớt những vấn đề này là cung cấp các mặc định hợp lý và đây là điều thường xảy ra trong thực tế. Một cách tiếp cận thay thế là hệ thống quan sát cách sử dụng của chính nó và thay đổi các thiết lập tham số một cách động. Điều này dẫn đến cái được gọi là hệ thống tự cấu hình. Tuy nhiên, chỉ riêng thực tế là nhiều cơ chế cần được cung cấp để hỗ trợ nhiều chính sách khác nhau thường khiến việc mã hóa các hệ thống phân tán trở nên rất phức tạp. Việc mã hóa cứng các chính sách vào một hệ thống phân tán có thể làm giảm đáng kể tính phức tạp, nhưng phải trả giá bằng tính linh hoạt kém hơn.

1.2.4 Độ tin cậy

Như tên gọi của nó, độ tin cậy đề cập đến mức độ mà một hệ thống máy tính có thể được tin cậy để hoạt động như mong đợi. Trái ngược với các hệ thống máy tính đơn lẻ, độ tin cậy trong các hệ thống phân tán có thể khá phức tạp do lỗi một phần: ở đâu đó có một thành phần bị lỗi trong khi toàn bộ hệ thống dường như vẫn đáp ứng được kỳ vọng (cho đến một thời điểm hoặc thời điểm nhất định). Mặc dù các hệ thống máy tính đơn lẻ cũng có thể gặp phải lỗi không xuất hiện ngay lập tức, nhưng việc có một bộ sưu tập lớn các hệ thống máy tính được kết nối mạng có thể làm phức tạp vấn đề đáng kể. Trên thực tế, ngược lại ta nên cho rằng tại bất kỳ thời điểm nào, luôn có lỗi một phần xảy ra. Một mục tiêu quan trọng của hệ thống phân tán là che giấu những lỗi đó, cũng như che giấu việc phục hồi sau những lỗi đó. Việc che giấu này là bản chất của khả năng chịu đựng lỗi, do đó được gọi là khả năng chịu lỗi.

Các khái niệm cơ bản

Độ tin cậy là một thuật ngữ bao gồm một số yêu cầu hữu ích đối với các hệ thống phân tán, bao gồm những yêu cầu sau [Kopetz và Verissimo, 1993]:

- Tính khả dụng
- Độ tin cậy
- An toàn
- Khả năng bảo trì

Tính khả dụng được định nghĩa là thuộc tính mà một hệ thống sẵn sàng để sử dụng ngay lập tức. Nói chung, nó đề cập đến khả năng hệ thống đang hoạt động chính xác tại bất kỳ thời điểm nào và có thể thực hiện các chức năng của mình thay mặt cho người dùng. Nói cách khác, một hệ thống có tính khả dụng cao là hệ thống có nhiều khả năng hoạt động tại một thời điểm nhất định.

Độ tin cậy đề cập đến đặc tính mà một hệ thống có thể chạy liên tục mà không bị lỗi. Trái ngược với tính khả dụng, độ tin cậy được định nghĩa theo khoảng thời gian thay vì một khoảng khắc trong thời gian. Một hệ thống có độ tin cậy cao là hệ thống có nhiều khả năng tiếp tục hoạt động mà không bị gián đoạn trong một khoảng thời gian tương đối dài. Đây là một sự khác biệt tinh tế nhưng quan trọng khi so sánh với tính khả dụng. Nếu một hệ thống ngừng hoạt động trung bình trong một mili giây ngẫu nhiên mỗi giờ, thì tính khả dụng của nó là hơn 99,999 phần trăm, nhưng vẫn không đáng tin cậy. Tương tự như vậy, một hệ thống không bao giờ gặp sự cố nhưng bị tắt trong hai tuần cụ thể vào mỗi tháng Tám có độ tin cậy cao nhưng chỉ có 96 phần trăm tính khả dụng. Hai điều này không giống nhau.

An toàn đề cập đến tình huống khi một hệ thống tạm thời không hoạt động đúng cách, không có sự kiện thảm khốc nào xảy ra. Ví dụ, nhiều hệ thống kiểm soát quy trình, chẳng hạn như hệ thống được sử dụng để kiểm soát các nhà máy điện hạt nhân hoặc đưa người đi vào không gian, được yêu cầu cung cấp mức độ an toàn cao. Nếu các hệ thống kiểm soát như vậy tạm thời hỏng chỉ trong một khoảng khắc rất ngắn, hậu quả có thể là thảm khốc. Nhiều ví dụ trong quá khứ (và có thể còn nhiều ví dụ khác nữa) cho thấy việc xây dựng các hệ thống an toàn khó khăn như thế nào.

Cuối cùng, khả năng bảo trì đề cập đến mức độ dễ dàng sửa chữa một hệ thống bị lỗi. Một hệ thống có khả năng bảo trì cao cũng có thể cho thấy mức độ sẵn sàng cao, đặc biệt là nếu lỗi có thể được phát hiện và sửa chữa tự động. Tuy nhiên, như chúng ta sẽ thấy, việc tự động phục hồi sau lỗi dễ nói hơn làm.

Theo truyền thống, khả năng chịu lỗi có liên quan đến ba số liệu sau:

- Thời gian trung bình đến khi hỏng (MTTF): Thời gian trung bình cho đến khi một thành phần thất bại.
- Thời gian trung bình để sửa chữa (MTTR): Thời gian trung bình cần thiết để sửa chữa một thành phần.
- Thời gian trung bình giữa các lần hỏng hóc (MTBF): Đơn giản là $MTTF + MTTR$.

Lưu ý rằng các số liệu này chỉ có ý nghĩa nếu chúng ta có khái niệm chính xác về lỗi thực sự là gì. Như chúng ta sẽ thấy trong Chương 8, việc xác định sự xuất hiện của lỗi thực sự có thể không rõ ràng như vậy.

Lỗi, sai sót, thất bại

Một hệ thống được cho là thất bại khi nó không thể đáp ứng được những lời hứa của mình. Cụ thể, nếu một hệ thống phân tán được thiết kế để cung cấp cho người dùng của nó một số dịch vụ, hệ thống đã thất bại khi một hoặc nhiều dịch vụ đó không thể được cung cấp (hoàn toàn). Lỗi là một phản trạng thái của hệ thống có thể dẫn đến thất bại. Ví dụ

Ví dụ, khi truyền các gói tin qua mạng, có thể dự đoán rằng một số gói tin đã bị hỏng khi chúng đến máy thu. Hỏng trong ngữ cảnh này có nghĩa là máy thu có thể cảm nhận sai giá trị bit (ví dụ, đọc 1 thay vì 0) hoặc thậm chí không thể phát hiện ra rằng có thứ gì đó đã đến.

Nguyên nhân gây ra lỗi được gọi là lỗi. Rõ ràng, việc tìm ra nguyên nhân gây ra lỗi là rất quan trọng. Ví dụ, phụ thuộc truyền dẫn không đúng hoặc không tốt có thể dễ dàng khiến các gói tin bị hỏng. Trong trường hợp này, việc loại bỏ lỗi từ phụ thuộc đối dễ dàng. Tuy nhiên, lỗi truyền dẫn cũng có thể do điều kiện thời tiết xấu, chẳng hạn như trong mạng không dây. Thay đổi thời tiết để giảm hoặc ngăn ngừa lỗi thì khó hơn một chút.

Một ví dụ khác, một chương trình bị sập rõ ràng là một lỗi, có thể xảy ra vì chương trình đã nhập một nhánh mã chứa lỗi lập trình (tức là lỗi lập trình). Nguyên nhân gây ra lỗi đó thường là một lập trình viên. Nói cách khác, lập trình viên là nguyên nhân gây ra lỗi (lỗi lập trình), dẫn đến lỗi (chương trình bị sập).

Xây dựng các hệ thống đáng tin cậy có liên quan chặt chẽ đến việc kiểm soát lỗi. Như Avizienis và cộng sự đã giải thích [2004], có thể phân biệt giữa việc ngăn ngừa, dung thứ, loại bỏ và dự báo lỗi. Đối với mục đích của chúng tôi, vấn đề quan trọng nhất là khả năng chịu lỗi, nghĩa là hệ thống có thể cung cấp dịch vụ ngay cả khi có lỗi. Ví dụ, bằng cách áp dụng mã sửa lỗi để truyền các gói tin, có thể dung thứ, ở một mức độ nhất định, các lỗi truyền từ phụ thuộc kém và giảm khả năng lỗi (gói tin bị hỏng) có thể dẫn đến lỗi.

Lỗi thường được phân loại thành lỗi tạm thời, lỗi không liên tục hoặc lỗi vĩnh viễn. Lỗi tạm thời xảy ra một lần rồi biến mất. Nếu thao tác được lặp lại, lỗi sẽ biến mất. Một con chim bay qua chùm tia của máy phát vi sóng có thể gây mất bit trên một số mạng (chưa kể đến một con chim bị nung). Nếu quá trình truyền hết thời gian và được thử lại, có thể nó sẽ hoạt động lần thứ hai.

Lỗi ngắt quãng xảy ra, sau đó tự biến mất, rồi lại tái phát, v.v. Tiếp xúc lỏng lẻo trên đầu nối thường gây ra lỗi ngắt quãng. Lỗi ngắt quãng gây ra rất nhiều phiền toái vì chúng khó chẩn đoán. Thông thường, khi bác sĩ lỗi xuất hiện, hệ thống hoạt động tốt.

Lỗi vĩnh viễn là lỗi tiếp tục tồn tại cho đến khi thành phần bị lỗi được thay thế. Chíp bị cháy, lỗi phần mềm và đầu đĩa bị hỏng là những ví dụ về lỗi vĩnh viễn.

Các hệ thống đáng tin cậy cũng cần phải cung cấp bảo mật, đặc biệt là về mặt bảo mật và toàn vẹn. Bảo mật là đặc tính mà thông tin chỉ được tiết lộ cho các bên được ủy quyền, trong khi tính toàn vẹn liên quan đến việc đảm bảo rằng các thay đổi đối với nhiều tài sản khác nhau chỉ có thể được thực hiện theo cách được ủy quyền. Thật vậy, chúng ta có thể nói về một hệ thống đáng tin cậy khi tính bảo mật và toàn vẹn không được áp dụng không? Chúng ta quay lại vấn đề bảo mật tiếp theo.

1.2.5 Bảo mật

Một hệ thống phân tán không an toàn thì không đáng tin cậy. Như đã đề cập, cần đặc biệt chú ý để đảm bảo tính bảo mật và tính toàn vẹn, cả hai đều liên quan trực tiếp đến việc tiết lộ và truy cập thông tin và tài nguyên được ủy quyền. Trong bất kỳ hệ thống máy tính nào, việc ủy quyền được thực hiện bằng cách kiểm tra xem một thực thể được xác định có quyền truy cập phù hợp hay không. Đổi lại, điều này có nghĩa là hệ thống phải biết rằng nó thực sự đang giao dịch với thực thể phù hợp. Vì lý do này, xác thực là điều cần thiết: xác minh tính chính xác của danh tính đã khai báo. Quan trọng không kém là khái niệm về lòng tin. Nếu một hệ thống có thể xác thực một người với một cách tích cực, thì việc xác thực đó có giá trị gì nếu người đó không đáng tin cậy? Chỉ vì lý do này, việc ủy quyền phù hợp là quan trọng, vì nó có thể được sử dụng để hạn chế mọi thiệt hại mà một người, người có thể không đáng tin cậy, có thể gây ra. Ví dụ, trong các hệ thống tài chính, việc ủy quyền có thể hạn chế số tiền mà một người được phép chuyển giữa các tài khoản khác nhau. Chúng ta sẽ thảo luận chi tiết về lòng tin, xác thực và ủy quyền trong Chương 9.

Các yếu tố chính cần thiết để hiểu về bảo mật

Một kỹ thuật thiết yếu để đảm bảo an toàn cho các hệ thống phân tán là mật mã. Đây không phải là nơi để thảo luận sâu rộng về mật mã (chúng ta cũng sẽ hoãn lại cho đến Chương 9), tuy nhiên để hiểu cách bảo mật phù hợp với các quan điểm khác nhau trong các chương sau, chúng tôi sẽ giới thiệu một cách không chính thức một số yếu tố cơ bản của nó.

Nói một cách đơn giản, bảo mật trong các hệ thống phân tán là tất cả về việc mã hóa và giải mã dữ liệu bằng khóa bảo mật. Cách dễ nhất để xem xét khóa bảo mật K là xem nó như một hàm hoạt động trên một số dữ liệu. Chúng tôi sử dụng ký hiệu $K(\text{dữ liệu})$ để thể hiện thực tế là khóa K hoạt động trên dữ liệu.

Có hai cách mã hóa và giải mã dữ liệu. Trong hệ thống mật mã đối xứng, mã hóa và giải mã diễn ra bằng một khóa duy nhất. Ký hiệu $EK(\text{dữ liệu})$ là mã hóa dữ liệu bằng khóa EK , và tương tự như vậy là $DK(\text{dữ liệu})$ để giải mã bằng khóa DK , thì trong hệ thống mật mã đối xứng, cùng một khóa được sử dụng để mã hóa và giải mã, tức là,

$$\text{nếu dữ liệu} = DK(EK(\text{dữ liệu})) \text{ thì } DK = EK.$$

Lưu ý rằng trong hệ thống mật mã đối xứng, khóa sẽ cần được giữ bí mật bởi tất cả các bên được phép mã hóa hoặc giải mã dữ liệu. Trong hệ thống mật mã bất đối xứng, khóa được sử dụng để mã hóa và giải mã là khác nhau.

Đặc biệt, có một khóa công khai PK có thể được sử dụng bởi bất kỳ ai và một khóa bí mật SK , đúng như tên gọi của nó, phải được giữ bí mật. Hệ thống mật mã bất đối xứng cũng được gọi là hệ thống khóa công khai.

Mã hóa và giải mã trong hệ thống khóa công khai có thể được sử dụng theo hai cách cơ bản khác nhau. Đầu tiên, nếu Alice muốn mã hóa dữ liệu mà chỉ Bob mới có thể giải mã, cô ấy nên sử dụng khóa công khai của Bob, PKB, dẫn đến dữ liệu được mã hóa PKB(data). Chỉ người giữ khóa bí mật liên quan mới có thể giải mã thông tin này, tức là Bob, người sẽ áp dụng thao tác SKB(PKB(data)), trả về dữ liệu.

Trường hợp sử dụng thứ hai và được áp dụng rộng rãi là thực hiện chữ ký số. Giả sử Alice cung cấp một số dữ liệu mà bất kỳ bên nào, như người giả sử là Bob, cần biết chắc chắn rằng dữ liệu đó đến từ Alice. Trong trường hợp đó, Alice có thể mã hóa dữ liệu bằng khóa bí mật SKA của cô ấy, dẫn đến SKA(data). Nếu có thể đảm bảo rằng khóa công khai PKA liên quan thực sự thuộc về Alice, thì việc giải mã thành công SKA(data) thành data là bằng chứng cho thấy Alice biết về dữ liệu: cô ấy là người duy nhất nắm giữ khóa bí mật SKA. Tất nhiên, chúng ta cần đưa ra giả định rằng Alice thực sự là người duy nhất nắm giữ SKA. Chúng ta quay lại một số giả định này trong Chương 9.

Thực tế, việc chứng minh rằng một thực thể đã nhìn thấy hoặc biết về một số dữ liệu thường được trả về trong các hệ thống phân tán an toàn. Việc đặt chữ ký số thực tế thường hiệu quả hơn bằng hàm băm. Hàm băm H có đặc tính là khi hoạt động trên một số dữ liệu, tức là H(dữ liệu), nó trả về một chuỗi có độ dài cố định, bất kể độ dài của dữ liệu. Bất kỳ thay đổi nào của dữ liệu thành data sẽ dẫn đến một giá trị băm khác H(dữ liệu). Hơn nữa, với một giá trị băm h, trên thực tế, không thể tính toán được dữ liệu gốc. Tất cả những điều này có nghĩa là để đặt chữ ký số, Alice tính $\text{sig} = \text{SKA}(H(\text{dữ liệu}))$ làm chữ ký của cô ấy và cho Bob biết về dữ liệu, H và sig. Để lưu ý mình, Bob có thể xác minh chữ ký đó bằng cách tính PKA(sig) và xác minh rằng nó khớp với giá trị H(dữ liệu).

Sử dụng mật mã trong hệ thống phân tán

Ứng dụng của mật mã trong các hệ thống phân tán có nhiều hình thức. Bên cạnh việc sử dụng chung cho mã hóa và chữ ký số, mật mã còn tạo thành cơ sở để hiện thực hóa một kênh an toàn giữa hai bên giao tiếp. Các kênh như vậy về cơ bản cho hai bên biết chắc chắn rằng họ thực sự đang giao tiếp với các thực thể mà họ mong đợi giao tiếp.

Nói cách khác, đây là kênh truyền thông hỗ trợ xác thực lẫn nhau.

Một ví dụ thực tế về kênh an toàn là sử dụng https khi truy cập trang web.

Hiện nay, nhiều trình duyệt yêu cầu các trang web hỗ trợ giao thức này và ít nhất sẽ cảnh báo người dùng khi không phải như vậy. Nhìn chung, việc sử dụng mật mã là cần thiết để thực hiện xác thực (và ủy quyền) trong các hệ thống phân tán.

Mật mã học cũng được sử dụng để tạo ra các cấu trúc dữ liệu phân tán an toàn.

Một ví dụ nổi tiếng là blockchain, theo nghĩa đen là một chuỗi các khối. Ý tưởng cơ bản rất đơn giản: băm dữ liệu trong khối B_i và đặt giá trị băm đó làm một phần của dữ liệu trong khối tiếp theo B_{i+1} . Bất kỳ thay đổi nào trong

B_i (ví dụ, là kết quả của một cuộc tấn công), sẽ yêu cầu kẻ tấn công cũng phải thay đổi giá trị băm được lưu trữ trong B_{i+1} . Tuy nhiên, vì người kế nhiệm của B_{i+1} chứa giá trị băm được tính toán trên dữ liệu trong B_{i+1} và do đó bao gồm giá trị băm ban đầu của B_i , kẻ tấn công cũng sẽ phải thay đổi giá trị băm mới của B_i được lưu trữ trong B_{i+1} . Tuy nhiên, việc thay đổi giá trị đó cũng có nghĩa là thay đổi giá trị băm của B_{i+1} và do đó là giá trị được lưu trữ trong B_{i+2} , đến lượt nó, yêu cầu phải tính toán giá trị băm mới cho B_{i+2} , v.v. Nói cách khác, bằng cách liên kết an toàn các khối thành một chuỗi, bất kỳ thay đổi thành công nào đối với một khối cũng yêu cầu tất cả các khối kế tiếp phải được sửa đổi. Những sửa đổi này sẽ không được chú ý, điều này hầu như không thể.

Mật mã cũng được sử dụng cho một cơ chế quan trọng khác trong các hệ thống phân tán: ủy quyền truy cập. Ý tưởng cơ bản là Alice có thể muốn ủy quyền một số quyền cho Bob, người này, đến lượt mình, có thể muốn chuyển một số quyền đó cho Chuck. Sử dụng các phương tiện thích hợp (mà chúng ta thảo luận trong Chương 9), một dịch vụ có thể kiểm tra an toàn rằng Chuck thực sự đã được ủy quyền để thực hiện một số thao tác nhất định, mà không cần dịch vụ đó phải kiểm tra với Alice xem việc ủy quyền đã được thực hiện hay chưa. Lưu ý rằng ủy quyền là điều mà chúng ta hiện đã quen: nhiều người trong chúng ta ủy quyền các quyền truy cập mà chúng ta có với tư cách là người dùng cho các ứng dụng cụ thể, chẳng hạn như ứng dụng email.

Một ứng dụng phân tán sắp tới của mật mã được gọi là tính toán đa bên: phương tiện để hai hoặc ba bên tính toán một giá trị mà dữ liệu của các bên đó là cần thiết, nhưng không cần phải thực sự chia sẻ dữ liệu đó. Một ví dụ thường được sử dụng là tính toán số phiếu bầu mà không cần biết ai đã bỏ phiếu cho ai.

Chúng ta sẽ thấy nhiều ví dụ hơn về bảo mật trong các hệ thống phân tán trong các chương sau. Với các giải thích ngắn gọn về cơ sở mật mã, chúng ta có thể thấy cách áp dụng bảo mật. Chúng ta sẽ sử dụng nhất quán các ký hiệu như trong Hình 1.4. Ngoài ra, các ví dụ về bảo mật có thể được bỏ qua cho đến khi nghiên cứu Chương 9.

Ký hiệu	Mô tả
KA, B	Khóa bí mật được chia sẻ bởi A và B
PKA	Khóa công khai của A
SKA	Khóa riêng tư (bí mật) của A
EK(dữ liệu)	Mã hóa dữ liệu bằng khóa EK (hoặc khóa K)
DK(dữ liệu)	Giải mã dữ liệu (đã mã hóa) bằng khóa DK (hoặc khóa K)
H(dữ liệu)	Băm dữ liệu được tính toán bằng hàm H

Hình 1.4: Ký hiệu cho các hệ thống mật mã được sử dụng trong cuốn sách này.

1.2.6 Khả năng mở rộng

Đối với nhiều người trong chúng ta, kết nối toàn cầu thông qua Internet cũng phổ biến như việc có thể gửi một gói hàng cho bất kỳ ai ở bất kỳ đâu trên thế giới. Hơn nữa, cho đến gần đây, chúng ta vẫn quen với việc có máy tính để bàn tư ở ng đối mạnh cho các ứng dụng văn phòng và lưu trữ, thì giờ đây chúng ta đang chứng kiến các ứng dụng và dịch vụ như vậy đang được đặt trong cái được gọi là "đám mây", dẫn đến sự gia tăng các thiết bị mạng nhỏ hơn nhiều như máy tính bảng hoặc thậm chí là máy tính xách tay chỉ có đám mây như Chromebook của Google. Với suy nghĩ này, khả năng mở rộng đã trở thành một trong những mục tiêu thiết kế quan trọng nhất đối với các nhà phát triển hệ thống phân tán.

Kích thước khả năng mở rộng

Khả năng mở rộng của một hệ thống có thể được đo lường theo ít nhất ba chiều khác nhau (xem [Neuman, 1994]):

Khả năng mở rộng quy mô: Một hệ thống có thể mở rộng quy mô, nghĩa là chúng ta có thể dễ dàng thêm nhiều người dùng và tài nguyên vào hệ thống mà không làm giảm đáng kể hiệu suất.

Khả năng mở rộng về mặt địa lý: Một hệ thống có khả năng mở rộng về mặt địa lý là hệ thống mà người dùng và tài nguyên có thể nằm cách xa nhau nhưng thực tế là độ trễ truyền thông có thể rất đáng kể nhưng lại hầu như không được chú ý.

Khả năng mở rộng quản lý: Một hệ thống có khả năng mở rộng quản lý là hệ thống vẫn có thể được quản lý dễ dàng ngay cả khi nó bao gồm nhiều tổ chức quản lý độc lập.

Chúng ta hãy xem xét kỹ hơn từng khía cạnh trong ba khía cạnh khả năng mở rộng này.

Khả năng mở rộng quy mô Khi một hệ thống cần mở rộng quy mô, cần giải quyết nhiều loại vấn đề khác nhau. Trước tiên, chúng ta hãy xem xét khả năng mở rộng quy mô. Nếu cần hỗ trợ nhiều người dùng hoặc tài nguyên hơn, chúng ta thường phải đối mặt với những hạn chế của các dịch vụ tập trung, mặc dù thường vì những lý do rất khác nhau. Ví dụ, nhiều dịch vụ được tập trung theo nghĩa là chúng được triển khai bởi một máy chủ duy nhất chạy trên một máy cụ thể trong hệ thống phân tán. Trong bối cảnh hiện đại hơn, chúng ta có thể có một nhóm máy chủ cộng tác được đặt cùng vị trí trên một cụm máy được ghép chặt chẽ được đặt vật lý tại cùng một vị trí. Vấn đề với sơ đồ này rất rõ ràng: máy chủ hoặc nhóm máy chủ có thể trở thành nút thắt cổ chai khi cần xử lý số lượng yêu cầu ngày càng tăng. Để minh họa cách điều này có thể xảy ra, chúng ta hãy giả sử rằng một dịch vụ được triển khai trên một máy chủ duy nhất. Trong trường hợp hợp đó, về cơ bản có ba nguyên nhân gốc rễ khiến trở thành nút thắt cổ chai:

- Khả năng tính toán bị giới hạn bởi CPU
- Khả năng lưu trữ,
- bao gồm tốc độ truyền I/O

- Mạng lưới giữa người dùng và dịch vụ tập trung

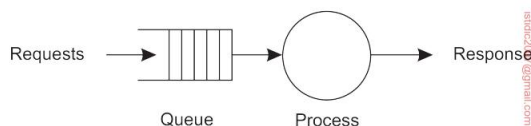
Trước tiên, hãy xem xét khả năng tính toán. Hãy tưởng tượng một dịch vụ tính toán các tuyến đường tối ưu có tính đến thông tin giao thông theo thời gian thực. Không khó để tưởng tượng rằng đây có thể chủ yếu là một dịch vụ bị ràng buộc bởi tính toán, cần vài (hàng chục) giây để hoàn thành một yêu cầu. Nếu chỉ có một máy duy nhất khả dụng, thì ngay cả một hệ thống cao cấp hiện đại cuối cùng cũng sẽ gặp vấn đề nếu số lượng yêu cầu tăng vượt quá một điểm nhất định.

Tương tự như vậy, nhưng vì những lý do khác nhau, chúng ta sẽ gặp phải vấn đề khi có một dịch vụ chủ yếu bị ràng buộc bởi I/O. Một ví dụ điển hình là một công cụ tìm kiếm tập trung được thiết kế kém. Vấn đề với các truy vấn tìm kiếm dựa trên nội dung là về cơ bản chúng ta cần phải khớp một truy vấn với toàn bộ tập dữ liệu. Ngay cả với các kỹ thuật lập chỉ mục tiên tiến, chúng ta vẫn có thể gặp phải vấn đề phải xử lý một lượng dữ liệu khổng lồ vượt quá dung lượng bộ nhớ chính của máy đang chạy dịch vụ. Do đó, phần lớn thời gian xử lý sẽ được xác định bởi tốc độ truy cập đĩa tương đối chậm và việc truyền dữ liệu giữa đĩa và bộ nhớ chính. Việc chỉ cần thêm nhiều đĩa hơn hoặc đĩa có tốc độ cao hơn sẽ không phải là giải pháp bền vững vì số lượng yêu cầu tiếp tục tăng.

Cuối cùng, mạng lưới giữa người dùng và dịch vụ cũng có thể là nguyên nhân gây ra khả năng mở rộng kém. Hãy tưởng tượng một dịch vụ video theo yêu cầu cần truyền phát video chất lượng cao đến nhiều người dùng. Một luồng video có thể dễ dàng yêu cầu băng thông từ 8 đến 10 Mbps, nghĩa là nếu một dịch vụ thiết lập kết nối điểm-điểm với khách hàng của mình, dịch vụ đó có thể sớm đạt đến giới hạn về dung lượng mạng của các đường truyền truyền ra của chính dịch vụ đó.

Có một số giải pháp để tấn công khả năng mở rộng quy mô, mà chúng tôi thảo luận bên dưới sau khi xem xét khả năng mở rộng về mặt địa lý và hành chính.

Lưu ý 1.5 (Nâng cao: Phân tích khả năng mở rộng quy mô)



Hình 1.5: Mô hình đơn giản của dịch vụ dưới dạng hệ thống xếp hàng.

Các vấn đề về khả năng mở rộng quy mô cho các dịch vụ tập trung có thể được phân tích chính thức bằng cách sử dụng lý thuyết xếp hàng và đưa ra một số giả định đơn giản hóa. Ở cấp độ khái niệm, một dịch vụ tập trung có thể được mô hình hóa như hệ thống xếp hàng đơn giản được hiển thị trong Hình 1.5: các yêu cầu được gửi đến dịch vụ, nơi chúng được xếp hàng cho đến khi có thông báo mới. Ngay khi quy trình có thể xử lý yêu cầu tiếp theo, nó sẽ lấy yêu cầu đó từ hàng đợi, thực hiện công việc của mình và tạo ra phản hồi. Chúng tôi phần lớn tuân theo Menasce và Almeida [2002] trong việc giải thích hiệu suất của một dịch vụ tập trung.

Thông thường, chúng ta có thể cho rằng hàng đợi có sức chứa vô hạn, nghĩa là không có giới hạn nào về số lượng yêu cầu có thể được chấp nhận để xử lý thêm. Nói một cách chính xác, điều này có nghĩa là tỷ lệ yêu cầu đến không bị ảnh hưởng bởi những gì hiện có trong hàng đợi hoặc đang được xử lý. Giả sử rằng tỷ lệ yêu cầu đến là λ yêu cầu mỗi giây và rằng khả năng xử lý của dịch vụ là μ yêu cầu mỗi giây, người ta có thể tính toán rằng phân số thời gian p_k có k yêu cầu trong hệ thống bằng:

$$\lambda p_k = 1 - \frac{\lambda}{\mu} \quad (1)$$

Nếu chúng ta định nghĩa việc sử dụng U của một dịch vụ là phần thời gian mà nó bận rộn, thì rõ ràng,

$$U = \sum_{k \geq 0} k p_k = (1 - U) \sum_{k \geq 0} k p_k = 1 - p_0 = \frac{\lambda}{\mu} \quad (2)$$

Sau đó chúng ta có thể tính toán số lượng yêu cầu trung bình N trong hệ thống như sau

$$N = \sum_{k \geq 0} k \cdot p_k = \sum_{k \geq 0} k \cdot (1 - U) \sum_{k \geq 0} k p_k = (1 - U) \sum_{k \geq 0} k^2 p_k = \frac{(1 - U)U}{(1 - U)^2} = \frac{U}{1 - U}.$$

Điều chúng tôi thực sự quan tâm là thời gian phản hồi R : mất bao lâu để dịch vụ xử lý một yêu cầu, bao gồm cả thời gian chờ trong hàng đợi.

Để đạt được mục đích đó, chúng ta cần thông lượng trung bình X . Xem xét rằng dịch vụ "bận" khi có ít nhất một yêu cầu đang được xử lý và điều này sau đó xảy ra với thông lượng là μ yêu cầu mỗi giây và trong một phần U của tổng thời gian, chúng ta có:

$$X = U \cdot \underbrace{\mu}_{\text{máy chủ đang hoạt động}} + (1 - U) \cdot \underbrace{0}_{\text{máy chủ nhàn rỗi}} = \frac{\lambda}{\mu} \cdot \mu = \lambda$$

Sử dụng công thức của Little [Trivedi, 2002], sau đó chúng ta có thể suy ra thời gian phản hồi là

$$R = \frac{N}{X} = \frac{S}{1 - U} \quad \frac{R}{S} = \frac{1}{1 - U}$$

trong đó $S = \frac{1}{\mu}$, thời gian dịch vụ thực tế. Lưu ý rằng nếu U nhỏ, tỷ lệ thời gian phản hồi trên dịch vụ gần bằng 1, nghĩa là yêu cầu được xử lý gần như ngay lập tức và ở tốc độ tối đa có thể. Tuy nhiên, ngay khi mức sử dụng tiến gần đến 1, chúng ta thấy tỷ lệ thời gian phản hồi trên máy chủ nhanh chóng tăng lên các giá trị rất cao, về cơ bản có nghĩa là hệ thống đang gần đến điểm dừng. Đây là nơi chúng ta thấy các vấn đề về khả năng mở rộng xuất hiện. Từ mô hình đơn giản này, chúng ta có thể thấy rằng giải pháp duy nhất là giảm thời gian dịch vụ S . Chúng tôi để lại bài tập cho người đọc khám phá cách có thể giảm S .

Khả năng mở rộng địa lý Khả năng mở rộng địa lý có những vấn đề riêng của nó. Một trong những lý do chính khiến việc mở rộng các hệ thống phân tán hiện có vẫn còn khó khăn

được thiết kế cho mạng cục bộ là nhiều mạng trong số đó dựa trên giao tiếp đồng bộ. Trong hình thức giao tiếp này, một bên yêu cầu dịch vụ, thường được gọi là máy khách, sẽ chặn cho đến khi máy chủ triển khai dịch vụ gửi lại phản hồi. Cụ thể hơn, chúng ta thường thấy một mẫu giao tiếp bao gồm nhiều tương tác máy khách-máy chủ, như trường hợp giao dịch cơ sở dữ liệu. Cách tiếp cận này thường hoạt động tốt trong mạng LAN, nơi giao tiếp giữa hai máy thường chỉ mất vài trăm micro giây. Tuy nhiên, trong hệ thống diện rộng, chúng ta cần cân nhắc rằng giao tiếp giữa các tiến trình có thể chậm hơn hàng trăm mili giây, chậm hơn ba cấp độ. Việc xây dựng các ứng dụng sử dụng giao tiếp đồng bộ trong hệ thống diện rộng đòi hỏi rất nhiều sự cẩn thận (và không chỉ là một chút kiên nhẫn), đặc biệt là với mẫu tương tác phong phú giữa máy khách và máy chủ.

Một vấn đề khác cản trở khả năng mở rộng địa lý là giao tiếp trong mạng diện rộng về bản chất kém tin cậy hơn nhiều so với mạng cục bộ. Ngoài ra, chúng ta thường phải xử lý băng thông hạn chế. Hiệu ứng là các giải pháp được phát triển cho mạng cục bộ không phải lúc nào cũng dễ dàng chuyển sang hệ thống diện rộng. Một ví dụ điển hình là phát trực tuyến video. Trong mạng gia đình, ngay cả khi chỉ có liên kết không dây, việc đảm bảo luồng video chất lượng cao ổn định, nhanh chóng từ máy chủ phục vụ đến màn hình khá đơn giản. Chỉ cần đặt máy chủ đó ở xa và sử dụng kết nối TCP tiêu chuẩn với màn hình chắc chắn sẽ thất bại: giới hạn băng thông sẽ ngay lập tức xuất hiện, nhưng việc duy trì cùng mức độ tin cậy cũng dễ gây đau đầu.

Một vấn đề khác nảy sinh khi các thành phần nằm cách xa nhau là thực tế là các hệ thống diện rộng thường chỉ có rất ít tiện ích cho giao tiếp đa điểm. Ngược lại, các mạng cục bộ thường hỗ trợ các cơ chế phát sóng hiệu quả. Các cơ chế như vậy đã được chứng minh là cực kỳ hữu ích để khám phá các thành phần và dịch vụ, điều này rất cần thiết theo quan điểm quản lý. Trong các hệ thống diện rộng, chúng ta cần phát triển các dịch vụ riêng biệt, chẳng hạn như dịch vụ đặt tên và dịch vụ thư mục, nơi có thể gửi các truy vấn. Đến lượt mình, các dịch vụ hỗ trợ này cũng cần có khả năng mở rộng và thường không có giải pháp rõ ràng nào tồn tại như chúng ta sẽ gặp trong các chương sau.

Khả năng mở rộng quản trị Cuối cùng, một câu hỏi khó và thường mở là làm thế nào để mở rộng một hệ thống phân tán trên nhiều miền quản trị độc lập. Một vấn đề lớn cần được giải quyết là các chính sách xung đột liên quan đến việc sử dụng tài nguyên (và thanh toán), quản lý và bảo mật.

Ví dụ, trong nhiều năm, các nhà khoa học đã tìm kiếm giải pháp để chia sẻ thiết bị (thường đắt tiền) của họ trong cái được gọi là lưu ý tính toán. Trong các lưu ý này, một hệ thống phi tập trung toàn cầu được xây dựng như một liên bang các hệ thống phân tán cục bộ, cho phép một chương trình chạy trên máy tính tại tổ chức A truy cập trực tiếp vào các tài nguyên tại tổ chức B.

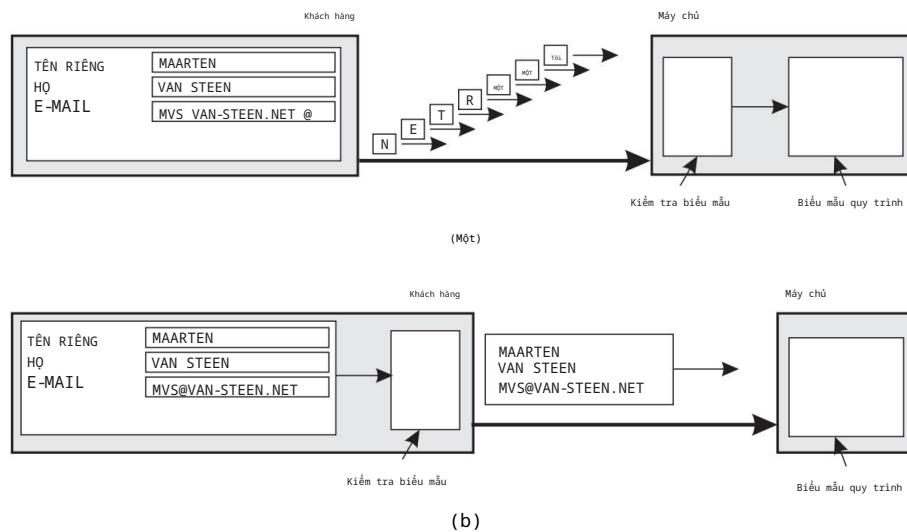
Nhiều thành phần của một hệ thống phân tán nằm trong một miền duy nhất thường có thể được người dùng hoạt động trong cùng miền đó tin cậy. Trong những trường hợp như vậy, quản trị hệ thống có thể đã thử nghiệm và chứng nhận các ứng dụng và có thể đã thực hiện các biện pháp đặc biệt để đảm bảo rằng các thành phần đó không thể bị can thiệp. Về bản chất, người dùng tin tưởng quản trị viên hệ thống của họ. Tuy nhiên, sự tin tưởng này không tự nhiên mở rộng ra ngoài ranh giới miền.

Nếu một hệ thống phân tán mở rộng sang một miền khác, cần phải thực hiện hai loại biện pháp bảo mật. Đầu tiên, hệ thống phân tán phải tự bảo vệ mình khỏi các cuộc tấn công độc hại từ miền mới. Ví dụ, người dùng từ miền mới có thể chỉ có quyền truy cập đọc vào hệ thống tệp trong miền gốc của nó. Tư tưởng tự như vậy, các tiện ích như bộ thiết lập hình ảnh đất tiền hoặc máy tính hiệu suất cao có thể không được cung cấp cho người dùng trái phép. Thứ hai, miền mới phải tự bảo vệ mình khỏi các cuộc tấn công độc hại từ hệ thống phân tán. Một ví dụ điển hình là việc tải xuống các chương trình, chẳng hạn như trong trường hợp học liên bang. Về cơ bản, miền mới không biết phải mong đợi gì từ mã nguồn ngoài như vậy. Vấn đề, như chúng ta sẽ thấy trong Chương 9, là làm thế nào để thực thi những hạn chế đó.

Là một ví dụ phản biện về các hệ thống phân tán trải dài trên nhiều miền quản trị mà rõ ràng không gặp phải các vấn đề về khả năng mở rộng quản trị, hãy xem xét các mạng ngang hàng chia sẻ tệp hiện đại. Trong những trường hợp này, người dùng cuối chỉ cần cài đặt một chương trình triển khai các chức năng tìm kiếm và tải xuống phân tán và trong vòng vài phút có thể bắt đầu tải xuống các tệp. Các ví dụ khác bao gồm các ứng dụng ngang hàng cho điện thoại qua Internet như các hệ thống Skype cũ hơn [Baset và Schulzrinne, 2006] và (lại cũ hơn) các ứng dụng phát trực tuyến âm thanh được hỗ trợ ngang hàng như Spotify [Kreitz và Niemelä, 2010]. Một ứng dụng hiện đại hơn (vẫn chưa chứng minh được khả năng mở rộng) là chuỗi khối. Điểm chung của các hệ thống phi tập trung này là người dùng cuối, chứ không phải các thực thể quản trị, hợp tác để duy trì hệ thống hoạt động. Tốt nhất, các tổ chức quản trị cơ bản như Nhà cung cấp dịch vụ Internet (ISP) có thể kiểm soát lưu lượng mạng mà các hệ thống ngang hàng này gây ra.

Kỹ thuật mở rộng

Sau khi thảo luận về một số vấn đề về khả năng mở rộng, chúng ta sẽ đi đến câu hỏi về cách giải quyết những vấn đề đó nói chung. Trong hầu hết các trường hợp, các vấn đề về khả năng mở rộng trong các hệ thống phân tán xuất hiện dưới dạng các vấn đề về hiệu suất do khả năng hạn chế của máy chủ và mạng. Việc đơn giản là cải thiện khả năng của chúng (ví dụ, bằng cách tăng bộ nhớ, nâng cấp CPU hoặc thay thế các mô-đun mạng) thường là một giải pháp, được gọi là mở rộng quy mô. Khi nói đến việc mở rộng quy mô, tức là mở rộng hệ thống phân tán về cơ bản bằng cách triển khai nhiều máy hơn, về cơ bản chỉ có ba kỹ thuật mà chúng ta có thể áp dụng: ẩn độ trễ giao tiếp, phân phối công việc và sao chép (xem thêm Neuman [1994]).



Hình 1.6: Sự khác biệt giữa việc để (a) máy chủ hoặc (b) máy khách kiểm tra biểu mẫu khi chúng đang đư ợc điền.

Ảnh ộ trễ giao tiếp Ảnh ộ trễ giao tiếp có thể áp dụng trong trư ờng hợp khả năng mở rộng địa lý. Ý tứ ờng cơ bản rất đơn giản: cố gắng tránh chờ phản hồi cho các yêu cầu dịch vụ từ xa càng nhiều càng tốt.

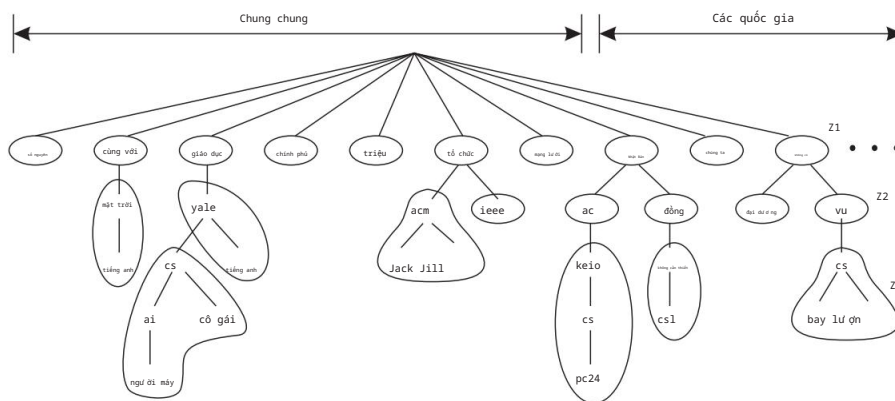
Ví dụ, khi một dịch vụ đã đư ợc yêu cầu tại một máy từ xa, một giải pháp thay thế cho việc chờ phản hồi từ máy chủ là thực hiện công việc hữu ích khác ở phía ngư ời yêu cầu. Về cơ bản, điều này có nghĩa là xây dựng ứng dụng yêu cầu theo cách mà nó chỉ sử dụng giao tiếp không đồng bộ. Khi có phản hồi, ứng dụng sẽ bị ngắt và một trình xử lý đặc biệt đư ợc gọi để hoàn tất yêu cầu đã phát hành trư ớc đó. Giao tiếp không đồng bộ thư ờng có thể đư ợc sử dụng trong các hệ thống xử lý hàng loạt và các ứng dụng song song, trong đó các tác vụ độc lập có thể đư ợc lên lịch để thực hiện trong khi một tác vụ khác đang chờ giao tiếp hoàn tất. Ngoài ra, một luồng điều khiển mới có thể đư ợc bắt đầu để thực hiện yêu cầu. Mặc dù nó chặn việc chờ phản hồi, các luồng khác trong quy trình có thể tiếp tục.

Tuy nhiên, có nhiều ứng dụng không thể sử dụng hiệu quả giao tiếp không đồng bộ. Ví dụ, trong các ứng dụng tư ơng tác khi ngư ời dùng gửi yêu cầu, họ thư ờng không có việc gì tốt hơn để làm ngoài việc chờ câu trả lời. Trong những trư ờng hợp như vậy, một giải pháp tốt hơn nhiều là giảm giao tiếp tổng thể, ví dụ, bằng cách di chuyển một phần tính toán thư ờng đư ợc thực hiện tại máy chủ sang quy trình máy khách yêu cầu dịch vụ. Một trư ờng hợp điển hình mà cách tiếp cận này hiệu quả là truy cập cơ sở dữ liệu bằng biểu mẫu.

Có thể điền vào biểu mẫu bằng cách gửi tin nhắn riêng cho từng trư ờng và chờ máy chủ xác nhận, như thể hiện trong Hình 1.6(a). Ví dụ, máy chủ có thể kiểm tra lỗi cú pháp trư ớc khi chấp nhận mục nhập.

Một giải pháp tốt hơn nhiều là gửi mã để điền vào biểu mẫu và có thể kiểm tra các mục nhập, cho khách hàng và yêu cầu khách hàng trả lại một bản đã hoàn thành hình thức, như thể hiện trong Hình 1.6(b). Cách tiếp cận này của mã vận chuyển được sử dụng rộng rãi được hỗ trợ bởi Web thông qua JavaScript.

Phân vùng và phân phối Một kỹ thuật mở rộng quan trọng khác là phân vùng và phân phối, bao gồm việc lấy một thành phần hoặc tài nguyên khác, chia nó thành các phần nhỏ hơn và sau đó trải các phần đó ra khắp nơi i hệ thống. Một ví dụ tốt về phân vùng và phân phối là Internet Hệ thống tên miền (DNS). Không gian tên DNS được tổ chức theo thứ bậc thành một cây các miền, được chia thành các vùng không chồng lấn, như được hiển thị cho DNS gốc trong Hình 1.7. Các tên trong mỗi vùng là được xử lý bởi một máy chủ tên duy nhất. Không đi sâu vào quá nhiều chi tiết ngay bây giờ (chúng ta sẽ quay lại DNS một cách chi tiết trong Chương 6), người ta có thể nghĩ đến từng tên được dẫn là tên của một máy chủ trên Internet và do đó được liên kết với một mạng địa chỉ của máy chủ đó. Về cơ bản, giải quyết một tên có nghĩa là trả lại mạng địa chỉ của máy chủ liên quan. Ví dụ, hãy xem xét tên flits.cs.vu.nl. Để giải quyết tên này, trước tiên nó được chuyển đến máy chủ của vùng Z1 (xem Hình 1.7) trả về địa chỉ của máy chủ cho vùng Z2, nơi phần còn lại của tên, flits.cs.vu, có thể được chuyển giao. Máy chủ cho Z2 sẽ trả về địa chỉ của máy chủ cho vùng Z3, có khả năng xử lý phần cuối của tên, và sẽ trả về địa chỉ của máy chủ liên quan.



Hình 1.7: Ví dụ về việc chia không gian tên DNS (gốc) thành các vùng.

Những ví dụ này minh họa cách dịch vụ đặt tên được cung cấp bởi DNS là được phân phối trên nhiều máy, do đó tránh được việc một máy chủ duy nhất phải xử lý tất cả các yêu cầu giải quyết tên.

Một ví dụ khác, hãy xem xét World Wide Web. Đối với hầu hết người dùng, Web có vẻ như là một hệ thống thông tin khổng lồ dựa trên tài liệu, trong đó mỗi tài liệu có tên riêng duy nhất dưới dạng URL. Về mặt khái niệm,

thậm chí có thể xuất hiện như thể chỉ có một máy chủ duy nhất. Tuy nhiên, Web được phân vùng vật lý và phân phối trên một vài trăm triệu máy chủ, mỗi máy chủ thường xử lý một số Trang web hoặc các phần của Trang web. Tên của máy chủ xử lý một tài liệu được mã hóa thành URL của tài liệu đó. Chỉ vì sự phân phối các tài liệu này mà Web có khả năng mở rộng đến kích thước hiện tại của nó. Tuy nhiên, lưu ý rằng việc tìm ra có bao nhiêu máy chủ cung cấp các dịch vụ dựa trên Web là hầu như không thể: Một Trang web ngày nay còn hơn cả một vài tài liệu Web tĩnh.

Sao chép Khi xem xét các vấn đề về khả năng mở rộng thường xuất hiện dưới dạng suy giảm hiệu suất, nhìn chung, tốt nhất là sao chép các thành phần hoặc tài nguyên, v.v. trên toàn bộ hệ thống phân tán. Sao chép không chỉ làm tăng tính khả dụng mà còn giúp cân bằng tải giữa các thành phần, dẫn đến hiệu suất tốt hơn. Hơn nữa, trong các hệ thống phân tán rộng rãi về mặt địa lý, việc có một bản sao gần đó có thể ẩn đi nhiều vấn đề về độ trễ truyền thông đã đề cập trước đó.

Caching là một dạng sao chép đặc biệt, mặc dù sự khác biệt giữa hai dạng này thường khó thực hiện hoặc thậm chí là nhân tạo. Giống như trữ ứng hợp sao chép, caching dẫn đến việc tạo một bản sao của một tài nguyên, thường ở gần máy khách đang truy cập tài nguyên đó. Tuy nhiên, trái ngược với sao chép, caching là quyết định do máy khách của một tài nguyên đưa ra chứ không phải do chủ sở hữu của tài nguyên đó.

Có một nhược điểm nghiêm trọng đối với việc lưu trữ đệm và sao chép có thể ảnh hưởng xấu đến khả năng mở rộng. Vì hiện tại chúng ta có nhiều bản sao của một tài nguyên, việc sửa đổi một bản sao sẽ khiến bản sao đó khác với các bản sao khác. Do đó, việc lưu trữ đệm và sao chép dẫn đến các vấn đề về tính nhất quán.

Mức độ không nhất quán có thể được chấp nhận phụ thuộc vào cách sử dụng tài nguyên. Ví dụ, nhiều người dùng Web thấy chấp nhận được khi trình duyệt của họ trả về một tài liệu được lưu trong bộ nhớ đệm mà tính hợp lệ của nó chưa được kiểm tra trong vài phút qua. Tuy nhiên, cũng có nhiều trữ ứng hợp cần đáp ứng các đảm bảo nhất quán mạnh, chẳng hạn như trong trữ ứng hợp sàn giao dịch chứng khoán điện tử và đấu giá. Vấn đề với nhất quán mạnh là bản cập nhật phải được truyền ngay lập tức đến tất cả các bản sao khác. Hơn nữa, nếu hai bản cập nhật xảy ra đồng thời, trữ ứng cũng yêu cầu các bản cập nhật được xử lý theo cùng một thứ tự ở mọi nơi, dẫn đến vấn đề sắp xếp toàn cục. Tệ hơn nữa, việc kết hợp tính nhất quán với các đặc tính mong muốn như tính khả dụng có thể là điều không thể, như chúng ta sẽ thảo luận trong Chương 8.

Do đó, sao chép thường đòi hỏi một số cơ chế đồng bộ hóa toàn cục. Thật không may, các cơ chế như vậy cực kỳ khó hoặc thậm chí không thể triển khai theo cách có thể mở rộng quy mô, nếu chỉ riêng vì độ trễ mạng có giới hạn dưới tự nhiên. Do đó, việc mở rộng quy mô bằng sao chép có thể đưa ra các giải pháp khác vốn không thể mở rộng quy mô. Chúng tôi sẽ quay lại vấn đề sao chép và tính nhất quán một cách chi tiết trong Chương 7.

Thảo luận Khi xem xét các kỹ thuật mở rộng quy mô này, người ta có thể lập luận rằng khả năng mở rộng quy mô là vấn đề ít gây ra vấn đề nhất theo quan điểm kỹ thuật. Thông thường, việc tăng công suất của máy sẽ cứu vãn được tình hình, mặc dù có lẽ phải trả một khoản chi phí tiền tệ cao. Khả năng mở rộng quy mô theo địa lý là một vấn đề khó khăn hơn nhiều, vì độ trễ mạng tự nhiên bị ràng buộc từ bên dưới. Do đó, chúng ta có thể buộc phải sao chép dữ liệu đến các vị trí gần nơi khách hàng ở, dẫn đến các vấn đề về duy trì tính nhất quán của các bản sao. Thực tế cho thấy việc kết hợp các kỹ thuật phân phối, sao chép và lưu trữ đệm với các hình thức nhất quán khác nhau thường dẫn đến các giải pháp chấp nhận được. Cuối cùng, khả năng mở rộng quy mô hành chính có vẻ là vấn đề khó giải quyết nhất, một phần vì chúng ta cần giải quyết các vấn đề phi kỹ thuật, chẳng hạn như chính trị của các tổ chức và sự hợp tác của con người. Việc giới thiệu và hiện đang sử dụng rộng rãi công nghệ ngang hàng đã chứng minh thành công những gì có thể đạt được nếu người dùng cuối được kiểm soát [Lua và cộng sự, 2005; Oram, 2001]. Tuy nhiên, rõ ràng là các mạng ngang hàng không phải là giải pháp chung cho mọi vấn đề về khả năng mở rộng quy mô hành chính.

1.3 Phân loại đơn giản các hệ thống phân tán

Chúng ta đã thảo luận về hệ thống phân tán so với hệ thống phi tập trung, tuy nhiên, cũng hữu ích khi phân loại hệ thống phân tán theo mục đích phát triển và sử dụng. Chúng ta phân biệt giữa các hệ thống được phát triển cho điện toán (hiệu suất cao), cho xử lý thông tin chung và các hệ thống được phát triển cho điện toán phổ biến, tức là cho "Internet vạn vật".

Giống như nhiều phân loại khác, ranh giới giữa ba loại này không nghiêm ngặt và có thể dễ dàng nghĩ ra sự kết hợp.

1.3.1 Điện toán phân tán hiệu suất cao

Một lớp quan trọng của hệ thống phân tán là lớp được sử dụng cho các tác vụ tính toán hiệu suất cao. Nói một cách đại khái, người ta có thể phân biệt giữa hai nhóm con. Trong điện toán cụm, phần cứng cơ bản bao gồm một tập hợp các nút tính toán tương tự, được kết nối với nhau bằng mạng tốc độ cao, thường nằm cạnh một mạng cục bộ phổ biến hơn để điều khiển các nút.

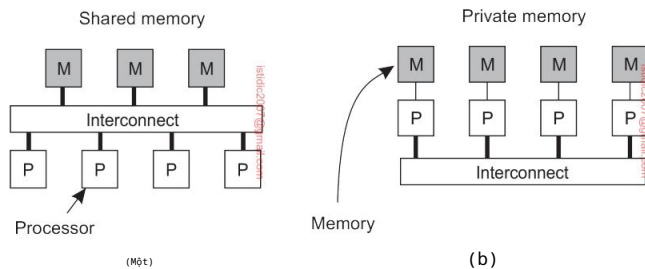
Ngoài ra, mỗi nút thường chạy cùng một hệ điều hành.

Tình hình trở nên rất khác trong trường hợp điện toán lưu trữ. Nhóm con này bao gồm các hệ thống phi tập trung thường được xây dựng như một liên bang các hệ thống máy tính, trong đó mỗi hệ thống có thể nằm trong một miền quản trị khác nhau và có thể rất khác nhau khi nói đến phần cứng, phần mềm và công nghệ mạng được triển khai.

Lưu ý 1.6 (Thông tin thêm: Xử lý song song)

Máy tính hiệu suất cao ít nhiều bắt đầu với sự ra đời của máy đa bộ xử lý. Trong trường hợp này, nhiều CPU được tổ chức theo cách mà tất cả chúng đều có thể truy cập vào cùng một bộ nhớ vật lý, như thể hiện trong Hình 1.8(a). Ngược lại, trong hệ thống nhiều máy tính, nhiều máy tính được kết nối thông qua mạng và không có chia sẻ bộ nhớ chính, như thể hiện trong Hình 1.8(b).

Mô hình bộ nhớ chia sẻ tỏ ra rất tiện lợi trong việc cải thiện hiệu suất của chương trình và tương đối dễ lập trình.



Hình 1.8: So sánh giữa (a) kiến trúc đa bộ xử lý và (b) kiến trúc đa máy tính.

Bản chất của nó là nhiều luồng điều khiển đang thực thi cùng một lúc, trong khi tất cả các luồng đều có quyền truy cập vào dữ liệu được chia sẻ. Quyền truy cập vào dữ liệu đó được kiểm soát thông qua các cơ chế đồng bộ hóa được hiểu rõ như semaphore (xem Ben-Ari [2006] hoặc Herlihy et al. [2021] để biết thêm thông tin về việc phát triển các chương trình song song). Thật không may, mô hình này không dễ dàng mở rộng quy mô: cho đến nay, các máy đã được phát triển trong đó chỉ có vài chục (và đôi khi là hàng trăm) CPU có quyền truy cập hiệu quả vào bộ nhớ được chia sẻ. Ở một mức độ nhất định, chúng ta đang thấy những hạn chế tương tự đối với bộ xử lý đa lõi.

Để khắc phục những hạn chế của hệ thống bộ nhớ chia sẻ, điện toán hiệu suất cao đã chuyển sang hệ thống bộ nhớ phân tán. Sự thay đổi này cũng có nghĩa là nhiều chương trình phải sử dụng truyền thông điệp thay vì sửa đổi dữ liệu chia sẻ như một phương tiện tiện giao tiếp và đồng bộ hóa giữa các luồng. Thật không may, các mô hình truyền thông điệp đã chứng minh là khó hơn nhiều và dễ xảy ra lỗi hơn so với các mô hình lập trình bộ nhớ chia sẻ. Vì lý do này, đã có nhiều nghiên cứu đáng kể trong nỗ lực xây dựng cái gọi là máy tính đa bộ nhớ chia sẻ phân tán, hay đơn giản là hệ thống DSM [Amza et al., 1996].

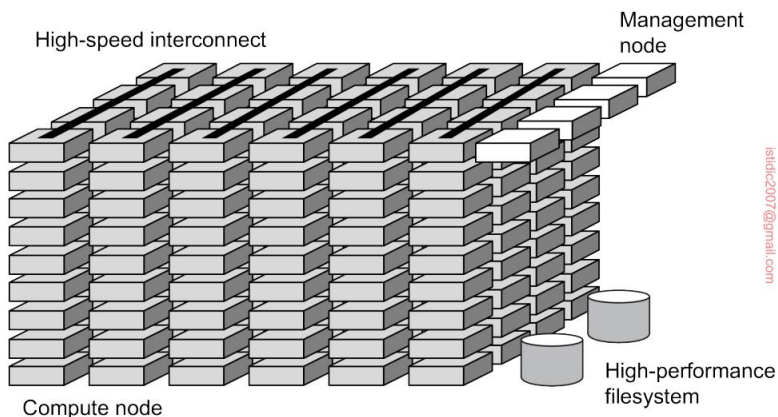
Về bản chất, hệ thống DSM cho phép bộ xử lý định vị một vị trí bộ nhớ tại một máy tính khác như thể đó là bộ nhớ cục bộ. Điều này có thể đạt được bằng cách sử dụng các kỹ thuật hiện có có sẵn cho hệ điều hành, ví dụ, bằng cách ánh xạ tất cả các trang bộ nhớ chính của các bộ xử lý khác nhau vào một không gian địa chỉ ảo duy nhất. Bất cứ khi nào bộ xử lý A định vị một trang nằm tại bộ xử lý B khác, lỗi trang xảy ra tại A cho phép hệ điều hành tại A lấy nội dung của trang được tham chiếu tại B theo cùng cách mà nó thường lấy nội dung đó cục bộ từ đĩa. Đồng thời, bộ xử lý B sẽ được thông báo rằng hiện tại không thể truy cập trang đó.

Việc mô phỏng các hệ thống bộ nhớ chia sẻ sử dụng nhiều máy tính cuối cùng đã phải từ bỏ vì hiệu suất không bao giờ có thể đáp ứng được kỳ vọng của các lập trình viên, những người muốn sử dụng các mô hình truyền thông điệp phức tạp hơn nhiều như hiệu suất tốt hơn (có thể dự đoán được). Một tác dụng phụ quan trọng của việc khám phá ranh giới phần cứng-phần mềm của xử lý song song là hiểu biết sâu sắc về các mô hình nhất quán, mà chúng tôi sẽ quay lại chi tiết trong Chương 7.

Máy tính cụm

Hệ thống điện toán cụm trở nên phổ biến khi tỷ lệ giá/hiệu suất của máy tính cá nhân và máy trạm được cải thiện. Đến một thời điểm nhất định, việc xây dựng một siêu máy tính sử dụng công nghệ có sẵn bằng cách chỉ cần kết nối một bộ sưu tập các máy tính tương đối đơn giản trong một mạng tốc độ cao trở nên hấp dẫn về mặt tài chính và kỹ thuật. Trong hầu hết mọi trường hợp, điện toán cụm được sử dụng để lập trình song song, trong đó một chương trình duy nhất (chuyên sâu về tính toán) được chạy song song trên nhiều máy. Nguyên tắc của tổ chức này được thể hiện trong Hình 1.9.

Kiểu điện toán hiệu suất cao này đã phát triển đáng kể. Như đã thảo luận rộng rãi bởi Gerofi et al. [2019], sự phát triển của siêu máy tính được tổ chức thành cụm đã đạt đến điểm mà chúng ta thấy các cụm có hơn 100.000 CPU, với mỗi CPU có 8 hoặc 16 lõi. Có nhiều mạng. Quan trọng nhất là mạng được hình thành bởi các kết nối tốc độ cao chuyên dụng giữa các nút khác nhau (nói cách khác, thứ ông không có thứ gọi là mạng tốc độ cao dùng chung cho các phép tính). Một mạng quản lý riêng biệt, cũng như các nút, được sử dụng để giám sát và kiểm soát



Hình 1.9: Một ví dụ về hệ thống máy tính cụm (trích từ [Gerofi et al., 2019].)

tổ chức và hiệu suất của toàn bộ hệ thống. Ngoài ra, một hệ thống tệp hoặc cơ sở dữ liệu hiệu suất cao đặc biệt được sử dụng, một lần nữa với mạng cục bộ chuyên dụng của riêng nó. Hình 1.9 không hiển thị thêm thiết bị, đặc biệt là I/O tốc độ cao cũng như các tiện ích mạng để truy cập và truyền thông từ xa.

Một nút quản lý thư ờng chịu trách nhiệm thu thập các công việc từ người dùng, sau đó phân phối các tác vụ liên quan giữa các nút tính toán khác nhau. Trong thực tế, một số nút quản lý được sử dụng khi xử lý các cụm rất lớn. Do đó, một nút quản lý thực sự chạy phần mềm cần thiết để thực hiện các chương trình và quản lý cụm, trong khi các nút tính toán được trang bị hệ điều hành tiêu chuẩn mở rộng với các chức năng thông thư ờng cho giao tiếp, lưu trữ, khả năng chịu lỗi, v.v.

Một sự phát triển thú vị, như đã giải thích trong Gerofi et al. [2019], là vai trò của hệ điều hành. Có một xu hướng rõ ràng là giảm thiểu hệ điều hành thành các hạt nhân nhẹ, về cơ bản là đảm bảo chi phí thấp nhất có thể. Một nhược điểm là các hệ điều hành như vậy trở nên chuyên biệt hóa cao và được tinh chỉnh theo phần cứng cơ bản. Sự chuyên biệt hóa này ảnh hưởng đến khả năng tương thích hoặc tính mở. Để bù đắp, hiện chúng ta đang dần thấy cái gọi là các chương pháp tiếp cận đa hạt nhân, trong đó một hệ điều hành hoàn chỉnh hoạt động bên cạnh một hạt nhân nhẹ, do đó đạt được điều tốt nhất của cả hai thể giới. Sự kết hợp này cũng là cần thiết vì ngày càng thư ờng xuyên hơn, một nút tính toán hiệu suất cao được yêu cầu chạy nhiều tác vụ độc lập cùng lúc. Hiện tại, 95% tất cả các máy tính hiệu suất cao đều chạy các hệ thống dựa trên Linux; các chương pháp tiếp cận đa hạt nhân được phát triển cho CPU đa lõi, với hầu hết các lõi chạy một hạt nhân nhẹ và lõi còn lại chạy một hệ thống Linux thông thư ờng. Theo cách này, các phát triển mới như trình chứa (chúng ta sẽ thảo luận trong Chương 3) cũng có thể được hỗ trợ. Các tác động đối với hiệu suất tính toán vẫn cần phải được xem xét.

Điện toán lưu ới

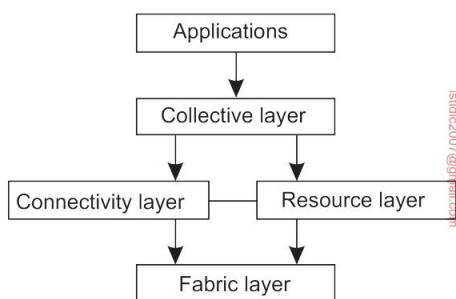
Một tính năng đặc trưng của điện toán cụm truyền thống là tính đồng nhất của nó. Trong hầu hết các trường hợp, các máy tính trong một cụm đều giống nhau, có cùng hệ điều hành và đều được kết nối thông qua cùng một mạng.

Tuy nhiên, như chúng ta vừa thảo luận, có một xu hướng liên tục hướng tới nhiều kiến trúc lai hơn trong đó các nút được cấu hình cụ thể cho các tác vụ nhất định. Sự đa dạng này thậm chí còn phổ biến hơn trong các hệ thống điện toán lưu ới: không có giả định nào được đưa ra liên quan đến sự giống nhau của phần cứng, hệ điều hành, mạng, miền quản trị, chính sách bảo mật, v.v. [Rajaraman, 2016].

Một vấn đề quan trọng trong hệ thống điện toán lưu ới là các nguồn lực từ nhiều tổ chức khác nhau được tập hợp lại để cho phép một nhóm người từ nhiều tổ chức khác nhau cộng tác, thực sự hình thành nên một liên bang các hệ thống. Sự hợp tác như vậy được thực hiện dưới hình thức một tổ chức ảo. Các quy trình thuộc cùng một tổ chức ảo có quyền truy cập vào

tài nguyên được cung cấp cho tổ chức đó. Thông thường, tài nguyên bao gồm máy chủ tính toán (bao gồm siêu máy tính, có thể được triển khai dưới dạng máy tính cụm), cơ sở lưu trữ và cơ sở dữ liệu. Ngoài ra, các thiết bị mạng đặc biệt như kính thiên văn, cảm biến, v.v. cũng có thể được cung cấp.

Do bản chất của nó, phần lớn phần mềm để thực hiện điện toán lưu trữ phát triển xung quanh việc cung cấp quyền truy cập vào các tài nguyên từ các miền quản trị khác nhau và chỉ cho những người dùng và ứng dụng thuộc về một tổ chức ảo cụ thể. Vì lý do này, trọng tâm thường là các vấn đề về kiến trúc. Một kiến trúc ban đầu được Foster và cộng sự [2001] đề xuất được thể hiện trong Hình 1.10, vẫn tạo thành cơ sở cho nhiều hệ thống điện toán lưu trữ.



Hình 1.10: Kiến trúc phân lớp cho hệ thống điện toán lưu trữ.

Kiến trúc bao gồm bốn lớp. Lớp vấp váp thấp nhất cung cấp giao diện cho các tài nguyên cục bộ tại một địa điểm cụ thể. Lưu ý rằng các giao diện này được thiết kế riêng để cho phép chia sẻ tài nguyên trong một tổ chức ảo. Thông thường, chúng sẽ cung cấp các chức năng để truy vấn trạng thái và khả năng của một tài nguyên, cùng với các chức năng để quản lý tài nguyên thực tế (ví dụ: khóa tài nguyên).

Lớp kết nối bao gồm các giao thức truyền thông để hỗ trợ các giao dịch lưu trữ trải dài trên nhiều tài nguyên. Ví dụ, cần có các giao thức để truyền dữ liệu giữa các tài nguyên hoặc chỉ để truy cập một tài nguyên từ một vị trí từ xa. Ngoài ra, lớp kết nối sẽ chứa các giao thức bảo mật để xác thực người dùng và tài nguyên. Lưu ý rằng trong nhiều trường hợp, người dùng là con người không được xác thực; thay vào đó, các chương trình hoạt động thay mặt cho người dùng sẽ được xác thực. Theo nghĩa này, việc ủy quyền các quyền từ người dùng cho các chương trình là một chức năng quan trọng cần được hỗ trợ trong lớp kết nối. Chúng ta quay lại với việc ủy quyền khi thảo luận về bảo mật trong các hệ thống phân tán ở Chương 9.

Lớp tài nguyên chịu trách nhiệm quản lý một tài nguyên duy nhất. Nó sử dụng các chức năng do lớp kết nối cung cấp và gọi trực tiếp các giao diện do lớp vấp váp cung cấp. Ví dụ, lớp này sẽ cung cấp các chức năng để lấy thông tin cấu hình trên một tài nguyên cụ thể hoặc nói chung là để thực hiện các hoạt động cụ thể như tạo quy trình hoặc đọc dữ liệu.

do đó, lớp tài nguyên được coi là chịu trách nhiệm kiểm soát truy cập và do đó sẽ dựa vào xác thực được thực hiện như một phần của lớp kết nối.

Lớp tiếp theo trong hệ thống phân cấp là lớp tập thể. Lớp này xử lý việc truy cập vào nhiều tài nguyên và thư ờng bao gồm các dịch vụ để khám phá tài nguyên, phân bổ và lập lịch các tác vụ trên nhiều tài nguyên, sao chép dữ liệu, v.v. Không giống như lớp kết nối và lớp tài nguyên, mỗi lớp bao gồm một bộ giao thức chuẩn tư ờng đối nhỏ, lớp tập thể có thể bao gồm nhiều giao thức phản ánh phổ rộng các dịch vụ mà nó có thể cung cấp cho một tổ chức ảo.

Cuối cùng, lớp ứng dụng bao gồm các ứng dụng hoạt động trong một tổ chức ảo và sử dụng môi trường điện toán lưu trữ.

Thông thư ờng, lớp tập thể, kết nối và tài nguyên tạo thành cốt lõi của cái có thể được gọi là lớp phần mềm trung gian lưu trữ. Các lớp này cùng nhau cung cấp quyền truy cập và quản lý các tài nguyên có khả năng phân tán trên nhiều trang web.

Một quan sát quan trọng từ góc độ phần mềm trung gian là trong điện toán lưu trữ, khái niệm về một site (hoặc đơn vị hành chính) là phổ biến. Sự phổ biến này được nhấn mạnh bởi sự chuyển dịch dần dần sang kiến trúc hướng dịch vụ trong đó các site cung cấp quyền truy cập vào các lớp khác nhau thông qua một bộ sưu tập các dịch vụ Web [Joseph và cộng sự, 2004]. Cho đến nay, điều này đã dẫn đến định nghĩa về một kiến trúc thay thế được gọi là Kiến trúc dịch vụ lưu trữ mở (OGSA) [Foster và cộng sự, 2006]. OGSA dựa trên các ý tưởng ban đầu do Foster và cộng sự [2001] xây dựng, tuy nhiên, việc trải qua một quá trình chuẩn hóa khiến nó trở nên phức tạp, nói một cách nhẹ nhàng nhất. Các triển khai OGSA thư ờng tuân theo các tiêu chuẩn dịch vụ Web.

1.3.2 Hệ thống thông tin phân tán

Một lớp quan trọng khác của hệ thống phân tán được tìm thấy trong các tổ chức phải đối mặt với vô số ứng dụng mạng, nhưng khả năng tư ờng tác hóa ra lại là một trải nghiệm đau đớn. Nhiều giải pháp phần mềm trung gian hiện có là kết quả của việc làm việc với cơ sở hạ tầng trong đó dễ dàng tích hợp các ứng dụng vào hệ thống thông tin toàn doanh nghiệp [Alonso và cộng sự, 2004; Bernstein, 1996; Hohpe và Woolf, 2004].

Chúng ta có thể phân biệt một số cấp độ mà tích hợp có thể diễn ra. Thông thư ờng, một ứng dụng mạng chỉ bao gồm một máy chủ chạy ứng dụng đó (thư ờng bao gồm một cơ sở dữ liệu) và cung cấp cho các chức năng trình từ xa, được gọi là máy khách. Các máy khách như vậy gửi yêu cầu đến máy chủ để thực hiện một hoạt động cụ thể, sau đó phản hồi được gửi lại. Tích hợp ở cấp độ thấp nhất cho phép máy khách gửi một số yêu cầu, có thể là cho các máy chủ khác nhau, thành một yêu cầu lớn hơn và thực hiện nó như một giao dịch phân tán. Ý tưởng chính là tất cả hoặc không có yêu cầu nào được thực hiện.

Khi các ứng dụng trở nên tinh vi hơn và dần dần được tách thành các thành phần độc lập (đặc biệt là phân biệt các thành phần cơ sở dữ liệu với các thành phần xử lý), rõ ràng là tích hợp cũng nên diễn ra bằng cách để các ứng dụng giao tiếp trực tiếp với nhau. Điều này hiện đã dẫn đến một ngành công nghiệp về tích hợp ứng dụng doanh nghiệp (EAI).

Xử lý giao dịch phân tán

Để làm rõ cuộc thảo luận của chúng tôi, chúng tôi tập trung vào các ứng dụng cơ sở dữ liệu. Trong thực tế, các hoạt động trên cơ sở dữ liệu được thực hiện dưới dạng giao dịch. Lập trình sử dụng giao dịch đòi hỏi các nguyên hàm đặc biệt phải được cung cấp bởi hệ thống phân tán cơ bản hoặc hệ thống thời gian chạy ngôn ngữ. Các ví dụ điển hình về nguyên hàm giao dịch được thể hiện trong Hình 1.11. Danh sách chính xác các nguyên hàm phụ thuộc vào loại đối tượng nào đang được sử dụng trong giao dịch [Gray và Reuter, 1993; Bernstein và Newcomer, 2009]. Trong hệ thống thư, có thể có các nguyên hàm để gửi, nhận và chuyển tiếp thư. Trong một hệ thống kế toán, chúng có thể khá khác nhau. Tuy nhiên, READ và WRITE là những ví dụ điển hình. Các câu lệnh thông thư ởng, lệnh gọi thủ tục, v.v. cũng được phép bên trong một giao dịch. Đặc biệt, các lệnh gọi thủ tục từ xa (RPC), tức là các lệnh gọi thủ tục đến các máy chủ từ xa, thư ởng cũng được đóng gói trong một giao dịch, dẫn đến cái được gọi là RPC giao dịch. Chúng tôi thảo luận chi tiết về RPC trong Phần 4.2.

Nguyên thủy	Sự miêu tả
BEGIN_TRANSACTION	Đánh dấu sự bắt đầu của một giao dịch
KẾT THÚC GIAO DỊCH	Kết thúc giao dịch và cố gắng cam kết
ABORT_TRANSACTION	Hủy giao dịch và khôi phục các giá trị cũ
ĐỌC	Đọc dữ liệu từ một tệp, một bảng hoặc một nơi khác
VIẾT	Ghi dữ liệu vào tệp, bảng hoặc nơi khác

Hình 1.11: Ví dụ về các nguyên hàm cho giao dịch.

BEGIN_TRANSACTION và END_TRANSACTION được sử dụng để phân định phạm vi của một giao dịch. Các hoạt động giữa chúng tạo thành phần thân của giao dịch. Đặc điểm đặc trưng của một giao dịch là tất cả các hoạt động này được thực hiện hoặc không có hoạt động nào được thực hiện. Chúng có thể là các lệnh gọi hệ thống, thủ tục thư viện hoặc các câu lệnh ngoặc trong một ngôn ngữ, tùy thuộc vào cách triển khai.

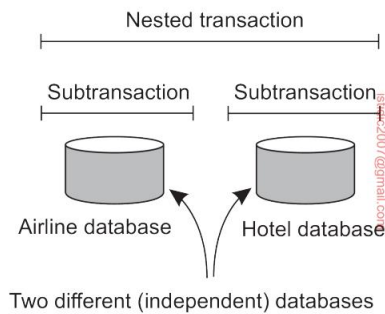
Thuộc tính tất cả hoặc không có gì của giao dịch này là một trong bốn thuộc tính đặc trưng mà giao dịch có. Cụ thể hơn, giao dịch tuân thủ cái gọi là thuộc tính ACID :

- Nguyên tử: Đối với thế giới bên ngoài, giao dịch diễn ra không thể chia cắt •

Nhất quán: Giao dịch không vi phạm các bất biến của hệ thống

- Riêng biệt: Các giao dịch đồng thời không can thiệp lẫn nhau
- Bền vững: Khi giao dịch được xác nhận, những thay đổi sẽ là vĩnh viễn

Trong các hệ thống phân tán, các giao dịch thường được xây dựng dưới dạng một số giao dịch con, cùng nhau tạo thành một giao dịch lồng nhau như thể hiện trong Hình 1.12. Giao dịch cấp cao nhất có thể phân nhánh các giao dịch con chạy song song với nhau, trên các máy khác nhau, để tăng hiệu suất hoặc đơn giản hóa việc lập trình. Mỗi giao dịch con này cũng có thể thực hiện một hoặc nhiều giao dịch phụ hoặc tách riêng các giao dịch con của mình.



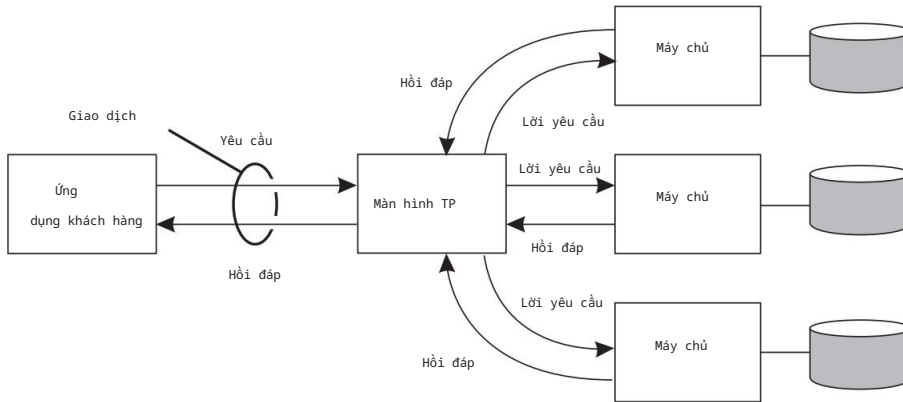
Hình 1.12: Một giao dịch lồng nhau.

Các giao dịch phụ tạo ra một vấn đề tính toán quan trọng. Hãy tưởng tượng rằng một giao dịch bắt đầu một số giao dịch phụ song song và một trong những giao dịch này sẽ thực hiện cam kết, khiến kết quả của giao dịch phụ này hiển thị với giao dịch cha. Sau khi tính toán thêm, giao dịch cha sẽ hủy, khôi phục toàn bộ hệ thống về trạng thái trước khi giao dịch cấp cao nhất bắt đầu. Do đó, kết quả của giao dịch phụ đã cam kết vẫn phải được hoàn tác. Do đó, tính vĩnh viễn được đề cập ở trên chỉ áp dụng cho các giao dịch cấp cao nhất.

Vì các giao dịch có thể được lồng nhau sâu tùy ý, nên cần có sự quản lý đáng kể để mọi thứ được thực hiện đúng. Tuy nhiên, ngữ nghĩa thì rõ ràng. Khi bất kỳ giao dịch hoặc giao dịch phụ nào bắt đầu, về mặt khái niệm, nó được cung cấp một bản sao riêng của tất cả dữ liệu trong toàn bộ hệ thống để nó có thể thao tác theo ý muốn. Nếu nó bị hủy, vũ trụ riêng của nó sẽ biến mất, như thể nó chưa từng tồn tại. Nếu nó cam kết, vũ trụ riêng của nó sẽ thay thế vũ trụ của cha mẹ. Do đó, nếu một giao dịch phụ cam kết và sau đó một giao dịch phụ mới được bắt đầu, giao dịch thứ hai sẽ thấy kết quả do giao dịch đầu tiên tạo ra. Tư tưởng tự nhiên như vậy, nếu một giao dịch bao quanh (cấp cao hơn) bị hủy, tất cả các giao dịch phụ cơ bản của nó cũng phải bị hủy. Và nếu một số giao dịch được bắt đầu đồng thời, kết quả sẽ giống như thể chúng chạy tuần tự theo một thứ tự không xác định.

Các giao dịch lồng nhau rất quan trọng trong các hệ thống phân tán, vì chúng cung cấp một cách tự nhiên để phân phối một giao dịch trên nhiều máy. Chúng tuân theo một sự phân chia hợp lý công việc của giao dịch ban đầu. Ví dụ, một giao dịch để lập kế hoạch cho một chuyến đi mà cần phải có ba chuyến bay khác nhau

Reserved có thể được chia thành ba giao dịch phụ một cách hợp lý. Mỗi giao dịch phụ này có thể được quản lý riêng biệt và độc lập.



Hình 1.13: Vai trò của màn hình TP trong hệ thống phân tán.

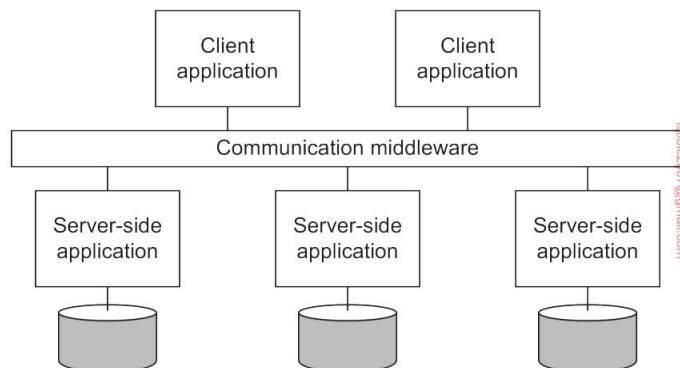
Kể từ những ngày đầu của hệ thống phần mềm trung gian doanh nghiệp, thành phần xử lý các giao dịch phân tán (hoặc lồng nhau) thuộc về lõi để tích hợp các ứng dụng ở cấp độ máy chủ hoặc cơ sở dữ liệu. Thành phần này được gọi là màn hình xử lý giao dịch hoặc gọi tắt là màn hình TP. Nhiệm vụ chính của nó là cho phép một ứng dụng truy cập nhiều máy chủ/cơ sở dữ liệu bằng cách cung cấp cho ứng dụng đó một mô hình lập trình giao dịch, như thể hiện trong Hình 1.13. Về cơ bản, màn hình TP điều phối việc cam kết các giao dịch phụ theo một giao thức chuẩn được gọi là cam kết phân tán, mà chúng ta sẽ thảo luận chi tiết trong Phần 8.5.

Một quan sát quan trọng là các ứng dụng muốn phối hợp nhiều giao dịch phụ thành một giao dịch duy nhất không phải tự triển khai sự phối hợp này. Chỉ cần sử dụng một màn hình TP, sự phối hợp này được thực hiện cho chúng. Đây chính xác là nơi phần mềm trung gian phát huy tác dụng: nó triển khai các dịch vụ hữu ích cho nhiều ứng dụng, tránh việc các nhà phát triển ứng dụng phải triển khai lại các dịch vụ đó nhiều lần.

Tích hợp ứng dụng doanh nghiệp

Như đã đề cập, càng nhiều ứng dụng được tách khỏi cơ sở dữ liệu mà chúng được xây dựng, thì càng rõ ràng rằng cần có các tiện ích để tích hợp các ứng dụng độc lập với cơ sở dữ liệu của chúng. Đặc biệt, các thành phần ứng dụng phải có khả năng giao tiếp trực tiếp với nhau chứ không chỉ thông qua hành vi yêu cầu/phản hồi được hỗ trợ bởi các hệ thống xử lý giao dịch.

Nhu cầu giao tiếp liên ứng dụng này dẫn đến nhiều mô hình giao tiếp. Ý tưởng chính là các ứng dụng hiện có có thể trao đổi thông tin trực tiếp, như thể hiện trong Hình 1.14.



Hình 1.14: Phần mềm trung gian đóng vai trò là công cụ hỗ trợ giao tiếp trong tích hợp ứng dụng doanh nghiệp.

Có một số loại phần mềm trung gian truyền thông. Với các lệnh gọi thủ tục từ xa (RPC), một thành phần ứng dụng có thể gửi yêu cầu đến một thành phần ứng dụng khác một cách hiệu quả bằng cách thực hiện lệnh gọi thủ tục cục bộ, kết quả là yêu cầu được đóng gói dưới dạng tin nhắn và được gửi đến bên được gọi. Tư ơng tự như vậy, kết quả sẽ được gửi lại và trả về ứng dụng dưới dạng kết quả của lệnh gọi thủ tục.

Khi công nghệ đối tượng ngày càng phổ biến, các kỹ thuật được phát triển để cho phép gọi đến các đối tượng từ xa, dẫn đến cái được gọi là gọi phư ơng thức từ xa (RMI). Về cơ bản, RMI giống như RPC, ngoại trừ việc nó hoạt động trên các đối tượng thay vì các hàm.

RPC và RMI có như ợc điểm là cả ngư ời gọi và ngư ời đư ợc gọi đều cần phải hoạt động tại thời điểm giao tiếp. Ngoài ra, họ cần biết chính xác cách tham chiếu đến nhau. Sự kết hợp chặt chẽ này thư ờng đư ợc coi là một như ợc điểm nghiêm trọng và đã dẫn đến cái đư ợc gọi là phần mềm trung gian hư ớng tin nhắn hoặc đơn giản là MOM. Trong trư ờng hợp này, các ứng dụng gửi tin nhắn đến các điểm tiếp xúc hợp lý, thư ờng đư ợc mô tả bởi một chủ thể. Tư ơng tự như vậy, các ứng dụng có thể chỉ ra sự quan tâm của chúng đối với một loại tin nhắn cụ thể, sau đó phần mềm trung gian giao tiếp sẽ đảm bảo rằng các tin nhắn đó đư ợc gửi đến các ứng dụng đó. Các hệ thống đư ợc gọi là xuất bản-đăng ký này tạo thành một lớp hệ thống phân tán quan trọng và đang mở rộng.

Lưu ý 1.7 (Thông tin thêm: Về việc tích hợp các ứng dụng)

Hỗ trợ tích hợp ứng dụng doanh nghiệp là mục tiêu quan trọng đối với nhiều sản phẩm phần mềm trung gian. Nhìn chung, có bốn cách để tích hợp ứng dụng [Hohpe và Woolf, 2004]:

Chuyển tập tin: Bản chất của tích hợp thông qua chuyển tập tin là ứng dụng tạo ra một tập tin chứa dữ liệu đư ợc chia sẻ sau đó đư ợc đọc bởi

các ứng dụng khác. Cách tiếp cận này về mặt kỹ thuật rất đơn giản, nên rất hấp dẫn. Tuy nhiên, nhược điểm là có rất nhiều điều cần phải thống nhất:

- Định dạng và bố cục tệp: văn bản, nhị phân, cấu trúc của tệp, v.v. Ngày nay, ngôn ngữ đánh dấu mở rộng (XML) đã trở nên phổ biến vì về nguyên tắc, các tệp của nó tự mô tả.
- Quản lý tệp:

chúng được lưu trữ ở đâu, chúng được đặt tên như thế nào, ai chịu trách nhiệm xóa tệp? • Truyền bá cập

nhật: Khi một ứng dụng tạo ra một tệp, có thể có một số ứng dụng cần đọc tệp đó để cung cấp chế độ xem của một hệ thống thống nhất duy nhất. Do đó, đôi khi cần triển khai các chương trình riêng biệt để thông báo cho các ứng dụng về các bản cập nhật tệp.

Cơ sở dữ liệu dùng chung: Nhiều vấn đề liên quan đến tích hợp thông qua tệp được giảm bớt khi sử dụng cơ sở dữ liệu dùng chung. Tất cả các ứng dụng sẽ có quyền truy cập vào cùng một dữ liệu và thư ờng thông qua ngôn ngữ cơ sở dữ liệu cấp cao như SQL. Hơn nữa, rất dễ thông báo cho các ứng dụng khi có thay đổi vì các trình kích hoạt thư ờng là một phần của cơ sở dữ liệu hiện đại. Tuy nhiên, có hai nhược điểm chính. Đầu tiên, vẫn cần phải thiết kế một lược đồ dữ liệu chung, điều này có thể không hề đơn giản nếu tập hợp các ứng dụng cần tích hợp không được biết trước hoàn toàn. Thứ hai, khi có nhiều lần đọc và cập nhật, cơ sở dữ liệu dùng chung có thể dễ dàng trở thành nút thắt cổ chai về hiệu suất.

Gọi thủ tục từ xa: Tích hợp thông qua tệp hoặc cơ sở dữ liệu ngầm định rằng các thay đổi của một ứng dụng có thể dễ dàng kích hoạt các ứng dụng khác hành động. Tuy nhiên, thực tế cho thấy đôi khi những thay đổi nhỏ thực sự có thể kích hoạt nhiều ứng dụng thực hiện hành động. Trong những trường hợp như vậy, điều quan trọng không phải là thay đổi dữ liệu mà là việc thực hiện một loạt các hành động.

Chuỗi hành động được nắm bắt tốt nhất thông qua việc thực hiện một thủ tục (có thể dẫn đến mọi loại thay đổi trong dữ liệu được chia sẻ). Để ngăn chặn mọi ứng dụng cần biết tất cả các nội dung bên trong của các hành động đó (như được triển khai bởi một ứng dụng khác), các kỹ thuật đóng gói chuẩn nên được sử dụng, như được triển khai với các lệnh gọi thủ tục truyền thống hoặc các lệnh gọi đối tượng. Đối với những tình huống như vậy, một ứng dụng có thể cung cấp một thủ tục tốt nhất cho các ứng dụng khác dưới dạng lệnh gọi thủ tục từ xa hoặc RPC. Về bản chất, RPC cho phép ứng dụng A sử dụng thông tin chỉ có sẵn cho ứng dụng B, mà không cấp cho A quyền truy cập trực tiếp vào thông tin đó. Có nhiều ưu điểm và nhược điểm đối với các lệnh gọi thủ tục từ xa, được thảo luận sâu hơn trong Chương 4.

Nhấn tin: Một nhược điểm chính của RPC là người gọi và người được gọi cần phải hoạt động cùng lúc để cuộc gọi thành công. Tuy nhiên, trong nhiều trường hợp, hoạt động đồng thời này thư ờng khó hoặc không thể thực hiện được

để đảm bảo. Trong những trường hợp như vậy, việc cung cấp một hệ thống nhắn tin mang theo các yêu cầu từ ứng dụng A để thực hiện một hành động tại ứng dụng B là điều cần thiết. Hệ thống nhắn tin đảm bảo rằng cuối cùng yêu cầu sẽ được chuyển đi và nếu cần, phản hồi cuối cùng cũng sẽ được trả về. Rõ ràng, nhắn tin không phải là giải pháp tối ưu cho tích hợp ứng dụng: nó cũng gây ra các vấn đề liên quan đến định dạng và bố cục dữ liệu, nó yêu cầu ứng dụng phải biết gửi tin nhắn đến đâu, cần có các kịch bản để xử lý tin nhắn bị mất, v.v. Giống như RPC, chúng ta sẽ thảo luận chi tiết về các vấn đề này trong Chương 4.

Bốn cách tiếp cận này cho chúng ta biết rằng tích hợp ứng dụng nói chung sẽ không đơn giản. Tuy nhiên, phần mềm trung gian (dưới dạng hệ thống phân tán) có thể giúp tích hợp đáng kể bằng cách cung cấp các tiện ích phù hợp như hỗ trợ RPC hoặc nhắn tin. Như đã nói, tích hợp ứng dụng doanh nghiệp là lĩnh vực mục tiêu quan trọng đối với nhiều sản phẩm phần mềm trung gian.

1.3.3 Hệ thống lan tỏa Các

hệ thống phân tán đã thảo luận cho đến nay phần lớn được đặc trưng bởi tính ổn định của chúng: các nút được cố định và có kết nối ít nhiều cố định và chất lượng cao với mạng. Ở một mức độ nào đó, tính ổn định này được hiện thực hóa thông qua các kỹ thuật khác nhau để đạt được tính minh bạch phân phối. Ví dụ, có nhiều cách để chúng ta có thể tạo ra ảo giác rằng chỉ thỉnh thoảng các thành phần mới có thể bị lỗi. Tư tưởng tự nhiên vậy, có đủ loại phương tiện để ẩn vị trí mạng thực tế của một nút, cho phép người dùng và ứng dụng tin rằng các nút vẫn ở nguyên vị trí.

Tuy nhiên, mọi thứ đã thay đổi kể từ khi các thiết bị điện toán di động và nhúng ra đời, dẫn đến những gì thường được gọi là hệ thống lan tỏa. Như tên gọi của nó, hệ thống lan tỏa có mục đích hòa nhập vào môi trường của chúng ta một cách tự nhiên. Nhiều thành phần của chúng nhất thiết phải trải rộng trên nhiều máy tính, khiến chúng có thể được coi là một loại hệ thống phi tập trung theo quan điểm của chúng tôi. Đồng thời, hầu hết các hệ thống lan tỏa đều có nhiều thành phần được trải rộng đủ khắp hệ thống, ví dụ, để xử lý các lỗi và những thứ tự nhiên. Theo nghĩa này, chúng cũng có thể được coi là hệ thống phân tán. Do đó, sự tách biệt có vẻ nghiêm ngặt giữa hệ thống phi tập trung và hệ thống phân tán được coi là ít nghiêm ngặt hơn so với những gì người ta có thể tư tưởng tự nhiên ban đầu.

Điều khiến chúng trở nên độc đáo, khi so sánh với các hệ thống thông tin và máy tính đã mô tả cho đến nay, là sự tách biệt giữa người dùng và các thành phần hệ thống mờ nhạt hơn nhiều. Thường không có giao diện chuyên dụng duy nhất, chẳng hạn như kết hợp màn hình/bàn phím. Thay vào đó, một hệ thống phổ biến thường được trang bị nhiều cảm biến thu thập các khía cạnh khác nhau của hành vi người dùng. Tư tưởng tự nhiên vậy, nó có thể có vô số bộ truyền động để cung cấp thông tin và phản hồi, thậm chí thường có mục đích điều khiển hành vi.

Nhiều thiết bị trong các hệ thống lan tỏa được đặc trưng bởi kích thước nhỏ, chạy bằng pin, di động và chỉ có kết nối không dây, mặc dù không phải tất cả các đặc điểm này đều áp dụng cho tất cả các thiết bị. Đây không nhất thiết là những đặc điểm hạn chế, như được minh họa bằng điện thoại thông minh [Roussos và cộng sự, 2005] và vai trò của chúng trong cái hiện được gọi là Internet vạn vật [Mattern và Floerkemeier, 2010 ; Stankovic, 2014]. Tuy nhiên, đáng chú ý là thực tế là chúng ta thường cần phải giải quyết những phức tạp của truyền thông không dây và di động, sẽ đòi hỏi các giải pháp đặc biệt để làm cho một hệ thống lan tỏa trở nên minh bạch hoặc không phổ biến nhất có thể.

Sau đây, chúng tôi phân biệt ba loại hệ thống lan tỏa khác nhau, mặc dù có sự chồng chéo đáng kể giữa ba loại này: hệ thống máy tính phổ biến, hệ thống di động và mạng cảm biến.

Sự phân biệt này cho phép chúng ta tập trung vào những khía cạnh khác nhau của các hệ thống lan tỏa.

Hệ thống máy tính phổ biến

Cho đến nay, chúng ta đã nói về các hệ thống phổ biến để nhấn mạnh rằng các thành phần của nó đã lan rộng khắp nhiều nơi trong môi trường của chúng ta. Trong một hệ thống điện toán phổ biến, chúng ta tiến thêm một bước nữa: hệ thống này phổ biến và liên tục hiện diện. Điều sau có nghĩa là người dùng sẽ liên tục tương tác với hệ thống, thậm chí thường không nhận thức được rằng tương tác đang diễn ra. Poslad [2009] mô tả các yêu cầu cốt lõi đối với một hệ thống điện toán phổ biến như sau:

1. (Phân phối) Các thiết bị được kết nối mạng, phân phối và có thể truy cập được thuộc về cha mẹ
2. (Tương tác) Tương tác giữa người dùng và thiết bị rất kín đáo
3. (Nhận thức ngữ cảnh) Hệ thống nhận thức được ngữ cảnh của người dùng để tối ưu hóa sự tương tác
4. (Tự động) Các thiết bị hoạt động tự động mà không cần sự can thiệp của con người và do đó có khả năng tự quản lý cao.
5. (Trí thông minh) Toàn bộ hệ thống có thể xử lý nhiều loại tình trạng khác nhau. hành động và tương tác nam

Chúng ta hãy xem xét những yêu cầu này theo góc nhìn hệ thống phân tán.

Quảng cáo 1: Phân phối Như đã đề cập, một hệ thống máy tính phổ biến là một ví dụ về hệ thống phân tán: các thiết bị và máy tính khác tạo thành các nút của một hệ thống chỉ đơn giản là được kết nối mạng và làm việc cùng nhau để tạo thành ảo giác về một hệ thống thống nhất, duy nhất. Phân phối cũng diễn ra một cách tự nhiên: sẽ có các thiết bị gần với người dùng (như cảm biến và bộ truyền động), được kết nối với máy tính ẩn khỏi tầm nhìn và thậm chí có thể hoạt động từ xa trong một

đám mây. Do đó, hầu hết, nếu không muốn nói là tất cả, các yêu cầu liên quan đến tính minh bạch trong phân phối được đề cập trong Mục 1.2.2, đều phải được tuân thủ.

Quảng cáo 2: Tư ơng tác Khi nói đến tư ơng tác với ngư ời dùng, các hệ thống điện toán phổ biến khác rất nhiều so với các hệ thống mà chúng ta đã thảo luận cho đến nay. Ngư ời dùng cuối đóng vai trò nổi bật trong thiết kế các hệ thống phổ biến, nghĩa là cần đặc biệt chú ý đến cách tư ơng tác giữa ngư ời dùng và hệ thống cốt lõi diễn ra. Đối với các hệ thống điện toán phổ biến, phần lớn tư ơng tác của con ngư ời sẽ là ngầm định, với hành động ngầm định được định nghĩa là hành động "không chủ yếu nhằm mục đích tư ơng tác với hệ thống máy tính mà hệ thống đó hiểu là đầu vào" [Schmidt, 2000]. Nói cách khác, ngư ời dùng có thể hầu như không biết rằng đầu vào đang được cung cấp cho hệ thống máy tính. Theo một góc độ nào đó, có thể nói rằng điện toán phổ biến dường như ẩn các giao diện.

Một ví dụ đơn giản là khi cài đặt ghế lái, vô lăng và gư ơng của ô tô được cá nhân hóa hoàn toàn. Nếu Bob ngồi vào ghế, hệ thống sẽ nhận ra rằng nó đang giao dịch với Bob và sau đó thực hiện các điều chỉnh phù hợp. Điều tư ơng tự cũng xảy ra khi Alice sử dụng ô tô, trong khi một ngư ời dùng không xác định sẽ được hướng dẫn thực hiện các điều chỉnh của riêng mình (sẽ được ghi nhớ để sử dụng sau). Ví dụ này đã minh họa cho vai trò quan trọng của các cảm biến trong điện toán phổ biến, cụ thể là các thiết bị đầu vào được sử dụng để xác định một tình huống (một ngư ời cụ thể rõ ràng muốn lái xe), phân tích đầu vào của ngư ời này dẫn đến các hành động (thực hiện các điều chỉnh). Đối lại, các hành động có thể dẫn đến các phản ứng tự nhiên, ví dụ như Bob thay đổi một chút cài đặt ghế. Hệ thống sẽ phải tính đến tất cả các hành động (ngầm định và rõ ràng) của ngư ời dùng và phản ứng phù hợp.

Quảng cáo. 3: Nhận thức bối cảnh Phản ứng với đầu vào cảm giác, cũng như đầu vào rõ ràng từ ngư ời dùng, thì nói thì dễ hơn làm. Điều mà một hệ thống điện toán phổ biến cần làm là tính đến bối cảnh diễn ra các tư ơng tác. Nhận thức bối cảnh cũng phân biệt các hệ thống điện toán phổ biến với các hệ thống truyền thống hơn mà chúng ta đã thảo luận trước đây và được Dey và Abowd [2000] mô tả là "bất kỳ thông tin nào có thể được sử dụng để mô tả tình huống của các thực thể (tức là một ngư ời, địa điểm hoặc đối tư ơng) được coi là có liên quan đến tư ơng tác giữa ngư ời dùng và ứng dụng, bao gồm cả ngư ời dùng và chính ứng dụng đó". Trong thực tế, bối cảnh thường được đặc trưng bởi vị trí, danh tính, thời gian và hoạt động: ở đâu, ai, khi nào và cái gì. Một hệ thống sẽ cần có đầu vào (cảm giác) cần thiết để xác định một hoặc một số loại bối cảnh này. Như đã thảo luận bởi Alegre và cộng sự [2016], việc phát triển các hệ thống nhận thức bối cảnh là rất khó, nếu chỉ vì lý do là khái niệm bối cảnh khó nắm bắt.

Điều quan trọng, theo quan điểm của hệ thống phân tán, là dữ liệu thô được thu thập bởi nhiều cảm biến khác nhau được nâng lên mức trừu tượng mà các ứng dụng có thể sử dụng. Một ví dụ cụ thể là phát hiện vị trí của một ngư ời,

ví dụ về tọa độ GPS, và sau đó ánh xạ thông tin đó đến một vị trí thực tế, chẳng hạn như góc phố, hoặc một cửa hàng cụ thể hoặc cơ sở đã biết khác. Câu hỏi đặt ra là quá trình xử lý đầu vào cảm giác này diễn ra ở đâu: tất cả dữ liệu được thu thập tại một máy chủ trung tâm được kết nối với cơ sở dữ liệu có thông tin chi tiết về một thành phố hay là điện thoại thông minh của người dùng nơi thực hiện việc lập bản đồ? Rõ ràng là có những sự đánh đổi cần được xem xét.

Dey [2010] thảo luận về các phương pháp tiếp cận chung hơn đối với việc xây dựng các ứng dụng nhận biết ngữ cảnh. Khi nói đến việc kết hợp tính linh hoạt và phân phối tiềm năng, cái gọi là không gian dữ liệu dùng chung trong đó các quy trình được tách rời theo thời gian và không gian rất hấp dẫn, nhưng như chúng ta sẽ thấy trong các chương sau, lại gặp phải các vấn đề về khả năng mở rộng. Một cuộc khảo sát về nhận biết ngữ cảnh và mối quan hệ của nó với phần mềm trung gian và hệ thống phân tán được cung cấp bởi Baldauf et al. [2007].

Quảng cáo 4: Tự chủ Một khía cạnh quan trọng của hầu hết các hệ thống máy tính phổ biến là việc quản lý hệ thống rõ ràng đã được giảm xuống mức tối thiểu. Trong một môi trường máy tính phổ biến, đơn giản là không có chỗ cho một quản trị viên hệ thống để duy trì mọi thứ hoạt động. Do đó, toàn bộ hệ thống phải có khả năng hoạt động tự chủ và tự động phản ứng với các thay đổi. Điều này đòi hỏi vô số kỹ thuật, trong đó một số kỹ thuật sẽ được thảo luận trong suốt cuốn sách này. Để đưa ra một vài ví dụ đơn giản, hãy nghĩ đến những điều sau:

Phân bổ địa chỉ: Để các thiết bị mạng có thể giao tiếp, chúng cần một địa chỉ IP. Địa chỉ có thể được phân bổ tự động bằng các giao thức như Giao thức cấu hình máy chủ động (DHCP) [Droms, 1997] (yêu cầu một máy chủ) hoặc Zeroconf [Guttman, 2001].

Thêm thiết bị: Việc thêm thiết bị vào hệ thống hiện có phải dễ dàng. Một bước tiến tới cấu hình tự động được thực hiện bằng giao thức Universal Plug and Play (UPnP) [Tiền đề UPnP, 2008]. Sử dụng UPnP, các thiết bị có thể khám phá nhau và đảm bảo rằng chúng có thể thiết lập các kênh giao tiếp giữa chúng.

Cập nhật tự động: Nhiều thiết bị trong hệ thống máy tính phổ biến có thể tự động xuyên kiểm tra qua Internet xem phần mềm của chúng có cần được cập nhật hay không. Nếu có, chúng có thể tải xuống các phiên bản mới của các thành phần và lý tưởng nhất là tiếp tục từ nơi chúng dừng lại.

Phải thừa nhận rằng đây là những ví dụ đơn giản, nhưng bức tranh phải rõ ràng rằng sự can thiệp thủ công phải được giữ ở mức tối thiểu. Chúng ta sẽ thảo luận chi tiết về nhiều kỹ thuật liên quan đến tự quản lý trong suốt cuốn sách.

Quảng cáo 5: Trí thông minh Cuối cùng, Poslad [2009] đề cập rằng các hệ thống máy tính phổ biến tự động sử dụng các phương pháp và kỹ thuật từ lĩnh vực trí tuệ nhân tạo

trí thông minh. Điều này có nghĩa là thư ờng cần triển khai nhiều thuật toán và mô hình tiên tiến để xử lý dữ liệu đầu vào không đầy đủ, phản ứng nhanh với môi trường thay đổi, xử lý các sự kiện bất ngờ, v.v. Mức độ mà điều này có thể hoặc nên được thực hiện theo cách phân tán là rất quan trọng theo quan điểm của các hệ thống phân tán. Thật không may, các giải pháp phân tán cho nhiều vấn đề trong lĩnh vực trí tuệ nhân tạo vẫn chưa được tìm thấy, nghĩa là có thể có sự căng thẳng tự nhiên giữa yêu cầu đầu tiên của các thiết bị phân tán và được kết nối mạng và xử lý thông tin phân tán tiên tiến.

Hệ thống máy tính di động

Như đã đề cập, tính di động thư ờng tạo thành một thành phần quan trọng của các hệ thống lan tỏa, và nhiều khía cạnh, nếu không muốn nói là tất cả, mà chúng ta vừa thảo luận cũng áp dụng cho điện toán di động. Có một số vấn đề đặt điện toán di động sang một bên so với các hệ thống lan tỏa nói chung (xem thêm Adelstein et al. [2005] và Tarkoma và Kangasharju [2009]).

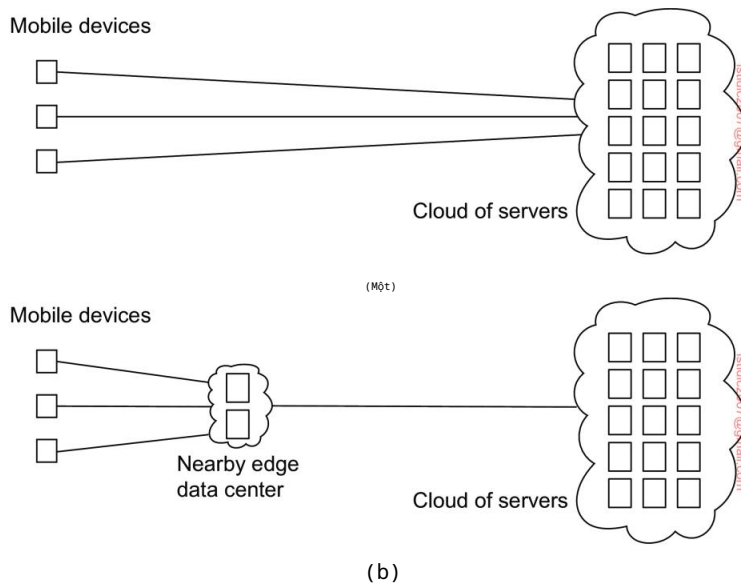
Đầu tiên, các thiết bị tạo thành một phần của hệ thống di động (phân tán) có thể rất khác nhau. Thông thư ờng, điện toán di động được thực hiện bằng các thiết bị như điện thoại thông minh và máy tính bảng. Tuy nhiên, hiện nay, các loại thiết bị hoàn toàn khác nhau đang sử dụng Giao thức Internet (IP) để giao tiếp, đưa điện toán di động vào một góc nhìn khác. Các thiết bị như vậy bao gồm điều khiển từ xa, máy nhắn tin, huy hiệu hoạt động, thiết bị ô tô, nhiều thiết bị hỗ trợ GPS, v.v. Một đặc điểm đặc trưng của tất cả các thiết bị này là chúng sử dụng giao tiếp không dây. Di động ngụ ý không dây, vì vậy có vẻ như vậy (mặc dù có những ngoại lệ đối với các quy tắc).

Thứ hai, trong điện toán di động, vị trí của thiết bị được cho là thay đổi theo thời gian. Vị trí thay đổi có tác động đến nhiều vấn đề. Ví dụ, nếu vị trí của thiết bị thay đổi thư ờng xuyên, thì có lẽ các dịch vụ có sẵn tại địa phương cũng vậy. Do đó, chúng ta có thể cần đặc biệt chú ý đến việc khám phá các dịch vụ một cách động, như ng cũng để các dịch vụ thông báo sự hiện diện của chúng. Tư ờng tự như vậy, chúng ta thư ờng muốn biết thiết bị thực sự ở đâu. Điều này có nghĩa là chúng ta cần biết tọa độ địa lý thực tế của thiết bị như trong các ứng dụng theo dõi và truy tìm, như ng cũng có thể yêu cầu chúng ta chỉ cần phát hiện vị trí mạng của thiết bị (như trong IP di động [Perkins, 2010; Perkins và cộng sự, 2011]).

Thay đổi địa điểm cũng có thể có tác động sâu sắc đến giao tiếp.

Trong một thời gian, các nhà nghiên cứu về điện toán di động đã tập trung vào cái được gọi là mạng ad hoc di động, còn được gọi là MANET. Ý tưởng cơ bản là một nhóm máy tính di động cục bộ sẽ cùng nhau thiết lập một mạng không dây cục bộ và sau đó chia sẻ tài nguyên và dịch vụ. Ý tưởng này chưa bao giờ thực sự trở nên phổ biến. Tư ờng tự như vậy, có những nhà nghiên cứu tin rằng người dùng cuối sẵn sàng chia sẻ tài nguyên cục bộ của họ cho các yêu cầu tính toán, lưu trữ hoặc giao tiếp của người dùng khác (xem, ví dụ, Ferrer

et al. [2019]). Thực tế đã chứng minh hết lần này đến lần khác rằng việc tự nguyện cung cấp tài nguyên không phải là điều người dùng sẵn lòng làm, ngay cả khi họ có nhiều tài nguyên. Hiệu ứng là trong trường hợp điện toán di động, chúng ta thường thấy các thiết bị di động đơn lẻ thiết lập kết nối với máy chủ cố định. Khi đó, việc thay đổi vị trí chỉ có nghĩa là các kết nối đó cần được bộ định tuyến chuyển giao trên đường dẫn từ thiết bị di động đến máy chủ. Sau đó, điện toán di động trở lại bản chất của nó: một thiết bị di động được kết nối với máy chủ (và không có gì khác). Trên thực tế, điều này có nghĩa là điện toán di động hoàn toàn là về các thiết bị di động sử dụng các dịch vụ dựa trên đám mây, như được phác họa trong Hình 1.15(a).



Hình 1.15: (a) Điện toán đám mây di động so với (b) Điện toán biên di động.

Bất chấp sự đơn giản về mặt khái niệm của mô hình điện toán di động này, thực tế là rất nhiều thiết bị sử dụng các dịch vụ từ xa đã dẫn đến cái được gọi là Điện toán biên di động, hay đơn giản là MEC [Abbas và cộng sự, 2018], trái ngược với Điện toán đám mây di động (MCC). Như chúng ta sẽ thảo luận thêm trong Chương 2, điện toán biên (di động), như được phác họa trong Hình 1.15(b), đang ngày càng trở nên quan trọng trong những trường hợp mà độ trễ, như các vấn đề về tính toán cũng đóng vai trò đối với thiết bị di động. Các ứng dụng ví dụ điển hình yêu cầu độ trễ ngắn và tài nguyên tính toán bao gồm thực tế tăng cường, trò chơi tương tác, giám sát thể thao theo thời gian thực và nhiều ứng dụng sức khỏe khác nhau [Dimou và cộng sự, 2022]. Trong những ví dụ này, sự kết hợp giữa giám sát, phân tích và phản hồi ngay lập tức nói chung khiến việc dựa vào các máy chủ có thể được đặt cách xa hàng nghìn dặm so với các thiết bị di động trở nên khó khăn.

Mạng cảm biến

Ví dụ cuối cùng của chúng tôi về hệ thống lan tỏa là mạng cảm biến. Trong nhiều trường hợp, các mạng này tạo thành một phần của công nghệ hỗ trợ cho tính lan tỏa và chúng tôi thấy rằng nhiều giải pháp cho mạng cảm biến quay trở lại trong các ứng dụng lan tỏa. Điều khiến mạng cảm biến trở nên thú vị theo quan điểm của hệ thống phân tán là chúng không chỉ là một tập hợp các thiết bị đầu vào. Thay vào đó, như chúng ta sẽ thấy, các nút cảm biến thường hợp tác để xử lý dữ liệu cảm biến một cách hiệu quả theo cách cụ thể của ứng dụng, khiến chúng rất khác so với, ví dụ, các mạng máy tính truyền thống. Akyildiz et al. [2002] và Akyildiz et al. [2005] cung cấp tổng quan từ góc độ mạng. Một phần giới thiệu theo định hướng hệ thống hơn về mạng cảm biến được đưa ra bởi Zhao và Guibas [2004], nhưng [Hahmy, 2021] cũng sẽ hữu ích.

Một mạng lưới cảm biến thường bao gồm hàng chục đến hàng trăm hoặc hàng nghìn nút tương đối nhỏ, mỗi nút được trang bị một hoặc nhiều thiết bị cảm biến. Ngoài ra, các nút thường có thể hoạt động như bộ truyền động [Akyildiz và Kasimoglu, 2004], một ví dụ điển hình là việc tự động kích hoạt vòi phun nước khi phát hiện ra hỏa hoạn. Nhiều mạng lưới cảm biến sử dụng giao tiếp không dây và các nút thường chạy bằng pin. Tài nguyên hạn chế, khả năng giao tiếp hạn chế và mức tiêu thụ điện năng bị hạn chế của chúng đòi hỏi hiệu quả phải là tiêu chí thiết kế hàng đầu.

Khi phóng to một nút riêng lẻ, chúng ta thấy rằng, về mặt khái niệm, chúng không khác nhiều so với máy tính "thông thường": phía trên phần cứng có một lớp phần mềm tương tự như những gì hệ điều hành truyền thống cung cấp, bao gồm quyền truy cập mạng cấp thấp, quyền truy cập vào cảm biến và bộ truyền động, quản lý bộ nhớ, v.v. Thông thường, hỗ trợ cho các dịch vụ cụ thể được bao gồm, chẳng hạn như bản địa hóa, lưu trữ cục bộ (hãy nghĩ đến các thiết bị flash bổ sung) và các tiện ích truyền thông thuận tiện như nhắn tin và định tuyến. Tuy nhiên, tương tự như các hệ thống máy tính được kết nối mạng khác, cần có hỗ trợ bổ sung để triển khai hiệu quả các ứng dụng mạng cảm biến. Trong các hệ thống phân tán, điều này có dạng phần mềm trung gian. Đối với mạng cảm biến, về nguyên tắc, chúng ta có thể áp dụng cách tiếp cận tương tự trong những trường hợp mà các nút cảm biến đủ mạnh và mức tiêu thụ năng lượng không phải là trở ngại để chạy một ngăn xếp phần mềm phức tạp hơn. Có thể có nhiều cách tiếp cận khác nhau (xem thêm [Zhang và cộng sự, 2021b]).

Theo quan điểm lập trình và được Mottola và Picco [2011] khảo sát rộng rãi, điều quan trọng là phải tính đến phạm vi của các nguyên thủy giao tiếp. Phạm vi này có thể thay đổi giữa việc giải quyết vấn đề lân cận vật lý của một nút và cung cấp các nguyên thủy cho giao tiếp toàn hệ thống. Ngoài ra, cũng có thể giải quyết được một nhóm nút cụ thể.

Tương tự như vậy, các phép tính có thể bị giới hạn ở một nút riêng lẻ, một nhóm nút hoặc ảnh hưởng đến tất cả các nút. Để minh họa, Welsh và Mainland [2004] sử dụng cái gọi là vùng trừu tượng cho phép một nút xác định một vùng lân cận nơi nó có thể, ví dụ, thu thập thông tin:

```

1   vùng = k_nearest_region.create(8); đọc =
2   get_sensor_reading();
3   vùng.putvar(khóa đọc, đọc); max_id =
4   vùng.reduce(OP_MAXID, khóa đọc);

```

Ở dòng 1, trước tiên một nút tạo một vùng gồm tám hàng xóm gần nhất, sau đó nó lấy giá trị từ cảm biến của nó. Sau đó, giá trị đọc này được ghi vào vùng đã xác định trước đó để xác định bằng khóa `reading_key`. Ở dòng 4, nút kiểm tra xem giá trị đọc cảm biến nào trong vùng đã xác định là lớn nhất, giá trị này được trả về trong biến `max_id`.

Khi xem xét rằng các mạng cảm biến tạo ra dữ liệu, người ta cũng có thể tập trung vào mô hình truy cập dữ liệu. Điều này có thể được thực hiện trực tiếp bằng cách gửi tin nhắn đến và giữa các nút hoặc di chuyển mã giữa các nút để truy cập dữ liệu cục bộ. Tiên tiến hơn là làm cho dữ liệu từ xa có thể truy cập trực tiếp, như thể các biến và những thứ tương tự có sẵn trong không gian dữ liệu được chia sẻ. Cuối cùng, và cũng khá phổ biến, là để mạng cảm biến cung cấp chế độ xem của một cơ sở dữ liệu duy nhất. Chế độ xem như vậy dễ hiểu khi nhận ra rằng nhiều mạng cảm biến được triển khai cho các ứng dụng đo lường và giám sát [Bonnet et al., 2002].

Trong những trường hợp này, một nhà điều hành muốn trích xuất thông tin từ (một phần của) mạng bằng cách chỉ cần đưa ra các truy vấn như " Tải trọng giao thông hướng bắc trên xa lộ 1 tại Santa Cruz là bao nhiêu?" Các truy vấn như vậy giống với các truy vấn của cơ sở dữ liệu truyền thống. Trong trường hợp này, câu trả lời có thể sẽ cần được cung cấp thông qua sự hợp tác của nhiều cảm biến dọc theo xa lộ 1, trong khi không động đến các cảm biến khác .

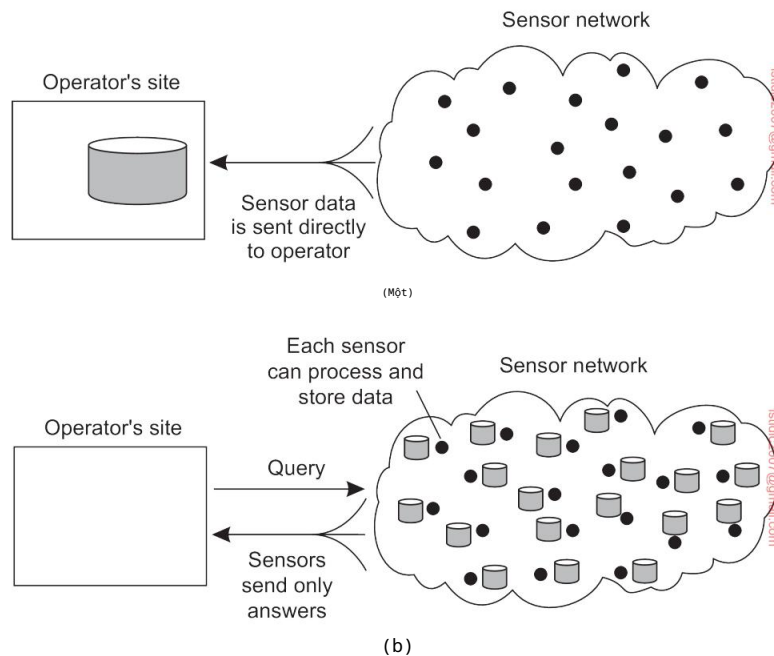
Để tổ chức một mạng lưu trữ cảm biến như một cơ sở dữ liệu phân tán, về cơ bản có hai thái cực, như thể hiện trong Hình 1.16. Đầu tiên, các cảm biến không hợp tác mà chỉ đơn giản gửi dữ liệu của chúng đến một cơ sở dữ liệu tập trung nằm tại địa điểm của nhà điều hành. Cách tiếp cận khác là chuyển tiếp các truy vấn đến các cảm biến có liên quan và để mỗi cảm biến tự tính toán câu trả lời, yêu cầu người vận hành phải tổng hợp các phản hồi.

Không có giải pháp nào trong số này hấp dẫn lắm. Giải pháp đầu tiên yêu cầu các cảm biến gửi tất cả dữ liệu đã đo được của chúng qua mạng, điều này có thể lãng phí tài nguyên và năng lượng của mạng. Giải pháp thứ hai cũng có thể lãng phí, vì nó loại bỏ khả năng tổng hợp của các cảm biến, điều này sẽ cho phép ít dữ liệu hơn được trả về cho người vận hành. Điều cần thiết là các tiện ích để xử lý dữ liệu trong mạng, tương tự như ví dụ trước về các vùng truy vấn.

Xử lý trong mạng có thể được thực hiện theo nhiều cách. Một cách rõ ràng là chuyển tiếp truy vấn đến tất cả các nút cảm biến dọc theo một cây bao gồm tất cả các nút và sau đó tổng hợp các kết quả khi chúng được truyền trở lại gốc , nơi có bộ khởi tạo. Tổng hợp sẽ diễn ra khi hai hoặc nhiều nhánh của cây kết hợp lại với nhau. Mặc dù sơ đồ này nghe có vẻ đơn giản, nhưng nó lại đưa ra những câu hỏi khó:

- Làm thế nào để chúng ta (động) thiết lập một cây hiệu quả trong mạng cảm biến? •

Tổng hợp kết quả diễn ra như thế nào? Có thể kiểm soát được không?



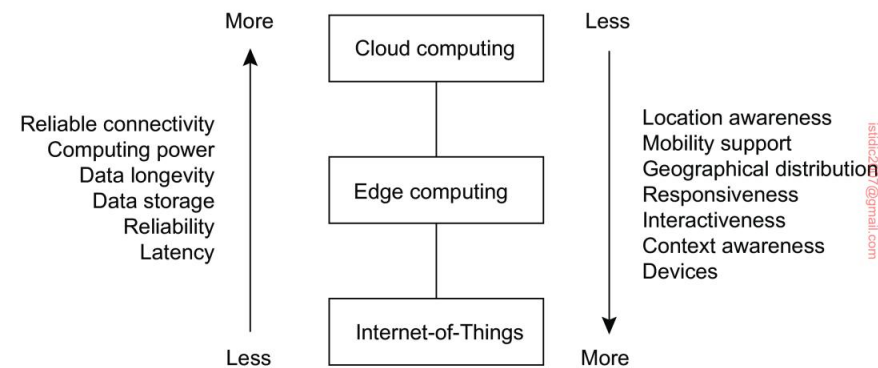
Hình 1.16: Tổ chức cơ sở dữ liệu mạng cảm biến, trong khi lưu trữ và xử lý dữ liệu (a) chỉ tại địa điểm của người vận hành hoặc (b) chỉ tại các cảm biến.

• Điều gì xảy ra khi liên kết mạng bị lỗi?

Những câu hỏi này đã được giải quyết một phần trong TinyDB, nơi triển khai giao diện khai báo (cơ sở dữ liệu) cho mạng cảm biến không dây [Madden và cộng sự, 2005]. Về bản chất, TinyDB có thể sử dụng bất kỳ thuật toán định tuyến dựa trên cây nào. Một nút trung gian sẽ thu thập và tổng hợp các kết quả từ các nút con của nó, cùng với các phát hiện của riêng nó, và gửi kết quả đó đến gốc. Để làm cho vấn đề hiệu quả hơn, các truy vấn kéo dài trong một khoảng thời gian, cho phép lập lịch cẩn thận các hoạt động để tài nguyên mạng và năng lượng được tiêu thụ tối ưu.

Tuy nhiên, khi các truy vấn có thể được khởi tạo từ các điểm khác nhau trên mạng, việc sử dụng cây đơn gốc như trong TinyDB có thể không đủ hiệu quả. Một cách khác, mạng cảm biến có thể được trang bị các nút đặc biệt nơi kết quả được chuyển tiếp đến, cũng như các truy vấn liên quan đến các kết quả đó. Để đưa ra một ví dụ đơn giản, các truy vấn và kết quả liên quan đến các phép đo nhiệt độ có thể được thu thập ở một vị trí khác so với các phép đo độ ẩm. Cách tiếp cận này tương ứng trực tiếp với khái niệm hệ thống xuất bản-đăng ký.

Điểm thú vị của mạng cảm biến, như đã thảo luận theo hướng này, là chúng ta thực sự cần tập trung vào việc tổ chức các nút cảm biến, chứ không phải bản thân các cảm biến. Tương tự như vậy, nhiều nút cảm biến sẽ được trang bị bộ truyền động, tức là các thiết bị ảnh hưởng trực tiếp đến môi trường. Một điển hình



Hình 1.17: Chế độ xem phân cấp từ đám mây đến thiết bị (được chuyển thể từ Yousef-pour et al. [2019]).

bộ truyền động là bộ điều khiển nhiệt độ trong phòng hoặc bật hoặc tắt thiết bị. Bằng cách xem và tổ chức mạng lưới như một hệ thống phân tán, người vận hành được cung cấp mức độ trừu tượng cao hơn để giám sát và kiểm soát tình hình.

Đám mây, cạnh, mọi thứ

Như có thể đã rõ ràng cho đến bây giờ, các hệ thống phân tán trải rộng trên một phạm vi rộng lớn các hệ thống máy tính được kết nối mạng khác nhau. Nhiều hệ thống như vậy hoạt động trong một bối cảnh mà các máy tính khác nhau được kết nối thông qua mạng cục bộ. Tuy nhiên, với sự phát triển của Internet vạn vật và khả năng kết nối với các dịch vụ từ xa được cung cấp thông qua các hệ thống dựa trên đám mây, các tổ chức mới trên các mạng diện rộng đang nổi lên. Hình 1.17 trình bày cách tiếp cận phân cấp hơn này .

Thông thường, ở cấp độ cao hơn trong hệ thống phân cấp, chúng ta thấy rằng các phẩm chất điển hình của hệ thống phân tán được cải thiện: chúng trở nên đáng tin cậy hơn, có nhiều khả năng hơn và nói chung là hoạt động tốt hơn. Ở cấp độ thấp hơn trong hệ thống phân cấp, chúng ta thấy rằng các khía cạnh liên quan đến vị trí được tạo điều kiện dễ dàng hơn, cũng như các phẩm chất hiệu suất liên quan đến độ trễ. Đồng thời, các phần thấp hơn cho thấy sự gia tăng về số lượng thiết bị và máy tính, trong khi ở cấp độ cao hơn, số lượng máy tính trở nên ít hơn.

1.4 Những cạm bẫy

Bây giờ thì rõ ràng là việc phát triển một hệ thống phân tán là một nhiệm vụ khó khăn. Như chúng ta sẽ thấy nhiều lần trong suốt cuốn sách này, có rất nhiều vấn đề cần xem xét, trong khi có vẻ như chỉ có sự phức tạp mới có thể là kết quả.

Tuy nhiên, bằng cách tuân theo một số nguyên tắc thiết kế, chúng ta có thể phát triển các hệ thống phân tán tuân thủ chặt chẽ các mục tiêu mà chúng tôi đề ra trong chương này.

Hệ thống phân tán khác với phần mềm truyền thống vì các thành phần được phân tán trên một mạng lưới. Việc không tính đến sự phân tán này trong thời gian thiết kế là nguyên nhân khiến nhiều hệ thống trở nên phức tạp không cần thiết và dẫn đến các lỗi cần phải vá sau này. Peter Deutsch, khi đó đang làm việc tại Sun Microsystems, đã xây dựng các lỗi này thành các giả định sai lầm sau đây mà nhiều người mắc phải khi phát triển ứng dụng phân tán lần đầu tiên:

- Mạng lưới đáng tin cậy
- Mạng được bảo mật
- Mạng đồng nhất • Cấu trúc mạng không thay đổi • Độ trễ bằng không • Băng thông vô hạn

- Chi phí vận chuyển bằng 0 •
- Có một người quản lý

Lưu ý cách các giả định này liên quan đến các thuộc tính độc đáo đối với các hệ thống phân tán: độ tin cậy, bảo mật, tính không đồng nhất và cấu trúc mạng; độ trễ và băng thông; chi phí vận chuyển; và cuối cùng là các miền quản trị. Khi phát triển các ứng dụng không phân tán, hầu hết các vấn đề này rất có thể sẽ không xuất hiện.

Hầu hết các nguyên tắc chúng ta thảo luận trong cuốn sách này đều liên quan trực tiếp đến các giả định này. Trong mọi trường hợp, chúng ta sẽ thảo luận về các giải pháp cho các vấn đề phát sinh do một hoặc nhiều giả định là sai. Ví dụ, các mạng đáng tin cậy đơn giản là không tồn tại và dẫn đến việc không thể đạt được tính minh bạch về lỗi. Chúng tôi dành toàn bộ một chương để giải quyết thực tế là giao tiếp mạng vốn không an toàn. Chúng tôi đã lập luận rằng các hệ thống phân tán cần phải mở và tính đến tính không đồng nhất. Tương tự như vậy, khi thảo luận về sao chép để giải quyết các vấn đề về khả năng mở rộng, về cơ bản chúng ta đang giải quyết các vấn đề về độ trễ và băng thông. Chúng ta cũng sẽ đề cập đến các vấn đề quản lý tại nhiều điểm khác nhau trong suốt cuốn sách này.

1.5 Tóm tắt

Hệ thống phân tán là tập hợp các hệ thống máy tính được kết nối mạng trong đó các quy trình và tài nguyên được phân tán trên nhiều máy tính khác nhau. Chúng tôi phân biệt giữa phân tán đủ và phân tán cần thiết, trong đó phân tán cần thiết liên quan đến các hệ thống phi tập trung. Sự phân biệt này rất quan trọng vì việc phân tán các quy trình và tài nguyên không thể được coi là mục tiêu riêng lẻ. Thay vào đó,

hầu hết các lựa chọn để đến với hệ thống phân tán đều xuất phát từ nhu cầu cải thiện hiệu suất của một hệ thống máy tính duy nhất về mặt, ví dụ, độ tin cậy, khả năng mở rộng và hiệu quả. Tuy nhiên, xét đến việc hầu hết các hệ thống tập trung vẫn dễ quản lý và bảo trì hơn nhiều, người ta nên cân nhắc kỹ trước khi quyết định phân tán các quy trình và tài nguyên. Cũng có những trường hợp đơn giản là không có lựa chọn nào, ví dụ như khi kết nối các hệ thống thuộc về các tổ chức khác nhau hoặc khi máy tính chỉ hoạt động từ các vị trí khác nhau (như trong điện toán di động).

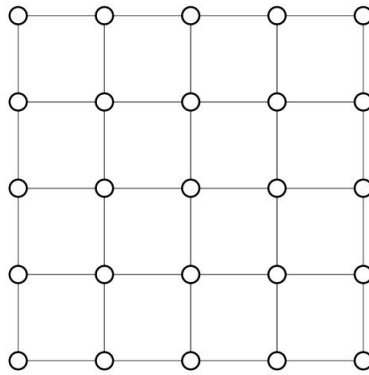
Mục tiêu thiết kế cho các hệ thống phân tán bao gồm chia sẻ tài nguyên và đảm bảo tính mở. Ngày càng quan trọng là thiết kế các hệ thống phân tán an toàn. Ngoài ra, các nhà thiết kế hướng đến việc ẩn nhiều sự phức tạp liên quan đến việc phân phối các quy trình, dữ liệu và kiểm soát. Tuy nhiên, tính minh bạch phân phối này không chỉ phải trả giá bằng hiệu suất, trong các tình huống thực tế, nó không bao giờ có thể đạt được hoàn toàn. Thực tế là cần phải đánh đổi giữa việc đạt được các hình thức minh bạch phân phối khác nhau vốn có trong thiết kế của các hệ thống phân tán và có thể dễ dàng làm phức tạp thêm việc hiểu chúng. Một mục tiêu thiết kế khó khăn cụ thể không phải lúc nào cũng hòa hợp tốt với việc đạt được tính minh bạch phân phối là khả năng mở rộng. Điều này đặc biệt đúng đối với khả năng mở rộng theo địa lý, trong trường hợp đó, việc ẩn độ trễ và hạn chế băng thông có thể trở nên khó khăn. Tương tự như vậy, khả năng mở rộng quản trị, theo đó một hệ thống được thiết kế để trải rộng trên nhiều miền quản trị, có thể dễ dàng xung đột với các mục tiêu đạt được tính minh bạch phân phối.

Có nhiều loại hệ thống phân tán khác nhau có thể được phân loại thành hướng đến hỗ trợ tính toán, xử lý thông tin và tính phổ biến. Các hệ thống điện toán phân tán thường được triển khai cho các ứng dụng hiệu suất cao, thường bắt nguồn từ lĩnh vực điện toán song song. Một lĩnh vực xuất hiện từ xử lý song song ban đầu là điện toán lưu trữ với trọng tâm là chia sẻ tài nguyên trên toàn thế giới, dẫn đến cái mà hiện nay được gọi là điện toán đám mây. Điện toán đám mây không chỉ dừng lại ở điện toán hiệu suất cao mà còn hỗ trợ các hệ thống phân tán được tìm thấy trong môi trường văn phòng truyền thống, nơi chúng ta thấy cơ sở dữ liệu đóng vai trò quan trọng. Thông thường, các hệ thống xử lý giao dịch được triển khai trong các môi trường này. Cuối cùng, một lớp hệ thống phân tán mới nổi là nơi các thành phần nhỏ, hệ thống được tạo thành theo cách tùy ý, nhưng trên hết là không còn được quản lý thông qua quản trị viên hệ thống nữa. Lớp cuối cùng này thường được biểu diễn bằng các môi trường điện toán phổ biến, bao gồm cả hệ thống điện toán di động cũng như môi trường giàu cảm biến.

Vấn đề còn phức tạp hơn nữa khi nhiều nhà phát triển ban đầu đưa ra các giả định về mạng cơ bản là sai. Sau đó, khi các giả định bị loại bỏ, có thể sẽ khó che giấu hành vi không mong muốn. Một ví dụ điển hình là giả định rằng độ trễ của mạng không đáng kể. Những phạm vi khác bao gồm giả định rằng mạng đáng tin cậy, tĩnh, an toàn và đồng nhất.

02

KIẾN TRÚC



Hệ thống phân tán thứ ờng là những phần mềm phức tạp, trong đó các thành phần theo định nghĩa đợc phân tán trên nhiều máy. Để làm chủ đợc sự phức tạp của chúng, điều quan trọng là các hệ thống này phải đợc tổ chức hợp lý. Có nhiều cách khác nhau để xem xét tổ chức của một hệ thống phân tán, nhưng một cách rõ ràng là phân biệt giữa, một mặt, tổ chức logic của bộ sũ u tập các thành phần phần mềm và mặt khác là hiện thực vật lý thực tế.

Tổ chức của các hệ thống phân tán chủ yếu liên quan đến các thành phần phần mềm tạo nên hệ thống. Các kiến trúc phần mềm này cho chúng ta biết cách tổ chức các thành phần phần mềm khác nhau và cách chúng tương tác. Trong chương này, trước tiên chúng ta sẽ chú ý đến một số phong cách kiến trúc thứ ờng đợc áp dụng để tổ chức các hệ thống máy tính (phân tán).

Một mục tiêu quan trọng của các hệ thống phân tán là tách các ứng dụng khỏi các nền tảng cơ bản bằng cách cung cấp cái gọi là lớp phần mềm trung gian. Việc áp dụng một lớp như vậy là một quyết định quan trọng về mặt kiến trúc và mục đích chính của nó là cung cấp tính minh bạch phân phối. Tuy nhiên, cần phải có sự đánh đổi để đạt đợc tính minh bạch, điều này đã dẫn đến nhiều kỹ thuật khác nhau để điều chỉnh phần mềm trung gian theo nhu cầu của các ứng dụng sử dụng nó. Chúng tôi thảo luận về một số kỹ thuật đợc áp dụng phổ biến hơn, vì chúng ảnh hưởng đến việc tổ chức chính phần mềm trung gian.

Việc thực hiện thực tế một hệ thống phân tán đòi hỏi chúng ta phải khởi tạo và đặt các thành phần phần mềm trên các máy thực. Có nhiều lựa chọn có thể đợc thực hiện khi thực hiện như vậy. Khởi tạo cuối cùng của một kiến trúc phần mềm cũng đợc gọi là kiến trúc hệ thống. Trong chương này, chúng ta sẽ xem xét các kiến trúc tập trung truyền thống trong đó một máy chủ duy nhất triển khai hầu hết các thành phần phần mềm (và do đó là chức năng), trong khi các máy khách từ xa có thể truy cập máy chủ đó bằng các phương tiện truyền thông đơn giản. Ngoài ra, chúng ta xem xét các kiến trúc ngang hàng phi tập trung trong đó tất cả các nút ít nhiều đều đóng vai trò ngang nhau. Nhiều hệ thống phân tán trong thế giới thực thường đợc tổ chức theo kiểu kết hợp, kết hợp các yếu tố từ kiến trúc tập trung và phi tập trung. Chúng ta thảo luận về một số ví dụ minh họa cho sự phức tạp của nhiều hệ thống phân tán trong thế giới thực.

2.1 Phong cách kiến trúc

Chúng tôi bắt đầu thảo luận về kiến trúc bằng cách đầu tiên xem xét tổ chức logic của một hệ thống phân tán thành các thành phần phần mềm, còn đợc gọi là kiến trúc phần mềm của nó [Bass et al., 2021; Richards và Ford, 2020].

Nghiên cứu về kiến trúc phần mềm đã phát triển đáng kể và hiện nay người ta thường chấp nhận rằng việc thiết kế hoặc áp dụng kiến trúc là rất quan trọng đối với sự phát triển thành công của các hệ thống phần mềm lớn.

Đối với cuộc thảo luận của chúng ta, khái niệm về phong cách kiến trúc là quan trọng. Một phong cách như vậy đợc hình thành theo các thành phần, cách mà các thành phần đợc

được kết nối với nhau, dữ liệu được trao đổi giữa các thành phần và cuối cùng, cách các thành phần này được cấu hình chung thành một hệ thống. Một thành phần là một đơn vị mô-đun có các giao diện được yêu cầu và cung cấp được xác định rõ ràng, có thể thay thế trong môi trường của nó [OMG, 2004]. Việc một thành phần có thể được thay thế và đặc biệt là trong khi hệ thống vẫn tiếp tục hoạt động là điều quan trọng.

Lý do là vì thông thường không có lựa chọn tất hệ thống để bảo trì.

Tốt nhất, chỉ một số phần của nó có thể tạm thời bị hỏng. Chỉ có thể thay thế một thành phần nếu giao diện của nó vẫn còn nguyên vẹn. Trong thực tế, chúng ta thấy rằng việc thay thế hoặc cập nhật một thành phần có nghĩa là một phần của hệ thống (chẳng hạn như máy chủ) chạy bản cập nhật thường xuyên và chuyển sang các thành phần được làm mới sau khi cài đặt hoàn tất. Có thể cần phải thực hiện các biện pháp đặc biệt khi một phần của hệ thống phân tán cần được khởi động lại để các bản cập nhật có hiệu lực. Các biện pháp như vậy có thể bao gồm việc sao chép các chế độ chờ để tiếp quản trong khi quá trình khởi động lại một phần đang diễn ra.

Một khái niệm khó nắm bắt hơn một chút là khái niệm về bộ kết nối, thường được mô tả là một cơ chế làm trung gian cho giao tiếp, phối hợp hoặc hợp tác giữa các thành phần [Bass và cộng sự, 2021]. Ví dụ, bộ kết nối có thể được hình thành bởi các tiện ích cho các cuộc gọi thủ tục (từ xa), truyền tin nhắn hoặc dữ liệu phát trực tuyến. Nói cách khác, bộ kết nối cho phép luồng điều khiển và dữ liệu giữa các thành phần.

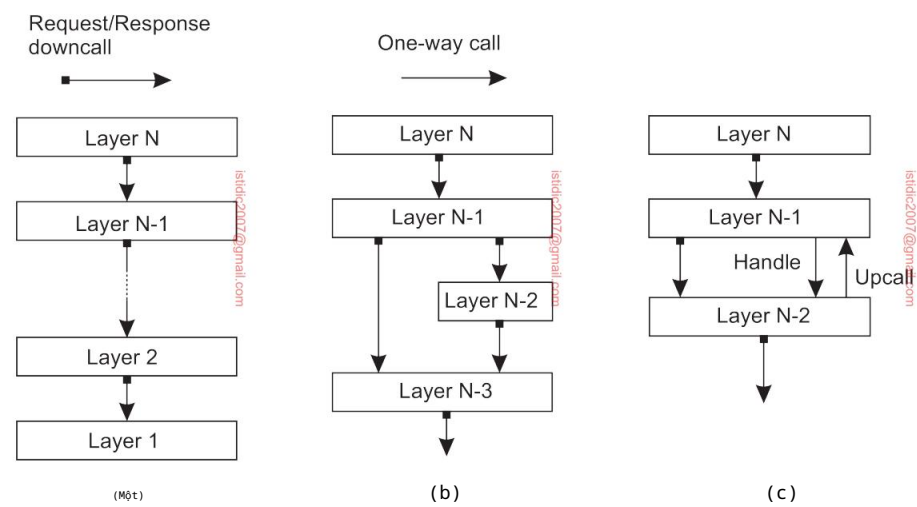
Sử dụng các thành phần và kết nối, chúng ta có thể đưa ra nhiều cấu hình khác nhau, lần lượt được phân loại thành các kiểu kiến trúc. Một số kiểu đã được xác định, trong đó những kiểu quan trọng nhất đối với các hệ thống phân tán là:

- Kiến trúc phân lớp • Kiến trúc hướng dịch vụ
- Kiến trúc xuất bản-đăng ký

Sau đây, chúng tôi sẽ thảo luận riêng từng phong cách này. Chúng tôi lưu ý trước rằng trong hầu hết các hệ thống phân tán trong thế giới thực, nhiều phong cách được kết hợp. Đáng chú ý, việc áp dụng phương pháp chia hệ thống thành nhiều lớp (hợp lý) là một nguyên tắc phổ biến đến mức nó thường được kết hợp với hầu hết các phong cách kiến trúc khác.

2.1.1 Kiến trúc phân lớp

Ý tưởng cơ bản cho phong cách phân lớp rất đơn giản: các thành phần được tổ chức theo kiểu phân lớp, trong đó một thành phần ở lớp L_j có thể thực hiện lệnh gọi xuống một thành phần ở lớp cấp thấp hơn L_i (với $i < j$) và thường mong đợi phản hồi. Chỉ trong những trường hợp ngoại lệ, lệnh gọi lên mới được thực hiện đến một thành phần cấp cao hơn. Ba trường hợp phổ biến được thể hiện trong Hình 2.1.



Hình 2.1: (a) Tổ chức phân lớp thuần túy. (b) Tổ chức phân lớp hỗn hợp. (c) Tổ chức phân lớp với upcall (áp dụng từ [Krakowiak, 2009]).

Hình 2.1(a) cho thấy một tổ chức chuẩn trong đó chỉ có các cuộc gọi xuống lớp thấp hơn tiếp theo được thực hiện. Tổ chức này thường được triển khai trong trường hợp truyền thông mạng.

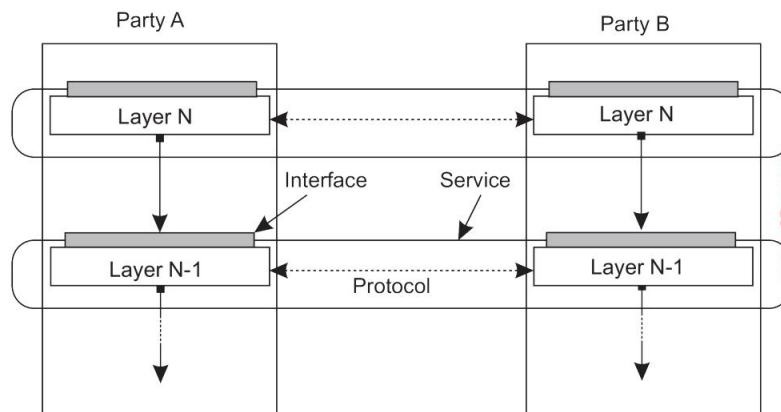
Trong nhiều tình huống, chúng ta cũng gặp phải tổ chức được thể hiện trong Hình 2.1 (b). Ví dụ, hãy xem xét một ứng dụng A sử dụng thư viện LOS để giao tiếp với hệ điều hành. Đồng thời, ứng dụng sử dụng thư viện toán học chuyên biệt Lmath đã được triển khai bằng cách cũng sử dụng LOS. Trong trường hợp này, tham chiếu đến Hình 2.1(b), A được triển khai ở lớp N-1, Lmath ở lớp N-2 và LOS chung cho cả hai, ở lớp N-3.

Cuối cùng, một tình huống đặc biệt được thể hiện trong Hình 2.1(c). Trong một số trường hợp, sẽ thuận tiện hơn nếu một lớp thấp hơn thực hiện lệnh upcall lên lớp cao hơn tiếp theo. Một ví dụ điển hình là khi một hệ điều hành báo hiệu sự kiện xảy ra, để kết thúc, nó gọi một hoạt động do người dùng xác định mà ứng dụng đã truyền tham chiếu trước đó (thường được gọi là handle).

Giao thức truyền thông nhiều lớp

Một kiến trúc phân lớp được biết đến rộng rãi và được áp dụng phổ biến là các ngăn xếp giao thức truyền thông. Chúng tôi sẽ tập trung ở đây chỉ vào bức tranh toàn cầu và hoãn thảo luận chi tiết cho Mục 4.1.1.

Trong các ngăn xếp giao thức truyền thông, mỗi lớp triển khai một hoặc nhiều dịch vụ truyền thông cho phép dữ liệu được gửi từ một đích đến một hoặc nhiều mục tiêu. Để đạt được mục đích này, mỗi lớp cung cấp một giao diện chỉ định



Hình 2.2: Một ngăn xếp giao thức truyền thông nhiều lớp, cho thấy sự khác biệt giữa một dịch vụ, giao diện của nó và giao thức mà nó triển khai.

các chức năng có thể được gọi. Về nguyên tắc, giao diện phải ẩn hoàn toàn việc triển khai thực tế của một dịch vụ. Một khái niệm quan trọng khác trong trừu tượng giao tiếp là giao thức (giao tiếp), mô tả các quy tắc mà các bên sẽ tuân theo để trao đổi thông tin. Điều quan trọng là phải hiểu sự khác biệt giữa dịch vụ do một lớp cung cấp, giao diện mà dịch vụ đó được cung cấp và giao thức mà một lớp triển khai để thiết lập giao tiếp. Sự khác biệt này được thể hiện trong Hình 2.2.

Để làm rõ sự khác biệt này, hãy xem xét một dịch vụ đáng tin cậy, hướng kết nối, được cung cấp bởi nhiều hệ thống truyền thông. Trong trừu tượng này, một bên giao tiếp trước tiên cần thiết lập kết nối với một bên khác trước khi cả hai có thể gửi và nhận tin nhắn. Độ tin cậy có nghĩa là sẽ có sự đảm bảo chắc chắn rằng các tin nhắn đã gửi thực sự sẽ được chuyển đến bên kia, ngay cả khi có nguy cơ cao là tin nhắn có thể bị mất (ví dụ, có thể xảy ra khi sử dụng phương tiện không dây). Ngoài ra, các dịch vụ như vậy thường cũng đảm bảo rằng các tin nhắn được chuyển theo cùng thứ tự như khi chúng được gửi.

Loại dịch vụ này được thực hiện trên Internet bằng Giao thức điều khiển truyền (TCP). Giao thức này chỉ định những thông điệp nào sẽ được trao đổi để thiết lập hoặc hủy kết nối, những gì cần phải làm để bảo toàn thứ tự của dữ liệu được truyền và những gì cả hai bên cần phải làm để phát hiện và sửa dữ liệu bị mất trong quá trình truyền. Dịch vụ được cung cấp dưới dạng giao diện lập trình ứng dụng đơn giản, bao gồm các lệnh gọi để thiết lập kết nối, gửi và nhận thông điệp và hủy kết nối một lần nữa. Trên thực tế, có nhiều giao diện khác nhau, thường phụ thuộc vào hệ điều hành hoặc ngôn ngữ lập trình được sử dụng. Tương tự như vậy, có nhiều cách triển khai giao thức và các giao diện của nó. (Tất cả các chi tiết đầy đủ có thể được tìm thấy trong [Stevens, 1994; Wright và Stevens, 1995].)

Lưu ý 2.1 (Ví dụ: Hai bên giao tiếp)

Để phân biệt rõ hơn giữa dịch vụ, giao diện và giao thức, hãy xem xét hai bên giao tiếp sau, còn được gọi là máy khách và máy chủ, được diễn đạt bằng Python (lưu ý rằng một số mã đã bị xóa để rõ ràng hơn).

```
1 từ ổ cắm nhập *
2

3 s = socket(AF_INET, SOCK_STREAM) 4 (conn, addr)
= s.accept() # trả về socket và addr mới. client 5 while True:
# mãi mãi

6 data = conn.recv(1024) # nhận dữ liệu từ máy khách 7 nếu không phải dữ liệu:
break # dừng nếu máy khách dừng 8 msg = data.decode()*** # xử lý dữ liệu
đến thành phần hồi 9 conn.send(msg.encode()) # trả về phản hồi 10 conn.close() # đóng kết nối
```

(a) Một máy chủ đơn giản

```
1 từ ổ cắm nhập *
2

3 s = socket(AF_INET, SOCK_STREAM) 4
s.connect((HOST, PORT)) # kết nối tới máy chủ (chặn cho đến khi được chấp nhận) 5 msg =
"Hello World" 6 # soạn tin nhắn # gửi tin
s.send(msg.encode()) 7 data # nhận phản hồi # in
= s.recv(1024) 8 kết quả # đóng kết nối
print(data.decode()) 9
s.close()
```

(b) Một khách hàng

Hình 2.3: Hai bên giao tiếp.

Trong ví dụ này, một máy chủ được tạo ra sử dụng dịch vụ hướng kết nối như được cung cấp bởi thư viện socket có sẵn trong Python. Dịch vụ này cho phép hai bên giao tiếp gửi và nhận dữ liệu đáng tin cậy qua kết nối.

Các chức năng chính có trong giao diện của nó là:

- `socket()`: để tạo một đối tượng biểu diễn kết nối
- `accept()`: một lệnh gọi chặn để chờ các yêu cầu kết nối đến; nếu thành công, lệnh gọi sẽ trả về một socket mới cho một kết nối riêng biệt
- `connect()`: để thiết lập kết nối với một bên được chỉ định
- `close()`: để hủy kết nối
- `send()`, `recv()`: để gửi và nhận dữ liệu qua kết nối tương ứng

Sự kết hợp của hằng số `AF_INET` và `SOCK_STREAM` được sử dụng để chỉ rõ giao thức TCP sẽ được sử dụng trong giao tiếp giữa hai bên.

Hai hằng số này có thể được xem như một phần của giao diện, trong khi việc sử dụng TCP là một phần của dịch vụ được cung cấp. Cách triển khai TCP, hoặc bất kỳ phần nào của dịch vụ truyền thông, đều được ẩn hoàn toàn khỏi các ứng dụng.

Cuối cùng, cũng lưu ý rằng hai chương trình này tuân thủ ngầm một giao thức cấp ứng dụng: rõ ràng là nếu máy khách gửi dữ liệu, máy chủ sẽ trả về. Trên thực tế, nó hoạt động như một máy chủ phản hồi, trong đó máy chủ thêm dấu hoa thị vào dữ liệu do máy khách gửi đi.

Phân lớp ứng dụng

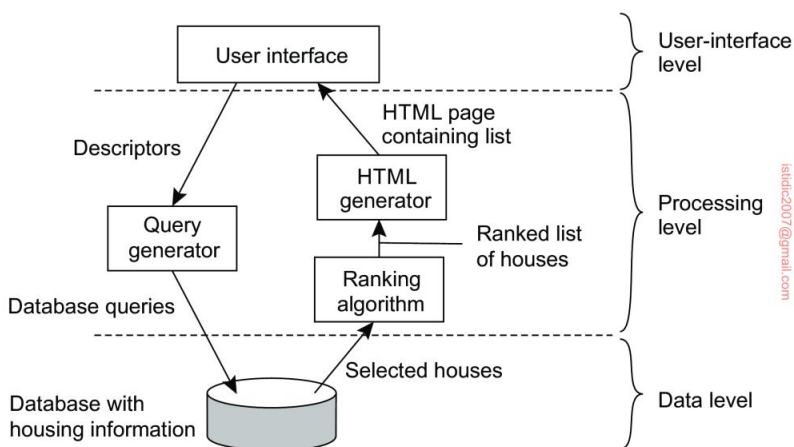
Bây giờ chúng ta hãy chuyển sự chú ý của mình sang lớp logic của các ứng dụng. Xem xét rằng một lớp lớn các ứng dụng phân tán được nhắm mục tiêu vào việc hỗ trợ người dùng hoặc ứng dụng truy cập vào cơ sở dữ liệu, nhiều người đã ủng hộ sự phân biệt giữa ba cấp logic, về cơ bản là theo phong cách kiến trúc phân lớp:

- Mức giao diện ứng dụng • Mức xử lý
- Mức dữ liệu

Phù hợp với lớp này, chúng ta thấy rằng các ứng dụng thư ờng có thể được xây dựng từ khoảng ba phần khác nhau: một phần xử lý tư ơng tác với người dùng hoặc một số ứng dụng bên ngoài, một phần hoạt động trên cơ sở dữ liệu hoặc hệ thống tệp và một phần giữa thư ờng chứa chức năng cốt lõi của ứng dụng. Phần giữa này được đặt hợp lý ở cấp độ xử lý. Trái ngược với giao diện người dùng và cơ sở dữ liệu, không có nhiều khía cạnh chung cho cấp độ xử lý. Do đó, chúng tôi sẽ đưa ra một số ví dụ để làm rõ hơn n cấp độ này .

Ví dụ đầu tiên, hãy xem xét một công cụ tìm kiếm Internet đơn giản, ví dụ như một công cụ dành riêng cho việc mua nhà. Các công cụ tìm kiếm như vậy xuất hiện dưới dạng các trang web có vẻ đơn giản mà qua đó ai đó có thể cung cấp các mô tả như thành phố hoặc khu vực, phạm vi giá, loại nhà, v.v. Phần cuối bao gồm một cơ sở dữ liệu khổng lồ về các ngôi nhà hiện đang được rao bán. Lớp xử lý không làm gì khác ngoài việc chuyển đổi các mô tả được cung cấp thành một tập hợp các truy vấn cơ sở dữ liệu, truy xuất các câu trả lời và xử lý hậu kỳ các câu trả lời này bằng cách, ví dụ, xếp hạng đầu ra theo mức độ liên quan và sau đó tạo ra một trang HTML. Hình 2.4 cho thấy cách tổ chức này.

Một ví dụ khác, hãy xem xét tổ chức của Trang web của cuốn sách này, cụ thể là giao diện cho phép ai đó nhận được bản sao kỹ thuật số được cá nhân hóa của cuốn sách ở định dạng PDF. Trong trư ờng hợp này, giao diện bao gồm một máy chủ Web dựa trên WordPress chỉ thu thập địa chỉ email của người dùng (và một số thông tin về phiên bản chính xác của cuốn sách đang được yêu cầu). Thông tin này được thêm vào bên trong tệp requests.txt. Lớp dữ liệu rất đơn giản: nó chỉ bao gồm một tập hợp các tệp LATEX và hình ảnh cùng nhau tạo nên toàn bộ cuốn sách.



Hình 2.4: Tổ chức đơn giản của công cụ tìm kiếm nhà ở trên Internet thành ba lớp khác nhau.

Tạo một bản sao được cá nhân hóa bao gồm những địa chỉ email của người dùng vào từng hình ảnh. Để thực hiện mục đích này, cứ năm phút một lần, một quy trình riêng biệt sẽ được bắt đầu, quy trình này sẽ lấy danh sách các yêu cầu và thêm từng địa chỉ email của người yêu cầu vào từng hình ảnh bitmap, tạo một bản sao mới của cuốn sách, lưu trữ tệp PDF đã tạo ở một vị trí đặc biệt (có thể truy cập thông qua một URL duy nhất) và gửi cho người yêu cầu một email thông báo rằng bản sao đã có sẵn để tải xuống. Quy trình này tiếp tục cho đến khi tất cả các yêu cầu đã được xử lý. Trong ví dụ này, do đó, chúng ta thấy rằng lớp xử lý quan trọng hơn lớp dữ liệu hoặc lớp giao diện ứng dụng về mặt nỗ lực tính toán và hành động cần thực hiện.

2.1.2 Kiến trúc hướng dịch vụ

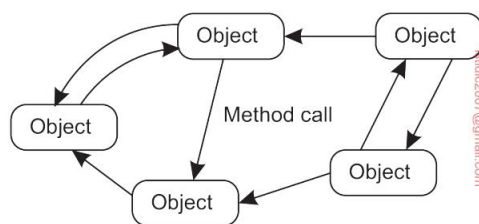
Mặc dù phong cách kiến trúc nhiều lớp rất phổ biến, nhưng một trong những nhược điểm chính của nó là sự phụ thuộc thứ tự xuyên giữa các lớp khác nhau. Các ví dụ điển hình về việc các phụ thuộc tiềm ẩn này đã được cân nhắc kỹ lưỡng có thể thấy trong việc thiết kế các ngăn xếp giao thức truyền thông. Các ví dụ điển hình bao gồm các ứng dụng về cơ bản được thiết kế và phát triển như các thành phần của các thành phần hiện có mà không quan tâm nhiều đến tính ổn định của giao diện hoặc bản thân các thành phần, chứ không nói đến sự chồng chéo về chức năng giữa các thành phần khác nhau. (Một ví dụ thuyết phục được đưa ra bởi Kucharski [2020], người mô tả sự phụ thuộc vào một thành phần đơn giản chèn một chuỗi nhất định bằng số không hoặc khoảng trắng. Tác giả đã rút thành phần này khỏi thư viện NPM, khiến hàng nghìn chương trình bị ảnh hưởng.)

Những sự phụ thuộc trực tiếp như vậy vào các thành phần cụ thể đã dẫn đến một phong cách kiến trúc phản ánh một tổ chức lỏng lẻo hơn thành một tập hợp các thành phần riêng biệt,

các thực thể độc lập. Mỗi thực thể đóng gói một dịch vụ. Cho dù chúng được gọi là dịch vụ, đối tượng hay dịch vụ siêu nhỏ, chúng đều có điểm chung là dịch vụ được thực thi như một quy trình (hoặc luồng) riêng biệt. Tất nhiên, việc chạy các thực thể riêng biệt không nhất thiết làm giảm sự phụ thuộc so với kiểu kiến trúc phân lớp.

Phong cách kiến trúc dựa trên đối tượng

Lấy cách tiếp cận dựa trên đối tượng làm ví dụ, chúng ta có một tổ chức logic như thể hiện trong Hình 2.5. Về bản chất, mỗi đối tượng tương ứng với những gì chúng ta đã định nghĩa là một thành phần và các thành phần này được kết nối thông qua cơ chế gọi thủ tục. Trong trường hợp của các hệ thống phân tán, một cuộc gọi thủ tục cũng có thể diễn ra qua mạng, nghĩa là đối tượng gọi không cần phải được thực thi trên cùng một máy với đối tượng được gọi. Trên thực tế, vị trí chính xác của đối tượng được gọi có thể trong suốt đối với người gọi: đối tượng được gọi cũng có thể chạy như một quy trình riêng biệt trên cùng một máy.

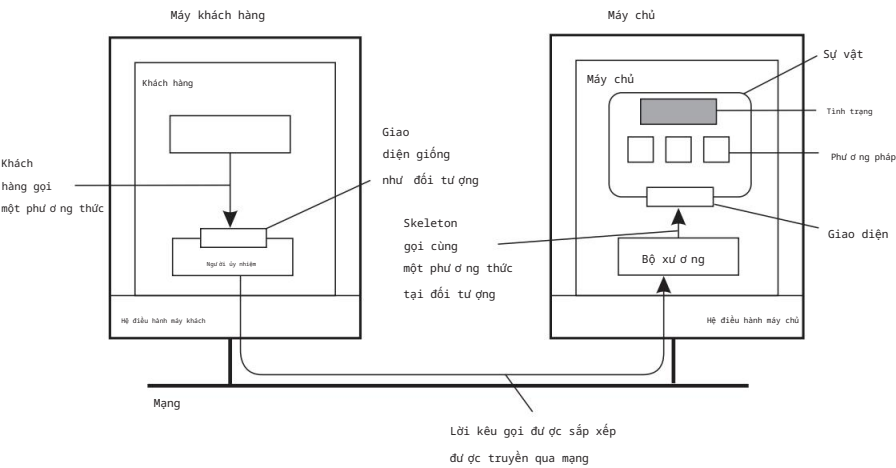


Hình 2.5: Phong cách kiến trúc dựa trên đối tượng.

Kiến trúc dựa trên đối tượng hấp dẫn vì chúng cung cấp một cách tự nhiên để đóng gói dữ liệu (gọi là trạng thái của đối tượng) và các hoạt động có thể được thực hiện trên dữ liệu đó (được gọi là phương thức của đối tượng) thành một thực thể duy nhất. Giao diện do đối tượng cung cấp che giấu các chi tiết triển khai, về cơ bản có nghĩa là về nguyên tắc, chúng ta có thể xem xét một đối tượng hoàn toàn độc lập với môi trường của nó. Cũng giống như các thành phần, điều này cũng có nghĩa là nếu giao diện được xác định rõ ràng và không bị thay đổi, thì đối tượng đó có thể được thay thế bằng đối tượng có cùng giao diện.

Sự tách biệt giữa các giao diện và các đối tượng triển khai các giao diện này cho phép chúng ta đặt một giao diện tại một máy, trong khi bản thân đối tượng nằm trên một máy khác. Tổ chức này, được thể hiện trong Hình 2.6, thường được gọi là đối tượng phân tán hoặc đôi khi là đối tượng từ xa.

Khi một máy khách liên kết với một đối tượng phân tán, một triển khai giao diện của đối tượng, được gọi là proxy, sau đó được tải vào không gian địa chỉ của máy khách. Proxy tương tự như cái gọi là client stub trong hệ thống RPC. Điều duy nhất nó làm là đóng gói các lệnh gọi phương thức vào các thông điệp và giải nén các thông điệp trả lời để trả về kết quả của lệnh gọi phương thức cho máy khách. Đối tượng thực tế



Hình 2.6: Tổ chức chung của một đối tượng từ xa với proxy phía máy khách.

nằm tại máy chủ, nơi nó cung cấp cùng một giao diện như trên máy khách. Các yêu cầu gọi đến trước tiên được chuyển đến một stub máy chủ, stub này sẽ giải nén chúng để thực hiện các lệnh gọi phương thức tại giao diện của đối tượng tại máy chủ. Stub máy chủ cũng chịu trách nhiệm đóng gói các giá trị trả về thành một thông báo và chuyển tiếp các thông báo trả lời này đến proxy phía máy khách.

Stub phía máy chủ thường được gọi là bộ khung vì nó cung cấp phương tiện cơ bản để cho phép phần mềm trung gian của máy chủ truy cập vào các đối tượng do người dùng xác định. Trên thực tế, nó thường chứa mã không đầy đủ dưới dạng lớp ngôn ngữ cụ thể cần được nhà phát triển chuyên môn hóa thêm.

Một đặc điểm, nhưng có phần trái ngược với trực giác, của hầu hết các đối tượng phân tán là trạng thái của chúng không được phân tán: nó nằm ở một máy duy nhất. Chỉ các giao diện được đối tượng triển khai mới có sẵn trên các máy khác. Các đối tượng như vậy cũng được gọi là các đối tượng từ xa. Trong một đối tượng phân tán chung, bản thân trạng thái có thể được phân phối vật lý trên nhiều máy, nhưng sự phân phối này cũng bị ẩn khỏi các máy khách đằng sau các giao diện của đối tượng.

Phong cách kiến trúc vi dịch vụ

Người ta có thể lập luận rằng kiến trúc dựa trên đối tượng tạo thành nền tảng cho việc đóng gói các dịch vụ thành các đơn vị độc lập. Đóng gói là từ khóa ở đây: dịch vụ nói chung được hiện thực hóa như một thực thể độc lập, mặc dù nó có thể sử dụng các dịch vụ khác. Bằng cách phân tách rõ ràng các dịch vụ khác nhau sao cho chúng có thể hoạt động độc lập, chúng ta đang mở đường cho các kiến trúc hướng dịch vụ, thường được viết tắt là SOA.

Được kích thích bởi các thiết kế hướng đối tượng và lấy cảm hứng từ cách tiếp cận Unix trong đó rất nhiều chương trình nhỏ và độc lập lẫn nhau có thể dễ dàng

được biên soạn để tạo thành các chương trình lớn hơn, các kiến trúc sư phần mềm đã và đang làm việc trên cái gọi là microservice. Điều cần thiết là mỗi microservice chạy như một quy trình (mạng) riêng biệt. Việc triển khai microservice có thể ở dạng đối tượng từ xa, nhưng đây không phải là yêu cầu bắt buộc. Hơn nữa, mặc dù mọi người nói về microservice, nhưng không có sự đồng thuận chung về quy mô của một dịch vụ như vậy. Tuy nhiên, điều quan trọng nhất là microservice thực sự đại diện cho một dịch vụ riêng biệt, độc lập. Nói cách khác, mô-đun hóa là chìa khóa để thiết kế microservice [Wolff, 2017].

Tuy nhiên, kích thước vẫn quan trọng. Bằng cách nêu rằng các dịch vụ siêu nhỏ chạy như các quy trình mạng riêng biệt, chúng ta cũng được lựa chọn nơi đặt dịch vụ siêu nhỏ. Như chúng ta sẽ thấy sau trong chương này, trong sự ra đời của cơ sở hạ tầng biên và sơ đồ mô-đun, các cuộc thảo luận đã bắt đầu về việc sắp xếp triển khai các ứng dụng phân tán trên các lớp khác nhau. Nói cách khác, chúng ta đặt cái gì ở đâu.

Thành phần dịch vụ thô

Trong kiến trúc hướng dịch vụ, một ứng dụng hoặc hệ thống phân tán về cơ bản được xây dựng như một thành phần của nhiều dịch vụ. Một điểm khác biệt (mặc dù không nghiêm ngặt) với các dịch vụ siêu nhỏ là không phải tất cả các dịch vụ này đều có thể thuộc về cùng một tổ chức quản trị. Chúng ta đã gặp hiện tượng này khi thảo luận về điện toán đám mây: rất có thể một tổ chức đang chạy ứng dụng kinh doanh của mình sẽ sử dụng các dịch vụ lưu trữ do nhà cung cấp đám mây cung cấp. Các dịch vụ lưu trữ này về mặt logic được đóng gói hoàn toàn vào một đơn vị duy nhất, trong đó có một giao diện được cung cấp cho khách hàng.

Tất nhiên, lưu trữ là một dịch vụ khá cơ bản, nhưng những tình huống phức tạp hơn dễ dàng xuất hiện trong đầu. Ví dụ, hãy xem xét một cửa hàng Web bán hàng hóa như sách điện tử. Một triển khai đơn giản, theo lớp ứng dụng mà chúng ta đã thảo luận trước đó, có thể bao gồm một ứng dụng để xử lý đơn hàng, đến lượt nó, hoạt động trên cơ sở dữ liệu cục bộ chứa sách điện tử. Xử lý đơn hàng thường bao gồm việc chọn mặt hàng, đăng ký và kiểm tra kênh giao hàng (có thể bằng cách sử dụng email), nhưng cũng đảm bảo rằng thanh toán diễn ra. Sau này có thể được xử lý bởi một dịch vụ riêng biệt, do một tổ chức khác điều hành, nơi khách hàng mua hàng được chuyển hướng để thanh toán, sau đó tổ chức sách điện tử được thông báo để có thể hoàn tất giao dịch. Ví dụ này cũng minh họa rằng khi các dịch vụ siêu nhỏ được coi là tương đối nhỏ, thì một dịch vụ chung có thể được mong đợi là tương đối lớn. Trên thực tế, không có gì lạ khi triển khai một dịch vụ dưới dạng tập hợp các dịch vụ siêu nhỏ.

Theo cách này, chúng ta thấy rằng vấn đề phát triển hệ thống phân tán một phần là do thành phần dịch vụ và đảm bảo rằng các dịch vụ đó hoạt động hài hòa. Thật vậy, vấn đề này hoàn toàn tương tự với các vấn đề tích hợp ứng dụng doanh nghiệp được thảo luận trong Phần 1.3.2. Điều quan trọng là, và vẫn là, mỗi dịch vụ cung cấp một giao diện (lập trình) được xác định rõ ràng. Trong

Trên thực tế, điều này cũng có nghĩa là mỗi dịch vụ đều cung cấp giao diện riêng, do đó, có thể khiến việc kết hợp các dịch vụ trở nên không hề đơn giản.

Kiến trúc dựa trên tài nguyên

Khi số lượng dịch vụ ngày càng tăng trên Web và việc phát triển các hệ thống phân tán thông qua thành phần dịch vụ trở nên quan trọng hơn, các nhà nghiên cứu bắt đầu xem xét lại kiến trúc của hầu hết các hệ thống phân tán dựa trên Web. Một trong những vấn đề với thành phần dịch vụ là việc kết nối các thành phần khác nhau có thể dễ dàng trở thành cơ sở phức tạp tích hợp.

Một cách khác, người ta cũng có thể xem hệ thống phân tán như một tập hợp lớn các tài nguyên được quản lý riêng lẻ bởi các thành phần. Tài nguyên có thể được thêm vào hoặc xóa đi bởi các ứng dụng (từ xa) và cũng có thể được truy xuất hoặc sửa đổi. Cách tiếp cận này hiện đã được áp dụng rộng rãi cho Web và được gọi là Chuyển trạng thái biểu diễn (REST) [Fielding, 2000]. Có bốn đặc điểm chính của những gì được gọi là kiến trúc RESTful [Pautasso và cộng sự, 2008]:

- 1. Tài nguyên được xác định thông qua một lược đồ đặt tên duy nhất
- 2. Tất cả các dịch vụ đều cung cấp cùng một giao diện, bao gồm tối đa bốn thao tác, như thể hiện trong Hình 2.7
- 3. Các tin nhắn được gửi đến hoặc đi từ một dịch vụ được tự mô tả đầy đủ
- 4. Sau khi thực hiện một thao tác tại một dịch vụ, thành phần đó sẽ quên mọi thứ về người gọi

Thuộc tính cuối cùng cũng được gọi là thực thi không trạng thái, một khái niệm mà chúng ta sẽ đề cập lại ở Chương 3.

Mô tả hoạt động	
ĐẶT	Sửa đổi một tài nguyên bằng cách chuyển một trạng thái mới
ĐƯA VÀO KIẾN	Tạo một nguồn tài nguyên mới
LẤY	Lấy lại trạng thái của một tài nguyên trong một số biểu diễn
XÓA BỎ	Xóa một tài nguyên

Hình 2.7: Bốn hoạt động có sẵn trong kiến trúc RESTful.

Để minh họa cách RESTful có thể hoạt động trong thực tế, hãy xem xét một dịch vụ lưu trữ đám mây, chẳng hạn như Simple Storage Service của Amazon (Amazon S3). Amazon S3, được mô tả trong [Murty, 2008] và gần đây hơn là trong [Culkin và Zazon, 2022], hỗ trợ hai tài nguyên: đối tượng, về cơ bản tương đương với tệp và thùng, tương đương với thư mục. Không có khái niệm đặt thùng vào thùng. Một đối tượng có tên ObjectName chứa trong thùng BucketName được tham chiếu bằng Mã định danh tài nguyên thống nhất (URI) sau:

`https://s3.amazonaws.com/BucketName/ObjectName`

Để tạo một thùng, hoặc một đối tượng, về cơ bản, một ứng dụng sẽ gửi yêu cầu PUT với URI của bucket/đối tượng. Về nguyên tắc, giao thức được sử dụng với dịch vụ là HTTP. Nói cách khác, nó chỉ là một HTTP khác yêu cầu, sau đó sẽ được S3 diễn giải chính xác. Nếu thùng

hoặc đối tượng đã tồn tại, thông báo lỗi HTTP sẽ được trả về.

Tương tự như vậy, để biết những đối tượng nào được chứa trong một thùng, một ứng dụng sẽ gửi yêu cầu GET với URI của thùng đó. S3 sẽ trả về danh sách

tên đối tượng, một lần nữa dư thừa dạng phản hồi HTTP thông thường.

Kiến trúc RESTful đã trở nên phổ biến vì tính đơn giản của nó.

Pautasso et al. [2008] đã so sánh các dịch vụ RESTful với các dịch vụ cụ thể giao diện, và như mong đợi, cả hai đều có ưu điểm và nhược điểm của chúng. Đặc biệt, tính đơn giản của kiến trúc RESTful có thể dễ dàng cấm các giải pháp dễ dàng cho các chương trình truyền thông phức tạp. Một ví dụ là nơi cần các giao dịch phân tán, thường yêu cầu các dịch vụ theo dõi trạng thái thực hiện.

Mặt khác, có nhiều

ví dụ trong đó kiến trúc RESTful hoàn toàn phù hợp với tích hợp đơn giản sơ đồ dịch vụ, nhưng vô số giao diện dịch vụ sẽ làm phức tạp vấn đề.

Lưu ý 2.2 (Nâng cao: Trên giao diện)

Rõ ràng, một dịch vụ không thể trở nên dễ dàng hơn hoặc khó khăn hơn chỉ vì giao diện cụ thể mà nó cung cấp. Một dịch vụ cung cấp chức năng và tốt nhất là cách rằng dịch vụ được truy cập được xác định bởi giao diện. Thật vậy, người ta có thể cho rằng cuộc thảo luận về RESTful so với giao diện dành riêng cho dịch vụ là nhiều về tính minh bạch của quyền truy cập. Để hiểu rõ hơn lý do tại sao rất nhiều người đang trả tiền chú ý đến vấn đề này, chúng ta hãy phóng to dịch vụ Amazon S3, cung cấp Giao diện REST cũng như giao diện truyền thống hơn (được gọi là SOAP). Thực tế là giao diện sau đã lỗi thời nói lên rất nhiều điều.

Hoạt động của thùng	Hoạt động đối tượng
Liệt kê tất cả các thùng của tôi	Đặt Đối Tượng Vào Dòng
TạoBucket	Đặt Đối Tượng
XóaBucket	Sao chép/Đổi tên
Danh sáchBucket	Lấy Đối Tượng
LấyBucketAccessControlPolicy	GetObjectExtended
ĐặtBucketAccessControlPolicy	DeleteObject
LấyBucketLoggingStatus	Lấy chính sách kiểm soát truy cập đối tượng
ĐặtBucketLoggingStatus	Đặt chính sách kiểm soát truy cập đối tượng

Hình 2.8: Các hoạt động trong giao diện SOAP S3 của Amazon, cho đến nay đã lỗi thời.

Giao diện SOAP bao gồm khoảng 16 thao tác, được liệt kê trong Hình 2.8. Tuy nhiên, nếu chúng ta truy cập Amazon S3 bằng thư viện Python boto3, chúng ta sẽ có hơn 100 hoạt động có sẵn. Ngược lại, giao diện REST

chỉ cung cấp rất ít thao tác, về cơ bản là những thao tác được liệt kê trong Hình 2.7. Những khác biệt này đến từ đâu? Câu trả lời tất nhiên là ở không gian tham số. Trong trường hợp kiến trúc RESTful, một ứng dụng sẽ cần cung cấp tất cả những gì nó muốn thông qua các tham số mà nó truyền qua một trong các thao tác. Trong giao diện SOAP của Amazon, số lượng tham số cho mỗi thao tác thường bị giới hạn và điều này chắc chắn đúng nếu chúng ta sử dụng thư viện Python boto3.

Dựa trên các nguyên tắc (để chúng ta có thể tránh những phức tạp của mã thực), giả sử chúng ta có một thùng giao diện cung cấp thao tác tạo, yêu cầu chuỗi đầu vào như mybucket, để tạo một thùng có tên "mybucket".

Thông thường, hoạt động này sẽ được gọi một cách đại khái như sau:

```
nhập bucket
bucket.create("mybucket")
```

Tuy nhiên, trong kiến trúc RESTful, lệnh gọi về cơ bản cần được mã hóa thành một chuỗi duy nhất, chẳng hạn như

```
ĐẶT "https://mybucket.s3.amazonaws.com/"
```

Sự khác biệt là rất rõ ràng. Ví dụ, trong trường hợp đầu tiên, nhiều lỗi cú pháp thường có thể đã được phát hiện trong thời gian biên dịch, trong khi trong trường hợp thứ hai, việc kiểm tra cần phải được hoãn lại cho đến khi chạy. Thứ hai, người ta có thể lập luận rằng việc chỉ định ngữ nghĩa của một hoạt động dễ dàng hơn nhiều với các giao diện cụ thể so với các giao diện chỉ cung cấp các hoạt động chung. Mặt khác, với các hoạt động chung, các thay đổi dễ dàng hơn nhiều để điều chỉnh, vì chúng thường liên quan đến việc thay đổi bố cục của các chuỗi mã hóa những gì thực sự cần thiết.

2.1.3 Kiến trúc xuất bản-đăng ký

Khi các hệ thống tiếp tục phát triển và các quy trình có thể dễ dàng tham gia hoặc rời đi hơn, điều quan trọng là phải có một kiến trúc trong đó các mối phụ thuộc giữa các quy trình trở nên lỏng lẻo nhất có thể. Một lớp lớn các hệ thống phân tán đã áp dụng một kiến trúc trong đó có sự tách biệt mạnh mẽ giữa xử lý và phối hợp. Ý tưởng là xem một hệ thống như một tập hợp các quy trình hoạt động tự chủ. Trong mô hình này, phối hợp bao gồm giao tiếp và hợp tác giữa các quy trình. Nó tạo thành chất keo gắn kết các hoạt động do các quy trình thực hiện thành một tổng thể [Gelernter và Carriero, 1992].

Cabri et al. [2000] cung cấp một phân loại các mô hình phối hợp có thể được áp dụng như nhau cho nhiều loại hệ thống phân tán. Điều chỉnh một chút thuật ngữ của họ, chúng tôi phân biệt giữa các mô hình theo hai chiều khác nhau, thời gian và tham chiếu, như thể hiện trong Hình 2.9.

Khi các tiến trình được kết hợp theo thời gian và tham chiếu, sự phối hợp diễn ra trực tiếp, được gọi là phối hợp trực tiếp. Sự kết hợp tham chiếu thường xuất hiện dưới dạng tham chiếu rõ ràng trong giao tiếp. Ví dụ, một tiến trình chỉ có thể giao tiếp nếu nó biết tên hoặc mã định danh của

	Tạm thời Tạm thời ghép nối tách rời	
Được ghép nối tham chiếu	Trực tiếp	Hộp thư
Tách rời tham chiếu	Sự kiện- dựa trên	Chia sẻ không gian dữ liệu

Hình 2.9: Ví dụ về các hình thức phối hợp khác nhau.

các quá trình khác mà nó muốn trao đổi thông tin. Ghép nối thời gian có nghĩa là các quy trình đang giao tiếp sẽ phải được thiết lập và chạy. Trong cuộc sống thực, nói chuyện qua điện thoại di động (và giả sử rằng điện thoại di động (chỉ có một chủ sở hữu) là một ví dụ về giao tiếp trực tiếp.

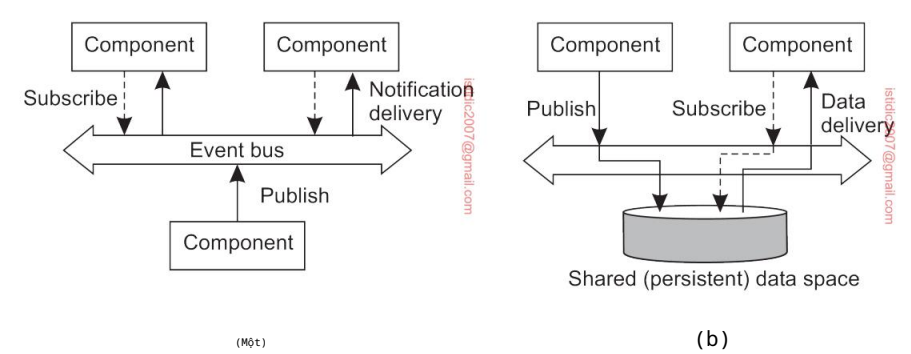
Một loại phối hợp khác xảy ra khi các quy trình được tách biệt về mặt thời gian nhưng được kết hợp về mặt tham chiếu, mà chúng tôi gọi là phối hợp hộp thư. Trong trường hợp này, không cần phải có hai tiến trình giao tiếp hoạt động đồng thời để cho sự giao tiếp diễn ra. Thay vào đó, giao tiếp diễn ra bằng cách đưa tin nhắn vào hộp thư (có thể được chia sẻ). Bởi vì nó là cần thiết để giải quyết rõ ràng hộp thư sẽ chứa các tin nhắn đó được trao đổi, có sự ghép nối tham chiếu.

Sự kết hợp của các hệ thống tách rời tham chiếu và kết nối thời gian tạo thành nhóm các mô hình để phối hợp dựa trên sự kiện. Trong tham chiếu các hệ thống tách biệt, các quy trình không biết nhau một cách rõ ràng. Chỉ điều mà một quá trình có thể làm là công bố một thông báo mô tả sự xuất hiện của một sự kiện (ví dụ, rằng nó muốn phối hợp các hoạt động, hoặc rằng nó chỉ tạo ra một số kết quả thú vị). Giả sử rằng thông báo xuất hiện dưới nhiều hình thức và kiểu khác nhau, các quy trình có thể đăng ký một loại thông báo cụ thể (xem thêm [Mühl et al., 2006]). Trong một mô hình phối hợp dựa trên sự kiện lý tưởng, một thông báo được công bố sẽ được chuyển giao chính xác đến những quy trình đã đăng ký. Tuy nhiên, nói chung là người đăng ký phải đang hoạt động tại thời điểm đó thông báo đã được công bố.

Một mô hình phối hợp nổi tiếng là sự kết hợp của tham chiếu và các quá trình tách biệt về mặt thời gian, dẫn đến cái được gọi là dữ liệu được chia sẻ không gian. Ý tưởng chính là các quy trình giao tiếp hoàn toàn thông qua các bộ dữ liệu, là những bản ghi dữ liệu có cấu trúc bao gồm một số trường, rất giống với hàng trong bảng cơ sở dữ liệu. Các quy trình có thể đưa bất kỳ loại tuple nào vào không gian dữ liệu. Để lấy một bộ, một quy trình cung cấp một mẫu tìm kiếm khớp với các bộ. Bất kỳ bộ nào khớp sẽ được trả về.

Do đó, không gian dữ liệu được chia sẻ được coi là thực hiện tìm kiếm liên kết cơ chế cho các bộ dữ liệu. Khi một quá trình muốn trích xuất một bộ dữ liệu từ dữ liệu không gian, nó chỉ định (một số) giá trị của các trường mà nó quan tâm. Bất kỳ bộ dữ liệu khớp với thông số kỹ thuật đó sau đó được xóa khỏi không gian dữ liệu và được chuyển đến quy trình.

Không gian dữ liệu dùng chung thường được kết hợp với sự phối hợp dựa trên sự kiện: một quy trình đăng ký một số bộ dữ liệu nhất định bằng cách cung cấp một mẫu tìm kiếm; khi một quy trình chen một bộ dữ liệu vào không gian dữ liệu, những người đăng ký phù hợp sẽ được thông báo. Trong cả hai trường hợp, chúng ta đang xử lý kiến trúc đăng ký-xuất bản và thực sự, đặc điểm chính là các quy trình không có tham chiếu rõ ràng nào đến nhau. Sự khác biệt giữa kiểu kiến trúc dựa trên sự kiện thuần túy và kiểu kiến trúc không gian dữ liệu dùng chung được thể hiện trong Hình 2.10. Chúng tôi cũng đã chỉ ra một sự trừu tượng của cơ chế mà theo đó các nhà xuất bản và người đăng ký được khớp, được gọi là bus sự kiện.



Hình 2.10: Phong cách kiến trúc không gian dữ liệu chia sẻ (b) dựa trên sự kiện.

Lưu ý 2.3 (Ví dụ: Không gian tuple Linda)

Để làm cho vấn đề cụ thể hơn một chút, chúng ta hãy xem xét kỹ hơn Linda, một mô hình lập trình được phát triển vào những năm 1980 [Carriero và Gelernter, 1989]. Không gian dữ liệu được chia sẻ trong Linda được gọi là không gian tuple, hỗ trợ ba phép toán:

- `in(t)`: xóa một tuple khớp với mẫu `t`
- `rd(t)`: lấy một bản sao của một tuple khớp với mẫu `t`
- `out(t)`: thêm tuple `t` vào không gian tuple

Lưu ý rằng nếu một tiến trình gọi `out(t)` hai lần liên tiếp, chúng ta sẽ thấy rằng hai bản sao của tuple `t` đã được lưu trữ. Do đó, về mặt hình thức, không gian tuple luôn được mô hình hóa như một đa tập. Cả `in` và `rd` đều là các hoạt động chặn: trình gọi sẽ bị chặn cho đến khi tìm thấy một tuple khớp hoặc đã khả dụng.

Hãy xem xét một ứng dụng microblog đơn giản trong đó các tin nhắn được gắn thẻ với tên của người đăng và một chủ đề, theo sau là một chuỗi ngắn. Mỗi tin nhắn được mô hình hóa như một bộ `<string,string,string>` trong đó chuỗi đầu tiên đặt tên cho người đăng, chuỗi thứ hai biểu diễn chủ đề và chuỗi thứ ba là nội dung thực tế. Giả sử rằng chúng ta đã tạo một không gian dữ liệu dùng chung có tên là MicroBlog, Hình 2.11 cho thấy cách Alice và Bob có thể đăng tin nhắn lên không gian đó và cách

Chuck có thể chọn một tin nhắn (được chọn ngẫu nhiên). Chúng tôi đã bỏ qua một số mã để rõ ràng hơn. Lưu ý rằng cả Alice và Bob đều không biết ai sẽ đọc bài đăng của họ.

```
1 blog = linda.universe._rd(("MicroBlog",linda.TupleSpace))[1]
2
3 blog._out(("bob","distsys","Tôi đang học chương 2")) 4
blog._out(("bob","distsys","Ví dụ về linda khá đơn giản")) 5 blog._out(("bob","gtcn","Cuốn
sách hay đấy!"))
```

(a) Mã của Bob để tạo một blog nhỏ và đăng ba tin nhắn.

```
1 blog = linda.universe._rd(("MicroBlog",linda.TupleSpace))[1]
2
3 blog._out(("alice","gtcn","Lý thuyết đồ thị này không dễ")) 4
blog._out(("alice","distsys","Tôi thích hệ thống hơn đồ thị"))
```

(b) Mã của Alice để tạo một blog nhỏ và đăng hai tin nhắn.

```
1 blog = linda.universe._rd(("MicroBlog",linda.TupleSpace))[1]
2
3 t1 = blog._rd(("bob","distsys",str)) 4 t2 =
blog._rd(("alice","gtcn",str)) 5 t3 =
blog._rd(("bob","gtcn",str))
```

(c) Chuck đang đọc tin nhắn từ blog nhỏ của Bob và Alice.

Hình 2.11: Một ví dụ đơn giản về việc sử dụng không gian dữ liệu được chia sẻ.

Trong dòng đầu tiên của mỗi đoạn mã, một tiến trình tra cứu không gian tuple có tên là "MicroBlog." Bob đăng ba tin nhắn: hai tin nhắn về chủ đề distsys và một tin nhắn về gtcn. Alice đăng hai tin nhắn, mỗi tin nhắn về một chủ đề. Cuối cùng, Chuck đọc ba tin nhắn: một tin nhắn từ Bob về distsys và một tin nhắn về gtcn và một tin nhắn từ Alice về gtcn.

Rõ ràng là vẫn còn nhiều chỗ để cải thiện. Ví dụ, chúng ta nên đảm bảo rằng Alice không thể đăng tin nhắn dưới tên của Bob. Tuy nhiên, vấn đề quan trọng cần lưu ý hiện nay là bằng cách chỉ cung cấp thẻ, người đọc như Chuck sẽ có thể nhận được tin nhắn mà không cần phải tham chiếu trực tiếp đến người đăng. Đặc biệt, Chuck cũng có thể đọc một tin nhắn được chọn ngẫu nhiên trên chủ đề distsys thông qua câu lệnh

```
t = blog._rd((chuỗi,"distsys",chuỗi))
```

Chúng tôi để lại bài tập cho người đọc để mở rộng các đoạn mã sao cho tin nhắn tiếp theo sẽ được chọn thay vì tin nhắn ngẫu nhiên.

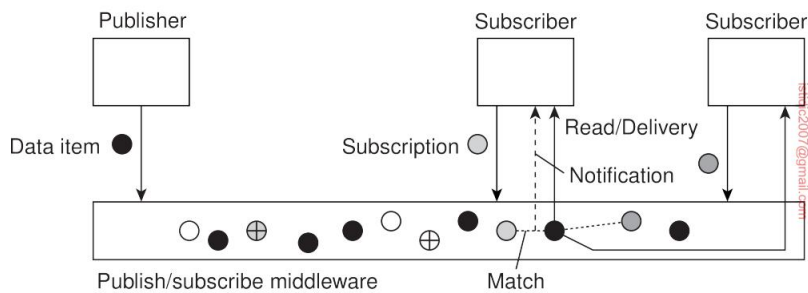
Một khía cạnh quan trọng của hệ thống đăng ký-xuất bản là giao tiếp diễn ra bằng cách mô tả các sự kiện mà người đăng ký quan tâm. Do đó, việc đặt tên đóng vai trò quan trọng. Chúng ta sẽ quay lại vấn đề đặt tên sau, nhưng hiện tại, vấn đề quan trọng là thư ờng thì các mục dữ liệu không được người gửi và người nhận xác định rõ ràng.

Trước tiên chúng ta hãy giả sử rằng các sự kiện được mô tả bằng một loạt các thuộc tính. Một thông báo mô tả một sự kiện được cho là đã được công bố khi nó được thực hiện

có sẵn cho các quy trình khác đọc. Để đạt được mục đích đó, cần phải chuyển một đăng ký đến phần mềm trung gian, chứa mô tả về sự kiện mà người đăng ký quan tâm. Mô tả như vậy thường bao gồm một số cặp (thuộc tính, giá trị), phổ biến đối với các hệ thống đăng ký-xuất bản dựa trên chủ đề.

Một giải pháp thay thế, trong các hệ thống đăng ký-xuất bản dựa trên nội dung, một đăng ký cũng có thể bao gồm các cặp (thuộc tính, phạm vi). Trong trường hợp này, thuộc tính được chỉ định dự kiến sẽ nhận các giá trị trong phạm vi được chỉ định. Đôi khi, các mô tả có thể được đưa ra bằng cách sử dụng tất cả các loại vị ngữ được xây dựng trên các thuộc tính, rất giống về bản chất với các truy vấn giống SQL trong trường hợp cơ sở dữ liệu quan hệ. Rõ ràng, mô tả càng được phép biểu đạt thì việc kiểm tra xem một sự kiện có khớp với mô tả hay không sẽ càng khó khăn.

Bây giờ chúng ta phải đối mặt với một tình huống trong đó các đăng ký cần được khớp với các thông báo, như thể hiện trong Hình 2.12. Thông thường, một sự kiện thực sự tương ứng với dữ liệu trở nên khả dụng. Trong trường hợp đó, khi khớp thành công, có hai kịch bản có thể xảy ra. Trong trường hợp đầu tiên, phần mềm trung gian có thể quyết định chuyển tiếp thông báo đã xuất bản, cùng với dữ liệu liên quan, đến tập hợp các đăng ký hiện tại của nó, tức là các quy trình có đăng ký khớp. Một cách khác, phần mềm trung gian cũng có thể chỉ chuyển tiếp một thông báo, tại thời điểm đó, các đăng ký có thể thực hiện thao tác đọc để truy xuất mục dữ liệu.



Hình 2.12: Nguyên tắc trao đổi dữ liệu giữa nhà xuất bản và người đăng ký.

Trong những trường hợp đó, dữ liệu liên quan đến một sự kiện được chuyển tiếp ngay lập tức đến người đăng ký, phần mềm trung gian thường sẽ không cung cấp dịch vụ lưu trữ dữ liệu. Việc lưu trữ được xử lý rõ ràng bởi một dịch vụ riêng biệt hoặc là trách nhiệm của người đăng ký. Nói cách khác, chúng ta có một hệ thống tách rời về mặt tham chiếu nhưng được kết nối về mặt thời gian.

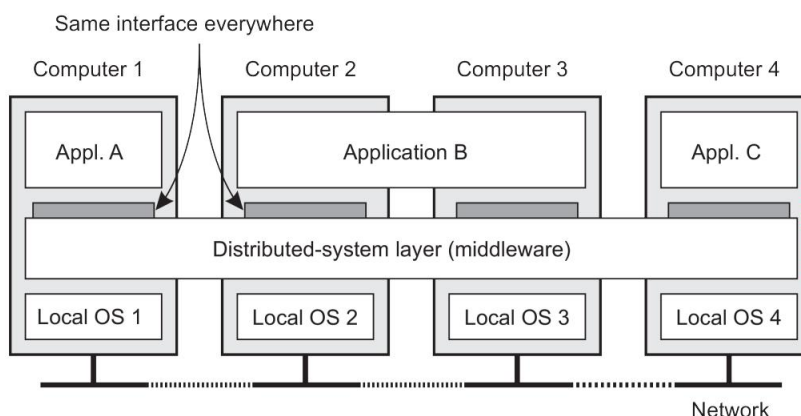
Tình huống này khác khi thông báo được gửi để người đăng ký cần phải đọc rõ ràng dữ liệu liên quan. Về cơ bản, phần mềm trung gian sẽ phải lưu trữ các mục dữ liệu. Trong những tình huống này, có các hoạt động bổ sung để quản lý dữ liệu. Cũng có thể đính kèm hợp đồng thuê vào một mục dữ liệu sao cho khi hợp đồng thuê hết hạn, mục dữ liệu sẽ tự động bị xóa.

Sự kiện có thể dễ dàng làm phức tạp quá trình xử lý đăng ký. Để minh họa, hãy xem xét một đăng ký như "thông báo khi phòng ZI.1060 không có người và cửa không khóa". Thông thư ởng, một hệ thống phân tán hỗ trợ các đăng ký như vậy có thể được triển khai bằng cách đặt các cảm biến độc lập để theo dõi tình trạng có người trong phòng (ví dụ: cảm biến chuyển động) và các cảm biến để ghi lại trạng thái khóa cửa. Theo cách tiếp cận đã thảo luận cho đến nay, chúng ta sẽ cần biên soạn các sự kiện nguyên thủy như vậy thành một mục dữ liệu có thể xuất bản, sau đó các quy trình có thể đăng ký. Biên soạn sự kiện hóa ra là một nhiệm vụ khó khăn, đặc biệt là khi các sự kiện nguyên thủy được tạo ra từ các nguồn phân tán trên toàn bộ hệ thống phân tán.

Rõ ràng, trong các hệ thống đăng ký-đăng ký như thế này, vấn đề cốt lõi là việc triển khai hiệu quả và có thể mở rộng của việc khớp các đăng ký với thông báo. Nhìn từ bên ngoài, kiến trúc đăng ký-đăng ký cung cấp nhiều tiềm năng để xây dựng các hệ thống phân tán quy mô rất lớn do sự tách biệt mạnh mẽ của các quy trình. Mặt khác, việc thiết kế các triển khai có thể mở rộng mà không mất đi tính độc lập này không phải là một bài tập đơn giản, đặc biệt là trong trường hợp của các hệ thống đăng ký-đăng ký dựa trên nội dung. Theo nghĩa này, mặc dù nhiều người cho rằng phong cách đăng ký-đăng ký cung cấp con đường hướng tới các kiến trúc có thể mở rộng, nhưng thực tế là các triển khai có thể dễ dàng tạo thành nút thắt cổ chai, chắc chắn là khi bảo mật và quyền riêng tư bị đe dọa, như chúng ta sẽ thảo luận trong Chương 9 và sau đó trong Chương 5.

2.2 Phần mềm trung gian và hệ thống phân tán

Để hỗ trợ phát triển các ứng dụng phân tán, các hệ thống phân tán thường được tổ chức để có một lớp phần mềm riêng biệt được đặt hợp lý trên các hệ điều hành tương ứng của các máy tính là một phần của



Hình 2.13: Một hệ thống phân tán được tổ chức trong một lớp phần mềm trung gian, mở rộng trên nhiều máy, cung cấp cho mỗi ứng dụng cùng một giao diện.

hệ thống. Tổ chức này được thể hiện trong Hình 2.13, dẫn đến cái được gọi là phần mềm trung gian [Bernstein, 1996].

Hình 2.13 cho thấy bốn máy tính được kết nối mạng và ba ứng dụng, trong đó ứng dụng B được phân phối trên máy tính 2 và 3. Mỗi ứng dụng được cung cấp cùng một giao diện. Hệ thống phân tán cung cấp sự thuận tiện cho các thành phần của một ứng dụng phân tán duy nhất để giao tiếp với nhau, nhưng cũng cho phép các ứng dụng khác nhau giao tiếp. Đồng thời, nó ẩn, tốt nhất và hợp lý nhất có thể, các khác biệt về phần cứng và hệ điều hành từ mỗi ứng dụng.

Theo một nghĩa nào đó, phần mềm trung gian đối với hệ thống phân tán cũng giống như hệ điều hành đối với máy tính: một trình quản lý tài nguyên cung cấp các ứng dụng để chia sẻ và triển khai hiệu quả các tài nguyên đó trên mạng. Bên cạnh việc quản lý tài nguyên, nó còn cung cấp các dịch vụ có thể tìm thấy trong hầu hết các hệ điều hành, bao gồm:

- Tiện ích cho giao tiếp giữa các ứng dụng.
- Dịch vụ bảo mật.
- Dịch vụ kế toán.
- Che giấu và phục hồi sau lỗi.

Sự khác biệt chính với các hệ điều hành thông thường của chúng là các dịch vụ trung gian được cung cấp trong môi trường mạng. Cũng lưu ý rằng hầu hết các dịch vụ đều hữu ích cho nhiều ứng dụng. Theo nghĩa này, trung gian cũng có thể được xem như một thùng chứa các thành phần và chức năng thường dùng mà giờ đây không còn phải được các ứng dụng triển khai riêng biệt nữa.

Ghi chú 2.4 (Ghi chú lịch sử: Thuật ngữ phần mềm trung gian)

Mặc dù thuật ngữ phần mềm trung gian trở nên phổ biến vào giữa những năm 1990, nhưng rất có thể nó được đề cập lần đầu tiên trong một báo cáo về hội nghị kỹ thuật phần mềm NATO, do Peter Naur và Brian Randell biên tập vào tháng 10 năm 1968 [Naur và Randell, 1968]. Thật vậy, phần mềm trung gian được đặt chính xác giữa các ứng dụng và các quy trình dịch vụ (thông thường với hệ điều hành).

2.2.1 Tổ chức phần mềm trung gian

Bây giờ chúng ta hãy phóng to vào tổ chức thực tế của phần mềm trung gian. Có hai loại mẫu thiết kế quan trọng thường được áp dụng cho tổ chức phần mềm trung gian: wrappers và interceptors. Mỗi loại nhắm đến các vấn đề khác nhau, nhưng giải quyết cùng một mục tiêu cho phần mềm trung gian: đặt được tính mở (như chúng ta đã thảo luận trong Phần 1.2.3).

Bao bì

Khi xây dựng một hệ thống phân tán từ các thành phần hiện có, chúng ta ngay lập tức gặp phải một vấn đề cơ bản: các giao diện do thành phần cũ cung cấp rất có thể không phù hợp với tất cả các ứng dụng. Trong Phần 1.3.2, chúng ta đã thảo luận về cách tích hợp ứng dụng doanh nghiệp có thể được thiết lập thông qua phần mềm trung gian như một trình tạo điều kiện giao tiếp, nhưng ở đó chúng ta vẫn ngầm cho rằng, cuối cùng, các thành phần có thể được truy cập thông qua giao diện gốc của chúng.

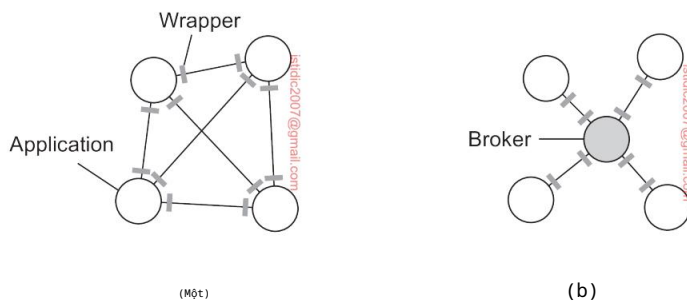
Wrapper hoặc adapter là một thành phần đặc biệt cung cấp giao diện được chấp nhận cho ứng dụng khách hàng, trong đó các chức năng được chuyển đổi thành các chức năng có sẵn tại thành phần. Về bản chất, nó giải quyết vấn đề về giao diện không tương thích (xem thêm Gamma et al. [1994]).

Mặc dù ban đầu được định nghĩa hẹp trong bối cảnh lập trình hướng đối tượng, trong bối cảnh của các hệ thống phân tán, các wrappers còn hơn cả các bộ chuyển đổi giao diện đơn giản. Ví dụ, một bộ điều hợp đối tượng là một thành phần cho phép các ứng dụng gọi các đối tượng từ xa, mặc dù các đối tượng đó có thể đã được triển khai dưới dạng kết hợp các hàm thư viện hoạt động trên các bảng của cơ sở dữ liệu quan hệ.

Một ví dụ khác, hãy xem xét lại dịch vụ lưu trữ S3 của Amazon. Như đã đề cập, có hai loại giao diện khả dụng, một loại tuân theo kiến trúc RESTful, loại còn lại tuân theo cách tiếp cận truyền thống hơn. Đối với giao diện RESTful, khách hàng sẽ sử dụng giao thức HTTP, về cơ bản là giao tiếp với máy chủ Web truyền thống hiện hoạt động như một bộ điều hợp với dịch vụ lưu trữ thực tế, bằng cách phân tích một phần các yêu cầu đến và sau đó chuyển chúng đến các máy chủ chuyên dụng bên trong S3.

Wrapper luôn đóng vai trò quan trọng trong việc mở rộng hệ thống bằng các thành phần hiện có. Khả năng mở rộng, yếu tố quan trọng để đạt được tính mở, thường được giải quyết bằng cách thêm wrapper khi cần. Nói cách khác, nếu ứng dụng A quản lý dữ liệu mà ứng dụng B cần, thì một cách tiếp cận là phát triển wrapper dành riêng cho B để ứng dụng này có thể truy cập vào dữ liệu của A. Rõ ràng, cách tiếp cận này không mở rộng tốt: với N ứng dụng, về mặt lý thuyết, chúng ta cần phát triển $N \times (N - 1) = O(N^2)$ giấy gói.

Một lần nữa, việc tạo điều kiện giảm số lượng wrapper thường được thực hiện thông qua phần mềm trung gian. Một cách để thực hiện điều này là triển khai cái gọi là broker, về mặt logic là một thành phần tập trung xử lý tất cả các truy cập giữa các ứng dụng khác nhau. Một loại thư ông được sử dụng là message broker mà chúng ta sẽ thảo luận về các vấn đề kỹ thuật trong Phần 4.3.3. Trong trường hợp của message broker, các ứng dụng chỉ cần gửi yêu cầu đến broker có chứa thông tin về những gì chúng cần. Broker, có kiến thức về tất cả các ứng dụng có liên quan, liên hệ với các ứng dụng phù hợp, có thể kết hợp và chuyển đổi các phản hồi và trả về kết quả cho ứng dụng ban đầu. Về nguyên tắc, vì broker cung cấp một giao diện duy nhất cho mỗi ứng dụng, chúng tôi



Hình 2.14: (a) Yêu cầu mỗi ứng dụng phải có một wrapper cho mỗi ứng dụng khác. (b) Giảm số lượng wrapper bằng cách sử dụng một broker.

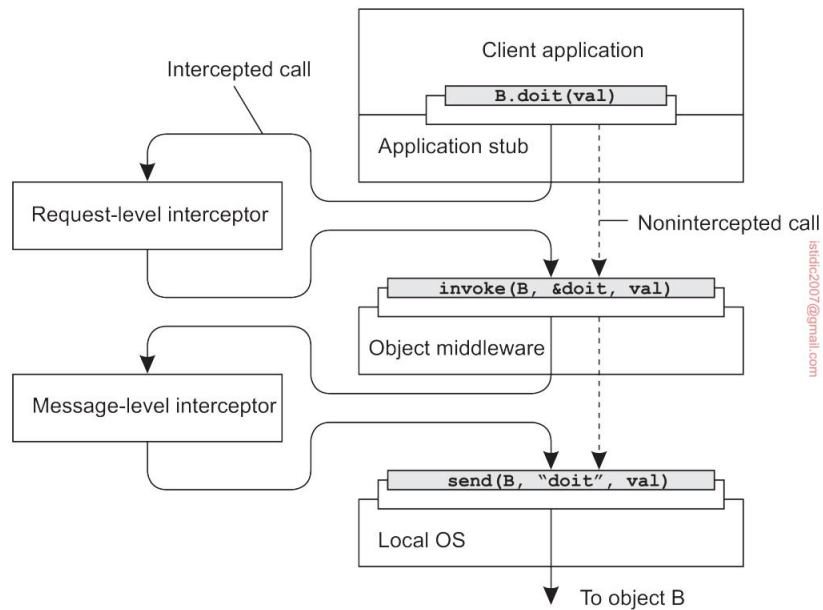
bây giờ cần nhiều nhất $2N = O(N)$ wrappers thay vì $O(N^2)$. Tình huống này được minh họa trong Hình 2.14.

Máy bay đánh chặn

Về mặt khái niệm, một interceptor không gì khác ngoài một cấu trúc phần mềm sẽ phá vỡ luồng điều khiển thông thường và cho phép thực thi mã khác (dành riêng cho ứng dụng). Interceptor là phương tiện chính để điều chỉnh phần mềm trung gian theo nhu cầu cụ thể của ứng dụng. Do đó, chúng đóng vai trò quan trọng trong việc làm cho phần mềm trung gian trở nên mở. Để làm cho interceptor trở nên chung chung có thể cần nỗ lực triển khai đáng kể, như được minh họa bởi Schmidt et al. [2000], và không rõ liệu trong những trường hợp như vậy, tính tổng quát có nên được ưu tiên hơn tính ứng dụng hạn chế và tính đơn giản hay không. Hơn nữa, thư ông chỉ có các tiện ích chặn giới hạn sẽ cải thiện việc quản lý phần mềm và toàn bộ hệ thống phân tán.

Để làm cho vấn đề cụ thể hơn, hãy xem xét việc chặn được hỗ trợ trong nhiều hệ thống phân tán dựa trên đối tượng. Ý tưởng cơ bản rất đơn giản: một đối tượng A có thể gọi một phương thức thuộc về một đối tượng B, trong khi đối tượng B nằm trên một máy khác với A. Như chúng tôi giải thích chi tiết sau trong sách, việc gọi đối tượng từ xa như vậy được thực hiện theo ba bước:

1. Đối tượng A được cung cấp một giao diện cục bộ giống với giao diện do đối tượng B cung cấp. A gọi phương thức có sẵn trong giao diện đó.
2. Cuộc gọi của A được chuyển đổi thành lệnh gọi đối tượng chung, có thể thực hiện thông qua giao diện gọi đối tượng chung do phần mềm trung gian cung cấp tại máy nơi A lưu trữ.
3. Cuối cùng, lệnh gọi đối tượng chung được chuyển thành một thông điệp được gửi qua giao diện mạng cấp độ vận chuyển do hệ điều hành cục bộ của A cung cấp.



Hình 2.15: Sử dụng bộ chặn để xử lý lệnh gọi đối tượng từ xa.

Sơ đồ này được thể hiện trong Hình 2.15. Sau bước đầu tiên, lệnh gọi `B.doit(val)` được chuyển thành một lệnh gọi chung, chẳng hạn như `invoke(B,&doit,val)` với tham chiếu đến phương thức của `B` và các tham số đi kèm với lệnh gọi. Bây giờ hãy tưởng tượng rằng đối tượng `B` được sao chép. Trong trường hợp đó, mỗi bản sao thực sự phải được gọi. Đây là một điểm rõ ràng mà việc chặn có thể giúp ích. Những gì trình chặn cấp yêu cầu sẽ thực hiện, chỉ cần gọi `invoke(B,&doit,val)` cho mỗi bản sao. Điểm tuyệt vời của tất cả những điều này là đối tượng `A` không cần phải biết về bản sao của `B`, như phần mềm trung gian đối tượng cũng không cần có các thành phần đặc biệt xử lý lệnh gọi được sao chép này. Chỉ có trình chặn cấp yêu cầu, có thể được thêm vào phần mềm trung gian, cần biết về bản sao của `B`.

Cuối cùng, một lệnh gọi đến một đối tượng từ xa sẽ phải được gửi qua mạng. Trên thực tế, điều này có nghĩa là giao diện nhắn tin do hệ điều hành cục bộ cung cấp sẽ cần phải được gọi. Ở cấp độ đó, một trình chặn cấp tin nhắn có thể hỗ trợ chuyển lệnh gọi đến đối tượng mục tiêu. Ví dụ, hãy tưởng tượng rằng tham số `val` thực sự tương ứng với một mảng dữ liệu lớn. Trong trường hợp đó, có thể khôn ngoan khi phân mảnh dữ liệu thành các phần nhỏ hơn để lắp ráp lại tại đích. Phân mảnh như vậy có thể cải thiện hiệu suất hoặc độ tin cậy. Một lần nữa, phần mềm trung gian không cần phải biết về phân mảnh này; trình chặn cấp thấp hơn sẽ xử lý mảnh bạch phần còn lại của giao tiếp với hệ điều hành cục bộ.

2.2.2 Phần mềm trung gian có thể sửa đổi

Những gì wrappers và interceptor cung cấp là phương tiện để mở rộng và điều chỉnh middleware. Nhu cầu điều chỉnh xuất phát từ thực tế là môi trường mà các ứng dụng phân tán được thực thi liên tục thay đổi. Những thay đổi bao gồm những thay đổi do tính di động, sự thay đổi lớn về chất lượng dịch vụ của mạng, phần cứng bị lỗi và hao pin, trong số những thay đổi khác.

Thay vì để các ứng dụng chịu trách nhiệm phản ứng với các thay đổi, nhiệm vụ này được đặt trong phần mềm trung gian. Hơn nữa, khi kích thước của hệ thống phân tán tăng lên, việc thay đổi các bộ phận của nó hiếm khi có thể được thực hiện bằng cách tạm thời tắt nó. Điều cần thiết là có thể thực hiện các thay đổi ngay lập tức.

Những ảnh hưởng mạnh mẽ này từ môi trường đã khiến nhiều nhà thiết kế phần mềm trung gian cân nhắc đến việc xây dựng phần mềm thích ứng. Chúng tôi theo Parlavantzas và Coulson [2007] khi nói về phần mềm trung gian có thể sửa đổi để diễn đạt rằng phần mềm trung gian không chỉ cần có khả năng thích ứng mà chúng ta còn có thể cố tình sửa đổi nó mà không làm nó ngừng hoạt động. Trong bối cảnh này, có thể coi các bộ chặn là phương tiện để thích ứng với luồng điều khiển chuẩn. Thay thế các thành phần phần mềm khi chạy là một ví dụ về việc sửa đổi hệ thống. Và thực sự, có lẽ một trong những cách tiếp cận phổ biến nhất đối với phần mềm trung gian có thể sửa đổi là xây dựng phần mềm trung gian động từ các thành phần.

Thiết kế dựa trên thành phần tập trung vào việc hỗ trợ khả năng sửa đổi thông qua thành phần. Một hệ thống có thể được cấu hình tĩnh tại thời điểm thiết kế hoặc động tại thời điểm chạy. Sau này yêu cầu hỗ trợ cho ràng buộc muộn, một kỹ thuật đã được áp dụng thành công trong môi trường ngôn ngữ lập trình, nhưng cũng cho các hệ điều hành nơi các mô-đun có thể được tải và dỡ tải theo ý muốn. Tự động lựa chọn triển khai tốt nhất của một thành phần trong thời gian chạy hiện đã được hiểu rõ [Yellin, 2003] nhưng một lần nữa, quá trình này vẫn phức tạp đối với các hệ thống phân tán, đặc biệt là khi xem xét rằng việc thay thế một thành phần đòi hỏi phải biết chính xác tác động của việc thay thế đó đối với các thành phần khác sẽ như thế nào. Thông thường, các thành phần ít độc lập hơn người ta vẫn nghĩ.

Điểm mấu chốt là để thích ứng với những thay đổi động đối với phần mềm tạo nên phần mềm trung gian, chúng ta cần ít nhất hỗ trợ cơ bản để tải và dỡ các thành phần khi chạy. Ngoài ra, đối với mỗi thành phần, cần có các thông số kỹ thuật rõ ràng về giao diện mà nó cung cấp cũng như các giao diện mà nó yêu cầu. Nếu trạng thái duy trì giữa các lần gọi đến một thành phần, thì cần có các biện pháp đặc biệt hơn nữa. Nhìn chung, rõ ràng là việc tổ chức phần mềm trung gian để có thể sửa đổi được đòi hỏi sự chú ý đặc biệt.

2.3 Kiến trúc hệ thống phân lớp

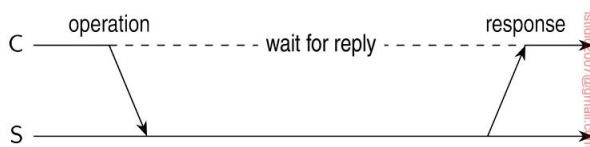
Bây giờ chúng ta hãy xem có bao nhiêu hệ thống phân tán thực sự được tổ chức bằng cách xem xét vị trí các thành phần phần mềm được đặt. Quyết định về phần mềm

các thành phần, sự tương tác của chúng và vị trí của chúng dẫn đến một trừu tượng hợp kiến trúc phần mềm, còn được gọi là kiến trúc hệ thống [Bass et al., 2021]. Chúng ta bắt đầu bằng việc thảo luận về kiến trúc phân lớp. Các dạng khác sẽ theo sau.

Mặc dù thiếu sự đồng thuận về nhiều vấn đề liên quan đến hệ thống phân tán, nhưng có một vấn đề mà nhiều nhà nghiên cứu và học viên đều đồng ý: suy nghĩ theo hướng khách hàng yêu cầu dịch vụ từ máy chủ giúp hiểu và quản lý tính phức tạp của hệ thống phân tán [Saltzer và Kaashoek, 2009]. Sau đây, trước tiên chúng ta sẽ xem xét một tổ chức nhiều lớp đơn giản, sau đó xem xét các tổ chức nhiều lớp.

2.3.1 Kiến trúc máy khách-máy chủ đơn giản

Trong mô hình máy khách-máy chủ cơ bản, các quy trình trong hệ thống phân tán được chia thành hai nhóm (có thể chồng chéo). Máy chủ là quy trình triển khai một dịch vụ cụ thể, ví dụ, dịch vụ hệ thống tệp hoặc dịch vụ cơ sở dữ liệu. Máy khách là quy trình yêu cầu dịch vụ từ máy chủ bằng cách gửi yêu cầu và sau đó chờ máy chủ phản hồi. Tương tác máy khách-máy chủ này, còn được gọi là hành vi yêu cầu-phản hồi, được thể hiện trong Hình 2.16.



Hình 2.16: Tương tác chung giữa máy khách C và máy chủ S. C gửi lệnh oper và chờ phản hồi từ S.

Giao tiếp giữa máy khách và máy chủ có thể được thực hiện bằng một giao thức không kết nối đơn giản khi mạng cơ sở khá đáng tin cậy, như trong nhiều mạng cục bộ. Trong những trường hợp này, khi máy khách yêu cầu một dịch vụ, nó chỉ cần đóng gói một thông báo cho máy chủ, xác định dịch vụ mà nó muốn, cùng với dữ liệu đầu vào cần thiết. Sau đó, thông báo được gửi đến máy chủ. Sau đó, máy chủ sẽ luôn chờ yêu cầu đến, xử lý yêu cầu đó và đóng gói kết quả vào một tin nhắn trả lời rồi gửi đến máy khách.

Sử dụng giao thức không kết nối có lợi thế rõ ràng là hiệu quả. Miễn là tin nhắn không bị mất hoặc bị hỏng, giao thức yêu cầu/phản hồi vừa phức tạp sẽ hoạt động tốt. Thật không may, việc khiến giao thức chống lại các lỗi truyền tải thỉnh thoảng không phải là điều dễ dàng. Điều duy nhất chúng ta có thể làm là có thể để máy khách gửi lại yêu cầu khi không có tin nhắn phản hồi nào đến. Tuy nhiên, vấn đề là máy khách không thể phát hiện xem tin nhắn yêu cầu ban đầu có bị mất hay không hoặc việc truyền trả lời có bị lỗi không. Nếu trả lời bị mất, thì việc gửi lại yêu cầu có thể dẫn đến việc thực thi thao tác hai lần. Nếu

hoạt động này giống như "chuyển 10.000 đô la từ tài khoản ngân hàng của tôi", thì rõ ràng, sẽ tốt hơn nếu chúng tôi chỉ cần báo cáo lỗi.

Mặt khác, nếu thao tác là "cho tôi biết tôi còn bao nhiêu tiền", thì việc gửi lại yêu cầu là hoàn toàn có thể chấp nhận được. Khi một thao tác có thể được lặp lại nhiều lần mà không gây hại, thì nó được gọi là idempotent.

Vì một số yêu cầu là idempotent và một số khác thì không, nên rõ ràng là không có giải pháp duy nhất nào để xử lý tin nhắn bị mất. Chúng tôi sẽ thảo luận chi tiết về cách xử lý lỗi truyền tải sang Mục 8.3.

Một giải pháp thay thế là nhiều hệ thống máy khách-máy chủ sử dụng giao thức hướng kết nối đáng tin cậy. Mặc dù giải pháp này không hoàn toàn phù hợp trong mạng cục bộ do hiệu suất tương đối thấp, nhưng nó hoạt động hoàn hảo trong các hệ thống diện rộng mà giao tiếp vốn không đáng tin cậy. Ví dụ, hầu như tất cả các giao thức ứng dụng Internet đều dựa trên kết nối TCP/IP đáng tin cậy. Trong trường hợp này, bất cứ khi nào máy khách yêu cầu một dịch vụ, trước tiên máy khách sẽ thiết lập kết nối với máy chủ trước khi gửi yêu cầu. Máy chủ thư ởng sử dụng cùng kết nối đó để gửi tin nhắn trả lời, sau đó kết nối sẽ bị hủy. Vấn đề có thể là việc thiết lập và hủy kết nối tương đương đối tốn kém, đặc biệt là khi tin nhắn yêu cầu và trả lời nhỏ.

Mô hình máy khách-máy chủ đã là chủ đề của nhiều cuộc tranh luận và tranh cãi trong nhiều năm. Một trong những vấn đề chính là làm thế nào để phân biệt rõ ràng giữa máy khách và máy chủ. Không có gì ngạc nhiên khi thư ởng không có sự phân biệt rõ ràng. Ví dụ, một máy chủ cho cơ sở dữ liệu phân tán có thể liên tục hoạt động như một máy khách vì nó đang chuyển tiếp các yêu cầu đến các máy chủ tệp khác nhau chịu trách nhiệm triển khai các bảng cơ sở dữ liệu. Trong trường hợp như vậy, bản thân máy chủ cơ sở dữ liệu chỉ xử lý các truy vấn.

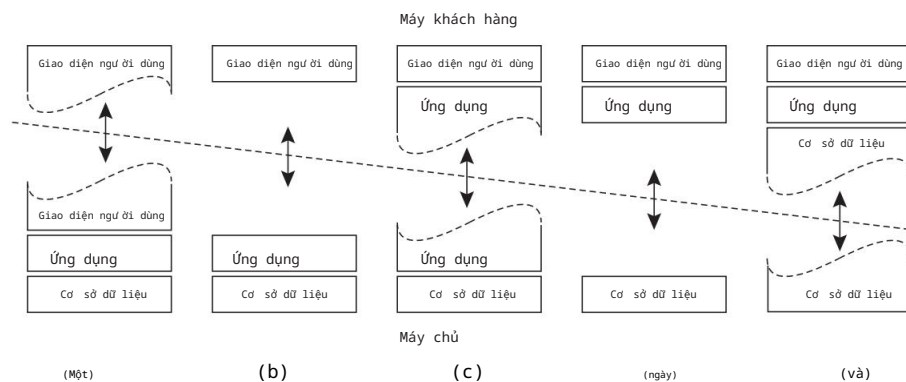
2.3.2 Kiến trúc đa tầng

Sự phân biệt thành ba cấp độ logic, như đã thảo luận cho đến nay, gợi ý một số khả năng phân phối vật lý một ứng dụng máy khách-máy chủ trên nhiều máy. Tổ chức đơn giản nhất là chỉ có hai loại máy:

1. Một máy khách chỉ chứa các chương trình thực hiện (một phần) cấp độ giao diện người dùng
2. Một máy chủ chứa phần còn lại, tức là các chương trình thực hiện xử lý và cấp dữ liệu

Trong tổ chức này, mọi thứ đều được máy chủ xử lý trong khi máy khách về cơ bản không hơn gì một thiết bị đầu cuối câm, có thể chỉ có giao diện đồ họa tiện lợi. Tuy nhiên, còn có nhiều khả năng khác. Như đã giải thích trong Phần 2.1.1, nhiều ứng dụng phân tán được chia thành ba lớp: (1) lớp giao diện người dùng, (2) lớp xử lý và (3) lớp dữ liệu. Một

cách tiếp cận để tổ chức máy khách và máy chủ sau đó là phân phối các lớp này trên các máy khác nhau, như thể hiện trong Hình 2.17 (xem thêm Umar [1997]). Bước đầu tiên, chúng ta chỉ phân biệt hai loại máy: máy khách và máy chủ, dẫn đến cái mà người ta còn gọi là (về mặt vật lý) là kiến trúc hai tầng.



Hình 2.17: Tổ chức máy khách-máy chủ trong kiến trúc hai tầng.

Một tổ chức có thể là chỉ có phần phụ thuộc vào thiết bị đầu cuối của giao diện người dùng trên máy khách, như thể hiện trong Hình 2.17(a), và cung cấp cho các ứng dụng quyền điều khiển từ xa đối với việc trình bày dữ liệu của chúng. Một giải pháp thay thế là đặt toàn bộ phần mềm giao diện người dùng ở phía máy khách, như được thể hiện trong Hình 2.17(b). Trong những trường hợp như vậy, về cơ bản chúng tôi chia ứng dụng vào giao diện đồ họa, giao tiếp với phần còn lại của ứng dụng (trú tại máy chủ) thông qua một giao thức ứng dụng cụ thể. Trong mô hình này, phần đầu (phần mềm máy khách) không thực hiện bất kỳ xử lý nào khác ngoài những xử lý cần thiết trình bày giao diện của ứng dụng.

Tiếp tục theo dòng lý luận này, chúng ta cũng có thể di chuyển một phần của ứng dụng vào đầu trung, như thể hiện trong Hình 2.17(c). Một ví dụ trong đó điều này có ý nghĩa là nơi ứng dụng sử dụng một biểu mẫu cần được điền đầy đủ trước khi có thể xử lý. Sau đó, phần đầu có thể kiểm tra tính chính xác và tính nhất quán của hình thức, và khi cần thiết tương tác với người dùng. Một ví dụ khác về tổ chức của Hình 2.17(c), là một trình xử lý văn bản trong đó các chức năng chỉnh sửa cơ bản được thực hiện trên máy khách phía mà chúng hoạt động trên dữ liệu được lưu trữ cục bộ hoặc trong bộ nhớ, như nơi mà các công cụ hỗ trợ nâng cao như kiểm tra chính tả và ngữ pháp thực thi ở phía máy chủ.

Trong nhiều môi trường máy khách-máy chủ, các tổ chức được hiển thị trong Hình 2.17 (d) và Hình 2.17(e) đặc biệt phổ biến. Các tổ chức này được sử dụng khi máy khách là PC hoặc máy trạm, được kết nối thông qua một mạng lưới đến một hệ thống tập tin phân tán hoặc cơ sở dữ liệu. Về cơ bản, hầu hết

ứng dụng đang chạy trên máy khách, nhưng tất cả các hoạt động trên tệp hoặc mục nhập cơ sở dữ liệu đều được chuyển đến máy chủ. Ví dụ, nhiều ứng dụng ngân hàng chạy trên máy của người dùng cuối, nơi người dùng chuẩn bị giao dịch và các hoạt động tương tự. Sau khi hoàn tất, ứng dụng sẽ liên hệ với cơ sở dữ liệu trên máy chủ của ngân hàng và tải các giao dịch lên để xử lý thêm. Hình 2.17(e) thể hiện tình huống mà đĩa cục bộ của máy khách chứa một phần dữ liệu. Ví dụ, khi duyệt Web, máy khách có thể dần dần xây dựng một bộ nhớ đệm lớn trên đĩa cục bộ của các trang Web được kiểm tra gần đây nhất.

Lưu ý 2.5 (Thông tin thêm: Có cách tổ chức nào là tốt nhất không?)

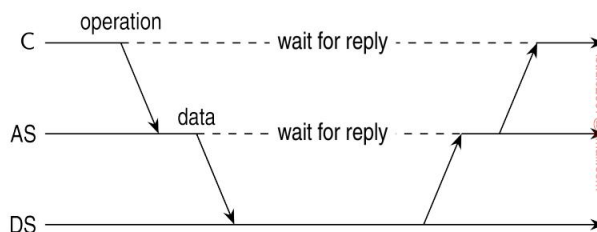
Chúng tôi lưu ý rằng có một xu hướng mạnh mẽ là tránh xa các cấu hình được hiển thị trong Hình 2.17(d) và Hình 2.17(e) trong những trường hợp đó, phần mềm máy khách được đặt tại máy người dùng cuối. Thay vào đó, hầu hết quá trình xử lý và lưu trữ dữ liệu được xử lý ở phía máy chủ. Lý do cho điều này rất đơn giản: mặc dù máy khách thực hiện được nhiều việc, nhưng chúng cũng khó quản lý hơn. Có nhiều chức năng hơn trên máy khách có nghĩa là nhiều người dùng cuối sẽ cần có khả năng xử lý phần mềm đó. Điều này ngụ ý rằng cần phải nỗ lực nhiều hơn để làm cho phần mềm có khả năng phục hồi trước hành vi của người dùng cuối. Ngoài ra, phần mềm phía máy khách phụ thuộc vào nền tảng cơ bản của máy khách (tức là hệ điều hành và tài nguyên), điều này có thể dễ dàng có nghĩa là nhiều phiên bản sẽ cần được duy trì. Theo quan điểm quản lý hệ thống, việc có những gì được gọi là máy khách béo không phải là tối ưu. Thay vào đó, máy khách mỏng như được thể hiện bởi các tổ chức được hiển thị trong Hình 2.17(a)-(c) dễ dàng hơn nhiều, có lẽ là phải đánh đổi bằng giao diện người dùng kém tinh vi hơn và hiệu suất mà máy khách cảm nhận được.

Điều này có nghĩa là sự kết thúc của fat client không? Không phải chút nào. Trước hết, có nhiều ứng dụng mà tổ chức fat client thường vẫn là tốt nhất. Chúng tôi đã đề cập đến các bộ ứng dụng văn phòng, nhưng nhiều ứng dụng đa phương tiện cũng yêu cầu xử lý được thực hiện ở phía máy khách. Hơn nữa, khi người dùng cuối cần hoạt động ngoại tuyến, chúng tôi thấy rằng việc cài đặt các ứng dụng là cần thiết. Thứ hai, với sự ra đời của công nghệ duyệt Web tiên tiến, giờ đây việc đặt và quản lý phần mềm phía máy khách một cách động dễ dàng hơn nhiều chỉ bằng cách tải các tập lệnh (đôi khi rất phức tạp) lên máy khách. Kết hợp với thực tế là loại phần mềm phía máy khách này chạy trong các môi trường được triển khai chung được xác định rõ ràng và do đó sự phụ thuộc vào nền tảng ít là vấn đề hơn nhiều, chúng tôi thấy rằng phản biện về sự phức tạp của quản lý thư viện không còn hợp lệ nữa. Điều này đã dẫn đến việc triển khai các môi trường máy tính để bàn ảo, mà chúng tôi sẽ thảo luận thêm trong Chương 3.

Cuối cùng, lưu ý rằng việc chuyển từ máy khách béo không có nghĩa là chúng ta không còn cần hệ thống phân tán nữa. Ngược lại, những gì chúng ta tiếp tục thấy là các giải pháp phía máy chủ đang ngày càng phân tán hơn khi một máy chủ duy nhất đang được thay thế bằng nhiều máy chủ chạy trên các máy khác nhau. Điện toán đám mây là một ví dụ điển hình trong trường hợp này: toàn bộ phía máy chủ đang được thực hiện trong các trung tâm dữ liệu và nói chung là trên nhiều máy chủ.

Khi chỉ phân biệt máy khách và máy chủ như chúng ta đã làm cho đến nay, chúng ta bỏ lỡ điểm là đôi khi máy chủ có thể cần hoạt động như máy khách, như đã trình bày

trong Hình 2.18, dẫn đến kiến trúc ba tầng (về mặt vật lý).



Hình 2.18: Ví dụ về máy chủ ứng dụng AS hoạt động như máy khách cho máy chủ cơ sở dữ liệu DS.

Trong kiến trúc này, theo truyền thống, các chương trình tạo thành một phần của lớp xử lý được thực hiện bởi một máy chủ riêng biệt, nhưng cũng có thể được phân phối một phần trên các máy khách và máy chủ. Một ví dụ điển hình về nơi kiến trúc ba tầng được sử dụng là trong xử lý giao dịch. Một quy trình riêng biệt, được gọi là giám sát xử lý giao dịch, điều phối tất cả các giao dịch trên các máy chủ dữ liệu có thể khác nhau.

Một ví dụ khác, nhưng rất khác, trong đó chúng ta thường thấy kiến trúc ba tầng là trong tổ chức của các Trang web. Trong trường hợp này, máy chủ Web đóng vai trò là điểm vào của một trang web, chuyển các yêu cầu đến máy chủ ứng dụng nơi diễn ra quá trình xử lý thực tế. Đến lượt mình, máy chủ ứng dụng này tương tác với máy chủ cơ sở dữ liệu. Chúng ta đã bắt gặp một tổ chức như vậy khi thảo luận về Trang web của cuốn sách này và các tiện ích để tạo và tải xuống bản sao được cá nhân hóa.

2.3.3 Ví dụ: Hệ thống tập tin mạng

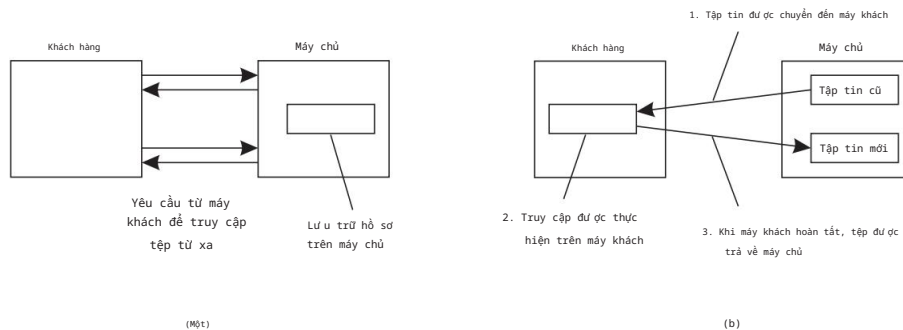
Nhiều hệ thống tệp phân tán được tổ chức giống như kiến trúc máy khách-máy chủ, trong đó Hệ thống tệp mạng (NFS) của Sun Microsystems là một trong những hệ thống được triển khai rộng rãi nhất cho các hệ thống Unix [Callaghan, 2000; Haynes, 2015; Noveck và Lever, 2020].

Ý tưởng cơ bản đằng sau NFS là mỗi máy chủ tệp cung cấp chế độ xem chuẩn hóa của hệ thống tệp cục bộ của nó. Nói cách khác, không quan trọng hệ thống tệp cục bộ đó được triển khai như thế nào; mỗi máy chủ NFS đều hỗ trợ cùng một mô hình. Cách tiếp cận này cũng đã được áp dụng cho các hệ thống tệp phân tán khác. NFS đi kèm với một giao thức truyền thông cho phép máy khách truy cập vào các tệp được lưu trữ trên máy chủ, do đó cho phép một tập hợp các quy trình không đồng nhất, có thể chạy trên các hệ điều hành và máy khác nhau, chia sẻ một hệ thống tệp chung.

Mô hình cơ bản của NFS và các hệ thống tương tự là mô hình dịch vụ tệp từ xa. Trong mô hình này, khách hàng được cung cấp quyền truy cập minh bạch vào hệ thống tệp được quản lý bởi máy chủ từ xa. Tuy nhiên, khách hàng thường không biết

của vị trí thực tế của các tệp. Thay vào đó, chúng được cung cấp một giao diện cho hệ thống tệp tương tự như giao diện do hệ thống tệp cục bộ thông thường cung cấp. Cụ thể, máy khách chỉ được cung cấp một giao diện chứa nhiều thao tác tệp khác nhau, nhưng máy chủ chịu trách nhiệm triển khai các thao tác đó.

Do đó, mô hình này cũng được gọi là mô hình truy cập từ xa. Nó được thể hiện trong Hình 2.19(a).



Hình 2.19: (a) Mô hình truy cập từ xa. (b) Mô hình tải lên/tải xuống.

Ngược lại, trong mô hình tải lên/tải xuống, máy khách truy cập tệp cục bộ sau khi tải xuống từ máy chủ, như thể hiện trong Hình 2.19(b). Khi máy khách hoàn tất tệp, tệp sẽ được tải lên lại máy chủ để máy khách khác có thể sử dụng. Dịch vụ FTP của Internet có thể được sử dụng theo cách này khi máy khách tải xuống toàn bộ tệp, sửa đổi tệp và sau đó đặt lại.

NFS đã được triển khai cho nhiều hệ điều hành, mặc dù các phiên bản Unix chiếm ưu thế. Đối với hầu hết các hệ thống Unix hiện đại, NFS thường được triển khai theo kiến trúc được thể hiện trong Hình 2.20.

Một máy khách truy cập hệ thống tệp bằng cách sử dụng các lệnh gọi hệ thống do hệ điều hành cục bộ của nó cung cấp. Tuy nhiên, giao diện hệ thống tệp Unix cục bộ được thay thế bằng giao diện cho Hệ thống tệp ảo (VFS), hiện là tiêu chuẩn thực tế để giao tiếp với các hệ thống tệp (phân tán) khác nhau [Kleiman, 1986].

Hầu như tất cả các hệ điều hành hiện đại đều cung cấp VFS và việc không làm như vậy ít nhiều buộc các nhà phát triển phải triển khai lại phần lớn các phần lớn của hệ điều hành khi áp dụng cấu trúc hệ thống tệp mới. Với NFS, các hoạt động trên giao diện VFS được chuyển đến hệ thống tệp cục bộ hoặc chuyển đến một thành phần riêng biệt được gọi là máy khách NFS, thành phần này đảm nhiệm việc xử lý quyền truy cập vào các tệp được lưu trữ tại máy chủ từ xa. Trong NFS, tất cả giao tiếp giữa máy khách và máy chủ đều được thực hiện thông qua cái gọi là các cuộc gọi thủ tục từ xa (RPC). Như đã đề cập trước đó, RPC về cơ bản là một cách chuẩn hóa để cho phép máy khách trên máy A thực hiện một cuộc gọi thông thường đến một thủ tục được triển khai trên máy khác B. Chúng tôi thảo luận chi tiết về RPC trong Chương 4. Máy khách NFS triển khai các hoạt động của hệ thống tệp NFS dưới dạng các cuộc gọi thủ tục từ xa đến máy chủ. Lưu ý rằng các hoạt động do giao diện VFS cung cấp có thể khác với các hoạt động do